

第 30 章: Operator Overloading (运算符重载)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 本章是"类机制深度巡礼"的延续。前几章我们已经摸过运算符重载的皮毛 (init、add), 本章正式把特殊方法 (special method) 这套"钩子"体系完整铺开: 索引、切片、迭代、属性访问、字符串显示、右端/就地运算、调用、比较、布尔测试、析构。学完本章, 你就掌握了让自定义类"长得像内建类型"的全部核心魔法。这是理解后续装饰器 (第 32 章)、描述符 (第 38 章)、元类 (第 40 章) 的必备地基。

30.1 开篇引言 (Introduction)

英文原文

This chapter continues our in-depth survey of class mechanics by focusing on operator overloading. We looked briefly at operator overloading in prior chapters. Here, we'll fill in more details and explore a handful of commonly used overloading methods, most of which we haven't yet encountered. Although we don't have space to demonstrate each of the many operator-overloading methods available, those we will code here

re are a representative sample large enough to uncover the possibilities of this Python class feature.

中文翻译

本章继续我们对类机制 (class mechanics) 的深入考察，主题是运算符重载 (operator overloading)。在前几章中我们已经简要看过运算符重载。本章将补充更多细节，并探索一批常用的重载方法，其中大部分我们之前还没有遇到过。虽然我们没有篇幅逐一演示所有可用的运算符重载方法，但本章将要编码的是一组代表性样本，足以揭示 Python 这一类特性的全部可能。

深度理解

- 核心概念：运算符重载 = 让自定义类拦截内建运算。Python 规定了一整套以双下划线开头和结尾的特殊方法名 (xxx)，当类的实例出现在对应的内建操作中时，Python 自动调用这些方法，方法返回值就是该操作的结果。
- 为什么这样设计：Python 的哲学是“一切皆对象，且行为一致”。如果 +、len()、for、[] 这些操作只能用于内建类型，自定义类永远是“二等公民”。重载机制让自定义类也能获得与内建类型一致的接口，从而与更多现成代码兼容。
- 生活例子：运算符重载就像酒店的“通用插座转换器”——所有电器（内建类型）都用一个插头标准，你的新电器（自定义类）只要装上转换器 (xxx 方法)，就能插进同一个墙插 (+、[]、len() 等操作)。

·初学者误区：以为重载意味着"改写运算符的行为"或需要特殊语法。其实你只是写普通的方法，名字特殊而已；触发它们是 Python 解释器在编译字节码阶段就做好的静态分派，不是运行时"魔法"。

30.2 基础：什么是运算符重载 (The Basics)

英文原文

Really "operator overloading" simply means intercepting built-in operations in a class's methods—Python automatically invokes your methods when instances of the class appear in built-in operations, and your method's return value becomes the result of the corresponding operation. Here's a review of the key ideas behind overloading:- Operator overloading lets classes intercept normal Python operations.- Classes can overload all Python built-in expression operators.- Classes can also overload other built-in operations such as printing, function calls, and attribute access.- Overloading is implemented by providing specially named methods in a class.- Python predefines the special method names that correspond to built-in operations. In other words, when methods of predefined special names are provided in a class, Python automatically calls them when instances of the class appear in their associated built-in operations or expressions. Your class provides the behavior of the corresponding operation

n for instance objects created from it. As you've learned, operator-overloading methods are never required and generally don't have defaults (apart from a handful that all classes get from the implied object root class). If you don't code or inherit an overloading method, it just means that your class does not support the corresponding operation. When used, though, these methods allow classes to emulate the interfaces of built-in objects, which makes them consistent, and compatible with more code.

中文翻译

实际上, "运算符重载"这个词的意思很简单: 在类的方法中拦截内建操作——当类的实例出现在内建操作中时, Python 会自动调用你的方法, 你的方法的返回值就是相应操作的结果。以下是重载背后关键思想的回顾: - 运算符重载让类能够拦截普通的 Python 操作。- 类可以重载所有 Python 内建的表达式运算符。- 类也可以重载其他内建操作, 比如打印、函数调用和属性访问。- 重载通过在类中提供特殊命名的方法来实现。- Python 预定义了与内建操作对应的特殊方法名。换句话说, 当类中提供了这些预定义特殊名字的方法时, 只要类的实例出现在与之关联的内建操作或表达式中, Python 就会自动调用它们。你的类为从它创建的实例对象提供相应操作的行为。正如你已经学到的, 运算符重载方法从来不是必需的, 通常也没有默认行为 (除了少数几个所有类都从隐含的 object 根类那里继承来的)。如果你没有编写或继承某个重载方法, 那只是意味着你的类不支持对应的操作。而一旦使用, 这些方法就能让类模拟内建对象的

接口，使类与内建对象保持一致，并与更多代码兼容。

深度理解

- 核心概念：重载不是"发明新运算符"，而是"接管已有运算符在自己类型上的语义"。+ 永远叫 +，但 + 作用于你的类实例时执行什么逻辑，由 add 决定。
- 底层实现：CPython 编译 $X + Y$ 时，会生成 BINARYOP + 字节码；虚拟机执行时按"左侧操作数 → 右侧操作数"的顺序查找 add/radd。这是解释器内置的分派 (dispatch) 机制——方法名与运算符的映射关系由语言定义死，与你的方法体写了什么无关。
- 设计原因：所有重载方法名用双下划线包裹，是为了避免与你自己的普通方法名冲突——一个"见不得人"的命名约定，让普通方法不会误触引擎内部接口。
- 解决什么实际问题：没有重载，`len(myobj)`、`myobj[3]`、`for x in myobj` 都无法用于自定义类，所有自定义类型只能"各自为政"。重载让类可以融入语言生态：交给 `sorted()`、`in`、`format()` 等通用工具直接处理。
- 初学者误区：①以为重载方法必须有默认值——其实绝大多数没有，缺了就是不支持，用了就报 `TypeError`；②以为打印总有默认行为——确实有 (object 根类提供的内存地址显示)，但那通常不是你想要的样子；③把 `init` 当构造函数——它其实是初始化器，真正的构造是 `new` (见下节 NOTE)。

30.3 构造器与表达式: `init` 和 `sub` (Constructors and Expressions)

英文原文

As a warm-up, consider the simple class in Example 30-1: its `Number` class, coded in module file `number.py`, provides a method to intercept instance construction (`init`), as well as one for catching subtraction expressions (`sub`). Special methods like these are the hooks that let you tie into built-in operations. Example 30-1. `number.py` As we've already learned, the `init` constructor method seen in this code is the most commonly used operator-overloading method in Python; it's present in most classes and used to initialize the newly created instance object using any arguments passed to the class name. The `sub` method plays the binary-operator role that `add` did in Chapter 27's introduction, intercepting subtraction expressions and returning a new instance of the class as its result (and running `init` along the way). We've already studied `init` and basic binary operators like `sub` in some depth, so we won't rehash their usage further here. In this chapter, we will tour some other tools available in this domain and look at example code that applies them in common use cases. NOTE — Construction convolution: Technically, instance creation first triggers the new method, which creates and returns the new instance object.

t, which is then passed into `init` for initialization. Since `new` has a built-in implementation and is redefined in only very limited roles, though, nearly all Python classes initialize by defining an `init` method. We'll explore one use case for `new` when we study metaclasses in Chapter 40; though rare, it is sometimes also used to customize creation of instances of immutable types.

中文翻译

作为热身，请看 Example 30-1 中那个简单的类：它的 `Number` 类写在模块文件 `number.py` 中，提供了一个拦截实例构造的方法（`init`），以及一个捕获减法表达式的方法（`sub`）。像这样的特殊方法就是让你接入内建操作的钩子（hook）。Example 30-1. `number.py` 我们已经知道，这段代码中的 `init` 构造器方法是 Python 中最常用的运算符重载方法；它出现在大多数类中，用于利用传给类名的任何参数来初始化新建的实例对象。`sub` 方法扮演二元运算符的角色，正如第 27 章引言中 `add` 所扮演的那样——拦截减法表达式，并返回该类的一个新实例作为结果（过程中还会运行 `init`）。我们已经在相当深度上学习过 `init` 和 `sub` 这类基础二元运算符，所以这里不再赘述它们的用法。在本章中，我们将游览这一领域的其他工具，并看看把这些工具用在常见场景中的示例代码。注——构造的“缠绕”（Construction convolution）：严格来说，实例创建首先触发 `new` 方法，它创建并返回新的实例对象，然后这个对象被传入 `init` 进行初始化。不过，由于 `new` 有内建实现、只在极少数特殊角色中才会被重定义，几乎所有 Python 类都是通过定义

init 方法来初始化的。我们将在第 40 章研究元类 (metaclass) 时探索 new 的一个用例；尽管罕见，它有时也用于定制不可变类型实例的创建。

代码分析 (Example 30-1. number.py)

```
class Number:
    def __init__(self, start):          # Number(start)
        self.data = start              #
    def __sub__(self, other):          # - other
        return Number(self.data - other) #

>>> from number import Number        #
>>> X = Number(5)                     # Number.__init__(X, 5)
>>> Y = X - 2                         # Number.__sub__(X, 2)
>>> Y.data                            # Y Number
3
```

·解释器如何处理：执行 Number(5) 时，字节码先是 CALLFUNCTION 前的取类名操作，Python 虚拟机走"构造管线"：先调 Number.new(Number, 5) 分配实例对象 X（此时 X.dict 为空），再把 X 作为第一个参数调用 Number.init(X, 5) 填上 data 属性。执行 X - 2 时，字节码为 BINARYOP -，解释器在 X 的类型上查到 sub，调用 Number.sub(X, 2)，其返回值 Number(3) 再次走完完整构造管线——所以"减法产生新实例"的同时，init 也被顺带运行了。

·设计思想：不可变风格的算术运算（返回新对象、不修改原对象）是 Python 算术运算符的默认约定，这也是为什么第 27 章、第 31 章反复强调"+ 应返回新实例而不是就地修改"。

深度理解

·核心概念：init 是初始化器，负责"填表"；new 才是真正的构造器，负责"造物"。绝大多数类只需重定义 init。

·底层实现：Type.call（元类层面的调用钩子）内部执行 new = cls.new(cls, ...)，若 new 返回 cls 的实例，再执行 cls.init(instance, ...)。new 默认从 object 继承，普通类不需要碰它；只有不可变类型（如 int 子类）或元类场景才需要重定义。

·为什么这样设计：把"分配内存"与"初始化状态"分开，让不可变类型能在 new 中直接返回缓存对象（如 int 的小整数缓存）、让元类能在 init 之前干预实例的创建。

·初学者误区：很多人写 def init(self): 后抱怨"对象没被创建出来"——其实对象早就被 new 造好了，init 只是"装修"；另外忘记 return self 没关系（init 必须返回 None，返回其他值会报 TypeError），而 new 则必须返回对象。

30.4 常用运算符重载方法 (Common Operator-Overloading Methods)

英文原文

Just about everything you can do to built-in objects such as integers and lists has a corresponding specially named method for overloading in classes. Table 30-1 lists a few of the most common; there are many mo

re. In fact, many overloading methods come in multiple versions (e.g., `add`, `radd`, and `iadd` for addition), which is one reason there are so many. See the Python language reference manual for an exhaustive list of the special method names available. Table 30-1. Common operator-overloading methods

All overloading methods have names that start and end with two underscores to keep them distinct from other names you define in your classes. The mappings from special method names to expressions or operations are predefined by the Python language and documented in full in its standard language manual. For example, the name `add` always maps to `+` expressions by Python language definition, regardless of what an `add` method's code actually does; it's largely just a dispatch mechanism. Operator-overloading methods may be inherited from superclasses if not defined, just like any other methods. Operator-overloading methods are also all optional—if you don't code or inherit one, that operation is simply unsupported by your class, and attempting it will raise an exception. Some built-in operations, like printing, have defaults inherited from the implied object root class, but most built-ins fail for class instances if no corresponding operator-overloading method is present. As we go along here, keep in mind that at most overloading methods are used only in advanced programs that require objects to behave like built-ins, though the `init` constructor we've already met t

ends to appear in most classes. With that qualifier, let's explore some of the additional methods in Table 30-1 by example. NOTE — Measure twice, post once: Although expressions trigger operator methods, be careful not to assume that there is a speed advantage to cutting out the middleperson and calling the operator method directly. In fact, calling the operator method directly might be twice as slow, presumably because of the overhead of a function call, which Python avoids or optimizes in built-in cases. Here's the story for `len` and `len` using Chapter 21's timing techniques on Python 3.12 and macOS. Calling `len` directly takes twice as long (and has since Python 2.X):

```
$ python3 -m timeit -n 10000 -r 10 \-s "L = list(range(100))" "x = L.len()"10000 loops, best of 10: 53.4 nsec per loop
```

```
$ python3 -m timeit -n 10000 -r 10 \-s "L = list(range(100))" "x = len(L)"10000 loops, best of 10: 25.3 nsec per loop
```

This is not as contrived as it may seem—recommendations for using the slower alternative in the name of speed have been known to crop up in venues that shall remain nameless here.

中文翻译

你对整数、列表这类内建对象几乎能做的每一件事，在类中都有一个对应的特殊命名方法可供重载。Table 30-1 列出

了最常见的一部分；其实还有很多。事实上，许多重载方法都有多个版本（例如加法的 `add`、`radd` 和 `iadd`），这也是为什么这类方法如此之多的原因之一。完整的特殊方法名列表请参阅 Python 语言参考手册。Table 30-1. 常用运算符重载方法所有重载方法的名字都以两个下划线开头和结尾，以便与你类中定义的其他名字区分开。特殊方法名到表达式或操作的映射关系由 Python 语言预定义，并完整记录在它的标准语言手册中。例如，按 Python 语言定义，`add` 这个名字永远映射到 `+` 表达式——无论 `add` 方法体里实际写了什么；它基本上就是一个分派（dispatch）机制。运算符重载方法如果没有定义，也可以像其他方法一样从父类继承。运算符重载方法也全部是可选的——如果你没有编码或继承某个方法，那么这个操作就只是不被你的类支持，尝试使用它会抛出异常。有些内建操作（比如打印）有从隐含的 `object` 根类继承来的默认行为，但大多数内建操作在没有对应重载方法时，对类实例都会失败。在继续之前请记住：大多数重载方法只用于那些要求对象表现得像内建类型的进阶程序，而我们早已认识的 `init` 构造器则出现在绝大多数类中。带着这个前提，让我们用示例来探索 Table 30-1 中的其他方法。注——“先量两次，再发布一次”（Measure twice, post once）：虽然表达式会触发运算符方法，但要注意：别以为“绕过中间人、直接调用运算符方法”会有速度优势。事实上，直接调用运算符方法可能会慢一倍，原因大概是函数调用的开销——内建场景中 Python 会避免或优化这种开销。以下是 `len` 与 `len` 的对比测试，使用第 21 章的计时技巧，运行于 Python 3.12 和 macOS。直接调用 `len` 耗时是 `len` 的两倍（自 Python 2.X 以来一直如此）：

```
$ python3 -m timeit -n 10000 -r 10 \-s "L = list(range
```

(100))" "x = L.len()" 10000 loops, best of 10: 53.4 nsec per loop \$ python3 -m timeit -n 10000 -r 10 \-s "L = list(range(100))" "x = len(L)" 10000 loops, best of 10: 25.3 nsec per loop 这件事不像看上去那么牵强——"为了速度而推荐使用更慢的替代方案"的建议，已经在某些不便点名的地方屡见不鲜。

Table 30-1：常用运算符重载方法（中英对照）

方法 Method	实现功能 Implement	被调用于 Called for			
init	构造器 Constructor	对象创建：X = Class(args)			
del	析构器 Destructor	X 的对象回收			
add	运算符 +（及其他）	X + Y；若无 iadd 则 X +=			
or	运算符 \	（按位或）	X \	Y；若无 ior 则 X \	= Y
repr、str	打印、转换	print(X)、X、repr(X)、str(X)、f'{X!r}			
call	函数调用	X(pargs, kargs)			
getattr	属性获取	X.undefi ned（未定义属性）			
setattr	属性赋值	X.any = value			
delattr	属性删除	del X.any			
getattribute	属性获取	X.any（所有属性）			

getitem	索引、切片、迭代	X[i]、X[i:j]; 无 iter 时用于 for 等迭代			
setitem	索引与切片赋值	X[i] = value、X[i:j] = iterable			
delitem	索引与切片删除	del X[i]、del X[i:j]			
len	长度	len(X); 无 bool 时用于真			
bool	布尔测试	bool(X)、真值测			
lt、gt、le、ge、eq、ne	比较	X < Y、X > Y、X <= Y、X >= Y、X == Y、X != Y			
radd	右端运算	other +			
iadd	就地增强运算符	X += Y (否则用 add)			
iter、next	迭代工具	l = iter(X)、next(l)、for 及其他迭代; 无 contains 时			
contains	成员测试	item in X			
index	整数值	hex(X)、bin(X)、oct(X)、O[X]、O[			
enter、exit	上下文管理器 (第 34 章)	with obj as var:			
get、set、delete	描述符属性 (第 38 章)	X.attr、X.attr = value、del X.attr			

new	创建 (第 40 章)	对象创建, 在 init 之前			
-----	-------------	-----------------	--	--	--

深度理解

·核心概念：这张表是本章的"地图"。注意每个方法都有明确的三要素：什么时候被调用 (Called for)、实现什么操作 (Implements)、对应什么语法 (Method)。你不需要背表，但要建立"语法 → 方法名"的反射式联想：看见 `x[i]` 想到 `getitem`，看见 `x += y` 想到 `iadd/add`。

·底层实现：Python 解释器在编译阶段就把运算符转成对特殊方法的查找：`X + Y` 编译为 `BINARYOP + 指令`，执行时虚拟机在 `type(X)` 的 MRO (方法解析顺序) 中查找 `tpasnumber->nbadd` 槽位对应的 `add` 方法——这就是"分派机制"的含义：名字映射是死的，行为是活的。

·设计原因：方法名用双下划线包裹是为了隔离开命名空间——你几乎不可能无意中定义出一个与语言钩子撞名的普通方法；同时这也让内建函数 (`len`、`iter`、`repr`) 有了统一的"内建函数调用特殊方法"的约定。

·解决什么实际问题：Table 30-1 中的每种方法都对应一种"类型融入语言"的需求：`iter` 让对象能进 `for` 循环，`contains` 让 `in` 更快，`bool` 让 `if obj` 有意义.....框架设计者靠这些方法定义"协议 (protocol)"。

·初学者误区：①以为 `init` 是唯一重要的——其实它是"使用频率第一"，但"威力"远不如 `getattr`、`getitem` 这类"万能钩子"；②以为 `len` 返回负数/小数也行——它必须返回非负整数，否则 `len()` 报 `TypeError`；③以为重载方法

可以被普通方法替代——语法糖（如 `x[i]`）只能由特殊方法触发，普通方法名无法被 `[]` 语法调用。

NOTE 深究："直接调用魔术方法更快"是反直觉陷阱

·核心结论：`len(L)` 比 `L.len()` 快一倍。原因在于 CPython 内部：`len()` 是内建函数，C 实现直接走 `PySIZE` 这样的 C 级槽位，几乎没有 Python 层开销；而 `L.len()` 是一次完整的 Python 方法调用——要先在类型上查属性、绑定 `self`、压栈、进入字节码解释循环。

·生活例子：就像"坐地铁直达"（内建函数，C 实现直通）对比"打车还要绕路"（Python 方法调用，多一层包装）。直达更快，不是因为路径短，而是因为少了中间环节。

·学习建议：性能优化时优先使用内建函数和运算符语法（`len`、`sum`、`for`），而非手动调用特殊方法；这在数据密集型代码里差别可达数倍。这也是 Lutz 警示"以提速为名推荐更慢写法"的场合——社区论坛里确实出现过这种误导性建议。

30.5 索引与切片：`getitem` 和 `setitem` (Indexing and Slicing)

英文原文

Our first new method set allows your classes to mimic some of the behaviors of sequences and mappings. If defined in a class (or inherited by it), the `getitem` method is called automatically for instance-indexing o

perations. When an instance `X` appears in an indexing expression like `X[i]`, Python calls the `getitem` method inherited by the instance, passing `X` to the first argument and the index `i` in brackets to the second argument. For example, the following class returns the square of an index value—atypical perhaps but illustrative of the mechanism in general:

```
>>> class Indexer: ...
def getitem(self, index): ... return index ** 2
>>> X = Indexer()
>>> X[2] # X[i] calls X.getitem(i)
4
>>> for i in range(5): ... print(X[i], end=") # Runs getitem(X, i) each time
0 1 4 9 16
```

It's up to your class to define what this expression means, though it should generally imitate a sequence index or mapping key fetch; returning the index squared as done here works but probably won't qualify as "best practice."

中文翻译

我们遇到的第一组新方法，能让你的类模仿序列（sequence）和映射（mapping）的部分行为。如果在类中定义了（或继承了）`getitem` 方法，那么实例索引操作就会自动调用它。当实例 `X` 出现在 `X[i]` 这样的索引表达式中时，Python 调用该实例继承来的 `getitem` 方法，把 `X` 传给第一个参数，把方括号里的索引 `i` 传给第二个参数。例如，下面这个类返回索引值的平方——这也许不太典型，但能很好地说明这个机制本身：

```
>>> class Indexer: ...
def getitem(self, index): ... return index ** 2
>>> X = Indexer()
>>> X[2]
4
>>> for i in range(5): ... print(X[i], end=") # 每次都运行 getitem(X, i)
0 1 4 9 16
```

个表达式意味着什么，由你的类自己定义；不过它一般应该模仿序列索引或映射关键字的获取。像这里这样返回索引的平方能用，但大概称不上“最佳实践”。

代码分析

```
>>> class Indexer:
...     def __getitem__(self, index):
...         return index ** 2
>>> X = Indexer()
>>> X[2]                # X[i]  X.__getitem__(i)
4
>>> for i in range(5):
...     print(X[i], end=' ') # __getitem__(X, i)
0 1 4 9 16
```

·解释器如何处理：执行 `X[2]` 时，编译器生成 `BINARYSUBSCR` 字节码；虚拟机检查 `type(X)`（即 `Indexer`）的 `tp_as_sequence->sq_item`（在找不到时再查 `tp_as_mapping->mp_subscript`），找到后把对象本身作为 `self`、下标 2 作为参数调用。所以 `X[2]` 等价于 `X.getitem(2)`。

·设计思想：`getitem` 不检查类型、不限制“索引必须是整数”——`X['key']` 同样会触发它（这就是映射风格的入口）。它只是一个“把方括号里的东西交给你”的钩子。

深度理解

·核心概念：`getitem` 是索引语义的唯一入口，同时服务于索引、切片、甚至迭代（见 30.5.4）。一个方法，三套语法共用。

·底层实现：CPython 的对象布局里，序列与映射共享 `getitem` 槽位。发现目标后，返回值的类型没有任何强制——

—这就是为什么上面能返回平方数而不是真"取"元素。

·为什么这样设计：让序列与映射共享同一入口，降低了 C 层槽位数量，也让"单个方法同时支持序列和映射"成为可能（如 collections 里不少类就同时提供整数索引和关键字访问）。

·实际问题：框架开发者常用它实现"延迟加载"（懒索引）：虚拟大数据对象按需从磁盘/数据库取 index 对应的值，而内存中不保存整个数据。

·初学者误区：①以为 getitem 只用于"取存在的值"——其实越界也不会自动报错，得自己处理；②以为 $X[i] = v$ 也走 getitem——不，赋值走 setitem；③忘了它真实的第一个参数是实例（self），第二个才是索引，写反了函数签名会得到奇怪的报错。

30.5.1 拦截切片 (Intercepting Slices)

英文原文

Surprisingly, in addition to indexing, getitem is also called for slice expressions. Formally speaking, built-in object types handle slicing the same way. For example, the following demos slicing at work on a built-in list, using upper and lower bounds, omitted parts, and a stride (see Chapter 7 if you need a refresher on slicing):

```
>> L = [5, 6, 7, 8, 9] >> L[2:4] # Slice with slice syntax: 2..(4-1) [7, 8] >> L[1:] [6, 7, 8, 9] >> L[:-1] [5, 6, 7, 8] >> L[::2] [5, 7, 9]
```

Really, though, slicing bounds are bundled up into a slice object and passed to the list's implementation of indexing. In fact, you can always pass a slice object manually—slice syntax is mostly syntactic sugar for indexing with a slice object:

```
>> L[slice(2, 4)] # Slice with slice objects [7, 8] >> L[
slice(1, None)] [6, 7, 8, 9] >> L[slice(None, -1)] [5, 6, 7
, 8] >> L[slice(None, None, 2)] [5, 7, 9]
```

This matters in classes with a `getitem` method—this method will be called both for basic indexing (with an index or key) and for slicing (with a slice object). Our previous class won't handle slicing because its `math` assumes integer indexes are passed, but the following class will. When called for indexing, the argument is an integer as before:

```
>> class Indexer:
... def init(self, data): ... self.data = data ... def getitem
(self, index): ... # Called for index or slice ... print('geti
tem:', index) ... return self.data[index] # Perform inde
x or slice >> X = Indexer([5, 6, 7, 8, 9]) >> X[0] # Inde
xing sends getitem an integer getitem: 0 5 >> X[1] g
etitem: 1 6 >> X[-1] getitem: -1 9
```

When called for slicing, though, the method receives a slice object, which is simply passed along to the embedded list indexer in a new index expression:

```
>> X[2:4] # Slicing sends getitem a slice object getit
```

```
em: slice(2, 4, None) [7, 8] >> X[1:] getitem: slice(1, None, None) [6, 7, 8, 9] >> X[:-1] getitem: slice(None, -1, None) [5, 6, 7, 8] >> X[:2] getitem: slice(None, None, 2) [5, 7, 9]
```

Where needed, `getitem` can test the type of its argument, and extract slice object bounds—slice objects have attributes `start`, `stop`, and `step`, any of which can be `None` if omitted:

```
>> class Indexer:
... def getitem(self, index): ... if isinstance(index, int): #
Test usage mode ... print('indexing', index) ... else: ...
print('slicing', index.start, index.stop, index.step) >>
X = Indexer() >> X[99] indexing 99 >> X[1:99:2] slicing 1 99 2 >> X[1:] slicing 1 None None
```

Run a `help(slice)` in a REPL for more info on this very special-case built-in object type, and see the section "Membership: contains, iter, and getitem" for another example of slice interception at work.

中文翻译

令人惊讶的是，除了索引，`getitem` 也会被切片表达式调用。严格来说，内建对象类型处理切片的方式与此相同。例如，下面的演示在内建列表上展示了切片：使用上界、下界、省略部分和步长（如需回顾切片语法，请参见第 7 章）：

```
>>> L = [5, 6, 7, 8, 9] >>> L[2:4] # 切片语法：2..(4-1)
[7, 8] >>> L[1:] [6, 7, 8, 9] >>> L[:-1] [5, 6, 7, 8] >>>
L[:2] [5, 7, 9] 不过实际上，切片边界会被捆绑进一个 slice
```

对象 (slice object) , 再传给列表的索引实现。事实上, 你随时可以手动传入 slice 对象——切片语法在很大程度上就是"用 slice 对象做索引"的语法糖: >>> L[slice(2, 4)] # 用 slice 对象切片 [7, 8] >>> L[slice(1, None)] [6, 7, 8, 9] >>> L[slice(None, -1)] [5, 6, 7, 8] >>> L[slice(None, None, 2)] [5, 7, 9] 这对拥有 getitem 方法的类很重要——这个方法既会被基本索引调用 (传入索引或关键字), 也会被切片调用 (传入 slice 对象)。我们之前的类无法处理切片, 因为它的数学运算假定传入的是整数索引; 但下面这个类就可以。当被索引调用时, 参数和之前一样是整数: >>> class Indexer: ... def init(self, data): ... self.data = data ... def getitem(self, index): ... # 为索引或切片而调用... print('getitem:', index) ... return self.data[index] # 执行索引或切片 >>> X = Indexer([5, 6, 7, 8, 9]) >>> X[0] # 索引把整数发给 getitem getitem: 0 5 >>> X[1] getitem: 1 6 >>> X[-1] getitem: -1 9 而当被切片调用时, 方法收到的是一个 slice 对象, 只需把它放进一个新的索引表达式中传给内部列表的索引器即可: >>> X[2:4] # 切片把 slice 对象发给 getitem getitem: slice(2, 4, None) [7, 8] >>> X[1:] getitem: slice(1, None, None) [6, 7, 8, 9] >>> X[:-1] getitem: slice(None, -1, None) [5, 6, 7, 8] >>> X[::2] getitem: slice(None, None, 2) [5, 7, 9] 在需要的地方, getitem 可以检测参数的类型, 并取出 slice 对象的边界——slice 对象有 start、stop 和 step 三个属性, 缺省的任何一个都可以是 None: >>> class Indexer: ... def getitem(self, index): ... if isinstance(index, int): # 检测使用模式... print('indexing', index) ... else: ... print('slicing', index.start, index.stop, index.step) >>> X =

Indexer() >>> X[99] indexing 99 >>> X[1:99:2] slicing 1 99 2 >>> X[1:] slicing 1 None None 在 REPL 里运行 help(slice) 可以了解更多关于这种特殊内建对象类型的信息；另外请参见"Membership: contains, iter, and getitem"一节，那里有切片拦截的另一个实例。

代码分析

```
>>> class Indexer:
...     def __init__(self, data):
...         self.data = data
...     def __getitem__(self, index):
...         #
...         print('getitem:', index)
...         return self.data[index] #
>>> X = Indexer([5, 6, 7, 8, 9])
>>> X[2:4] # slice __getitem__
getitem: slice(2, 4, None)
[7, 8]
```

·解释器如何处理：X[2:4] 编译成的字节码同样是 BINARY SUBSCR，但在生成该指令前，编译器先把切片边界封装成 slice 对象（创建 slice(2, 4, None) 的字节码），然后一并以单参数传给 getitem。所以 X[2:4] 等价于 X.getitem(slice(2, 4, None))。该方法拿到 slice 对象后，self.data[index] 再次执行同一套 BINARYSUBSCR 字节码——只是这次作用在内建列表上，由 list 的 C 实现解析 slice 对象。

·设计思想：把切片的三个边界统一打包成一个参数，避免了为"索引 vs 切片"设计两套方法签名；start/stop/step 为 None 表示"省略"，语义与切片语法完全对齐。

·实用技巧：若你的类要"包装"一个列表/字符串并自然支持切片，最省事的就是像上面这样原样转发 `self.data[index]`；需要自定义切片语义（如二维矩阵的 `M[0:4, 1:3]`、或 pandas 风格的 `df[['a','b']]`）时，再用 `isinstance(index, slice)` 分支处理。

深度理解

·核心概念：切片语法是"糖"，真相是 slice 对象。每次 `X[i:j:k]` 都会实例化一个 slice 对象再传给 `getitem`——理解这一点，是自定义切片行为的钥匙。

·底层实现：CPython 中 slice 是一个真正存在的内建类型（PySliceNew），`BINARYSUBSCR` 指令在栈顶留出一个参数；若参数类型是 slice，C 层走 `sqslice` 路径或直接按 slice 对象解析。`slice.indices(len)` 方法还能把对任意长度的有效边界换算出来，是自定义切片常客。

·为什么这样设计：一个入口（`BINARYSUBSCR`）承担索引与切片，减少了指令集和对象布局的复杂度；同时让"给切片传负步长（逆序）"这类特殊语义集中在 slice 类型内部实现。

·实际问题：自定义容器（如矩阵库、数据表）需要同时支持 `M[3]`（取行）和 `M[1:5]`（取行范围）——只需在 `getitem` 里 `isinstance` 分支即可，这是科学与数据处理库的实际写法。

·初学者误区：看到 `X[i:j]` 报"too many arguments"——原因是没有意识到切片参数会被合并成一个 slice 对象，导致写了 `def getitem(self, i, j)` 这种错误签名；正确的签

名永远是 (self, key)。

30.5.2 拦截项赋值 (Intercepting Item Assignments)

英文原文

If used, the `setitem` index assignment method similarly intercepts both index and slice assignments—it receives a slice object for the latter, which may be passed along in another index assignment or used directly in the same way:

```
>>> class IndexSetter: ...
def init(self, data): ...
self.data = data ...
def setitem(self, index, value): ...
# Catch index or slice assignment ...
print('set item:', index) ...
self.data[index] = value # Assign index or slice
>>> X = IndexSetter([5, 6, 7, 8, 9])
>>> X[0] = 555
set item: 0
>>> X[-2:] = [888, 999, 111]
set item: slice(-2, None, None)
>>> X.data
[555, 6, 7, 888, 999, 111]
```

In fact, `getitem` may be called automatically in even more contexts than indexing and slicing—it's also an iteration fallback option, as you'll see in a moment. First, though, let's clear up a potential point of confusion in this category.

中文翻译

如果使用 `setitem`，这个索引赋值方法同样同时拦截索引赋值和切片赋值——对后者它会收到一个 `slice` 对象，可以把它放进另一个索引赋值中转发，或者按同样方式直接使用：

```
>>> class IndexSetter: ...
def init(self, data): ...
self.dat
```

```
a = data ... def setitem(self, index, value): ... # 捕获索引或切片赋值... print('setitem:', index) ... self.data[index] = value # 执行索引或切片赋值>>> X = IndexSetter([5, 6, 7, 8, 9])>>> X[0] = 555 setitem: 0>>> X[-2:] = [888, 999, 111] setitem: slice(-2, None, None)>>> X.data [555, 6, 7, 888, 999, 111]
```

事实上，`getitem` 会在比索引和切片更多的场景中被自动调用——它还是迭代的回退选项，你一会儿就会看到。不过首先，先澄清这一类别中一个容易混淆的点。

代码分析

```
>>> class IndexSetter:
...     def __init__(self, data):
...         self.data = data
...     def __setitem__(self, index, value):
...         #
...         print('setitem:', index)
...         self.data[index] = value #
>>> X = IndexSetter([5, 6, 7, 8, 9])
>>> X[0] = 555
setitem: 0
>>> X[-2:] = [888, 999, 111]
setitem: slice(-2, None, None)
>>> X.data
[555, 6, 7, 888, 999, 111]
```

解释器如何处理：`X[0] = 555` 编译为 `STORESUBSCR` 字节码（栈上依次是容器、下标、值三个对象）；虚拟机在 `type(X)` 上查 `setitem`，调用 `X.setitem(0, 555)`。注意赋值方向：`value` 在签名里是第三个参数。切片赋值 `X[-2:] = [888, 999, 111]` 同样先把 `slice(-2, None, None)` 打包成一个下标。

·设计思想：read/write 对称——索引有 `getitem/setitem`，切片因此免费获得赋值能力，无需额外方法。

深度理解

·核心概念：`setitem(self, index, value)` 是"下标赋值"的唯一下钩点；通过检查 `index` 是整数还是 `slice`，一套方法同时支持 `X[i] = v` 和 `X[i:j] = iterable` 两种赋值语法。

·底层实现：`STORESUBSCR` 会检查下标是否越界、是否切片；对切片赋值时 C 层按 `slice` 对象调用 `list` 的片段替换逻辑（长度不同会插入/删除元素）。自定义类要自己保证语义合理。

·为什么这样设计：对称性设计——既然读取路径共用 `getitem`，赋值路径也共用 `setitem`，减少方法数量、保持心智模型统一。

·实际问题：可变容器（如自定义缓存、配置对象）常用它做写入校验——在真正落盘/更新前检查值合法、触发回调或写日志。

·初学者误区：写出 `def setitem(self, index, value)` 却漏掉 `value` 参数直接报 `"takes 2 positional arguments but 3 were given"`；或以为 `X[i] = v` 会调用 `getitem` 先取出再赋值——不会，赋值路径与读取路径完全独立。

30.5.3 但 `index` 表示"当作整数" (But `index Means As-Integer`)

英文原文

Don't mistake the (perhaps unfortunately named) `index` method for `getitem` index interception. The `index` method returns an integer value for an instance when one is needed—and in retrospect, might have been better named `asindex`. For example, it's used by built-ins that convert to digit strings:

```
>> class C:
... def index(self): ... return 255
>> X = C()
>> hex(X)
# Integer value '0xff'
>> bin(X)
'0b11111111'
>> oct(X)
'0o377'
```

Although this method does not intercept instance indexing like `getitem`, it is also used in contexts that require an integer—including indexing and slicing:

```
>> eds = [f'LP{i}e' for i in range(256)]
>> eds[255]
'LP255e'
>> X = C()
>> eds[X]
# As index (not X[i]!)
'LP255e'
>> eds[X:]
# As index (not X[i:]!)
['LP255e']
```

Though arguably misnamed, there is a rich history of former methods that `index` subsumes, which we'll mercifully omit here. The `getitem` method also subsumes former tools but retains a fallback role up next.

中文翻译

不要把（可能命名得不太好的）`index` 方法误解成 `getitem` 那样的索引拦截。`index` 方法是在需要一个整数时，为实例返回一个整数值——事后看来，它或许该改名为 `asindex`。

例如，它被那些把对象转换为数字字符串的内建函数所使用：

```
>>> class C: ... def index(self): ... return 255 >>> X = C() >>> hex(X) # 整数值'0xff' >>> bin(X)'0b11111111' >>> oct(X)'0o377'
```

虽然这个方法不像 `getitem` 那样拦截实例索引，但它也用于其他需要整数的场景——包括索引和切片：

```
>>> eds = [f'LP{i}e' for i in range(256)] >>> eds[255]'LP255e' >>> X = C() >>> eds[X] # 作为索引（不是 X[i]！）'LP255e' >>> eds[X:] # 作为索引（不是 X[i:]！）['LP255e']
```

尽管这个名字颇有争议，但 `index` 取代了一大批从前的老方法（这段丰富的历史我们就不赘述了）。`getitem` 方法同样也取代了早期的工具，但它接下来还保留着一个“回退”角色。

代码分析

```
>>> class C:
...     def __index__(self):
...         return 255
>>> X = C()
>>> hex(X)           #
'0xff'
>>> eds[X]           # X[i]
'LP255e'
```

·解释器如何处理：`hex(X)`、`bin(X)`、`oct(X)`、以及把对象放进 `[]` 当下标用（`eds[X]`）时，CPython 调用 `index()` 协议（C 层 `PyLongFromIndex` → `nbindex` 槽位），要求该方法必须返回 `int`。它对 `float` 这类“能转但不能无损映射到整数槽位”的类型抛 `TypeError`，保证索引不丢精度。

·与 `getitem` 的根本区别：`eds[X]` 是“`eds` 的 `getitem` 被

调用、并需要把 X 换成整数 255", 不是 Xgetitem。一个方法在"右边当数据源", 一个方法在"左边当下标"——方向相反, 容易绕晕。

深度理解

·核心概念: index = "给我这个对象对应的整数", 用于 hex/bin/oct、eds[X]、eds[X:]、format(obj,'b') 以及第三方库 (如切片的第三方索引)。它与 int 不同——int(X) 走 int, 而"当作下标"必须走 index。

·底层实现: 内建槽位 nbindex 专门服务于"无损整数化"。hex() 等函数内部先调用它变成 PyLong, 再执行进制格式化; BINARYSUBSCR 在遇到非 int 下标时也会尝试调用它 (失败则报 TypeError, 所以列表下标不能用 float)。

·为什么这样设计: 为 numpy 标量、自定义整数类型提供"无缝融入列表下标"的通道, 同时拒绝 float 的下标使用 (避免丢失精度隐患)。

·实际问题: 让自定义"整数包装"类型 (如带单位的值) 能被直接用作任何 [] 下标和切片边界。

·初学者误区: 把 index 当成"定义索引行为"——那是 getitem 的事; index 是数据源而非行为钩子, 方向迥异。

30.5.4 基于索引的迭代: getitem (Index Iteration)

英文原文

Our next hook isn't always obvious to beginners but turns out to be surprisingly useful. In the absence of the more specific iteration methods we'll get to in the next section, the `for` statement works by repeatedly indexing an object from zero to higher indexes, until an out-of-bounds `IndexError` exception is detected.

Because of that, `getitem` also turns out to be one way to overload iteration in Python—if only this method is defined, `for` loops call the class's `getitem` each time through, with successively higher offsets.

It's a case of "code one, get one free"—any built-in or user-defined object that responds to indexing also responds to `for` loop iteration:

```
>> class StepperIndex:
...     def getitem(self, i): ... return self.data[i]
>> X = StepperIndex()
>> X.data = 'hack' # X is a StepperIndex object
>> X[1] # Indexing calls getitem
'a'
>> for item in X: # for loops call getitem
...     print(item, end='') # for indexes items 0..N
h a c k
```

In fact, it's really a case of "code one, get a bunch free." Any class that supports `for` loops automatically supports all iteration tools in Python, many of which we've explored in earlier chapters (e.g., iteration tools were presented in Chapter 14). For instance, the `in` membership test, list comprehensions, the `map` built-in

, list and tuple assignments, and some type constructors will also call `getitem` automatically to iterate if it's defined:

```
>>'k' in X # All call getitem too True >> [c for c in X]
# Comprehension ['h','a','c','k'] >> list(map(str.upper,
X)) # map calls ['H','A','C','K'] >> (a, b, c, d) = X # Sequence
assignments >> a, d ('h','k') >> list(X), tuple(X), '.join(X) # And
so on... (['h','a','c','k'], ('h','a','c','k'),'hack') >> X
```

<main.StepperIndex object at 0x10c4bcc20> In practice, this technique can be used to create objects that provide a sequence interface and to add logic to built-in sequence type operations; we'll revisit this idea when extending built-in types in Chapter 32.

中文翻译

我们的下一个钩子对初学者来说不那么显眼，但它出人意料地有用。在没有下一节要讲的更专门的迭代方法时，`for` 语句的工作方式是：从 0 开始反复对对象做越来越高的索引，直到检测到越界的 `IndexError` 异常。正因如此，`getitem` 也成了 Python 中重载迭代的一种方式——只要定义了这一个方法，`for` 循环每一轮都会用递增的偏移量调用类的 `getitem`。这是典型的“写一个，白得一个”（code one, get one free）——任何能响应索引的内建对象或用户定义对象，也都能响应 `for` 循环迭代：

```
>>> class StepperIndex:
...     def getitem(self, i): ... return self.data[i]
>>> X = StepperIndex()
>>> X.data = 'hack' # X 是 StepperIndex 对象
>>> X[1] # 索引调用 getitem'a'
>>> for item in
```


X: # for 循环调用 getitem ... print(item, end='') # for 从 0 开始逐项索引h a c k 事实上，这其实是"写一个，白得一大堆"（code one, get a bunch free）。任何支持 for 循环的类，都会自动支持 Python 中的所有迭代工具——其中很多我们在前面章节已经探索过（例如第 14 章就介绍过迭代工具）。比如，in 成员测试、列表推导式、map 内建函数、列表和元组解包赋值、以及一些类型构造器，在定义了 getitem 时都会自动调用它来迭代：>>>'k' in X # 这些都调用 getitem True >>> [c for c in X] # 推导式['h', 'a', 'c', 'k'] >>> list(map(str.upper, X)) # map 调用['H', 'A', 'C', 'K'] >>> (a, b, c, d) = X # 序列解包赋值>>> a, d ('h', 'k') >>> list(X), tuple(X), ''.join(X) # 诸如此类.....(['h', 'a', 'c', 'k'], ('h', 'a', 'c', 'k'), 'hack') >>> X <main.StepperIndex object at 0x10c4bcc20> 在实践中，这个技巧可以用来创建提供序列接口的对象，并为内建序列类型的操作添加逻辑；我们将在第 32 章扩展内建类型时重新审视这个想法

。

代码分析

```
>>> class StepperIndex:
...     def __getitem__(self, i):
...         return self.data[i]
>>> X = StepperIndex()
>>> X.data = 'hack'
>>> for item in X:          # for __getitem__
...     print(item, end=' ') # for 0
h a c k
```

·解释器如何处理：for item in X 编译为 GETITER 指令。解释器先在 type(X) 上找 iter——没有找到，于是回退到"序列协议"：修改迭代策略为 iter(X) 返回一个内部序列迭

代器，它每次调用 next 都用 0、1、2.....递增去调 X.getitem(i)，直到方法抛 IndexError (STOPITERATION 由此触发，for 正常结束)。

·设计思想：这就是"写一个得一堆"的机制来源——in、推导式、map、解包、tuple() 全部复用了同一条 GETITER →getitem 回退路径，你只需要实现一个 getitem。

·注意局限：回退式迭代无法自定义停止机制——一切以 IndexError 为终。如果你的对象不是"0..N 连续下标可索引"的形态（比如数据来自数据库查询、无限流），就需要用下一节的 iter/next 协议。

深度理解

·核心概念：for 有两个迭代通道——协议通道 (iter/next，首选) 与索引回退通道 (getitem + IndexError，兜底)。只有 getitem 也能让类迭代，这是 Python 兼容旧式序列的遗产设计。

·底层实现：GETITER 字节码：若对象有 iter 用之；否则尝试 sqitem (即 getitem 的 C 槽位)，成功则包成一个"索引迭代器"；两者都没有则抛 TypeError:'X' object is not iterable。

·为什么这样设计：历史上的 Python (2.x 早期) 只有 getitem 这一种迭代手段。为了让"既支持索引又支持迭代"的类不用写两套代码，保留了这个回退通道——代价是行为受 IndexError 局限。

·实际问题：给"只读推进、浪费实现"的简单序列（如数学中的向量）贴一个小接口，就能立即获得全部迭代工具；

`getitem` 还能一块儿处理 `X[3]`，一箭双雕。

·初学者误区：①以为 `for` 非要 `iter` 不可——只有 `getitem` 也能迭代，但性能与灵活性都差些；②以为迭代会调用 `len` 来确定范围——不会，它以 `IndexError/StopIteration` 为终点；③`getitem` 里索引越界时没有抛 `IndexError`（比如返回了 `None`），会导致 `for` 永远停不下来。

30.6 可迭代对象：iter 和 next (Iterable Objects)

英文原文

Although the `getitem` technique of the prior section works, it's really just a legacy fallback for iteration. To day, all iteration tools in Python will try the `iter` method first before trying `getitem`. That is, they prefer the iteration protocol we learned about in Chapter 14 over repeatedly indexing an object; only if the object does not support the iteration protocol is indexing attempted instead. Generally speaking, you should prefer `iter` too—it supports general iteration tools better than `getitem` can. For a review of this model's essentials, see Figure 14-1 in Chapter 14. In brief, iteration tools work by running an iterable object's `iter` method to fetch an iterator object. If this works as planned, Python then repeatedly calls this iterator object's `next` method to produce items until it raises a `StopIteration`

n exception. Built-in functions `iter` and `next` are also available as a conveniences for manual iterations—`iter(X)` is the same as `X.iter()` and `next(l)` is the same as `l.next()`, and Python internals may vary. This iterable-object interface is given priority and attempted first. Only if no such `iter` method is found, Python falls back on the `getitem` scheme and repeatedly indexes by offsets as before until an `IndexError` exception is raised.

中文翻译

虽然上一节的 `getitem` 技巧可行，但它其实只是迭代的遗留回退方案。如今，Python 的所有迭代工具都会先尝试 `iter` 方法，然后才尝试 `getitem`。也就是说，它们更倾向于使用我们在第 14 章学到的迭代协议（iteration protocol），而不是反复给对象做索引；只有当对象不支持迭代协议时，才会改用索引方式。一般来说，你也应该优先使用 `iter`——它比 `getitem` 能更好地支持通用迭代工具。想回顾这个模型的核心，请参见第 14 章的 Figure 14-1。简言之：迭代工具的工作方式是运行可迭代（iterable）对象的 `iter` 方法去取回一个迭代器（iterator）对象。如果这一步顺利，Python 接着反复调用这个迭代器对象的 `next` 方法产生元素，直到它抛出 `StopIteration` 异常。内建函数 `iter` 和 `next` 也作为手动迭代的便利工具可用——`iter(X)` 等价于 `X.iter()`，`next(l)` 等价于 `l.next()`，而 Python 内部实现可能略有不同。这个可迭代对象接口拥有最高优先级，会被优先尝试。只有当找不到 `iter` 方法时，Python 才会回退到 `getitem` 方案，像之前那样反复按偏移量索引，直到抛出 `IndexError` 异常。

深度理解

- 核心概念：Python 的迭代是 iterable（可迭代对象）与 iterator（迭代器）两个角色分离的协议：iter(X) 从可迭代对象取迭代器；next(I) 从迭代器取元素。for/推导式/in 等全部建立在 iter()+next() 之上。
- 底层实现：GETITER 字节码 → 查 iter（失败再查 getitem）→ 得到迭代器；FORITER 字节码 → 反复调迭代器的 sqitem/next（C 层 tpiternext 槽位），StopIteration 被翻译成"循环终止"而非异常泄漏。
- 为什么这样设计：把"容器"与"游标"分离，让一个容器可以同时被多个迭代器遍历（各自独立记位置）——这是"多个活跃迭代互相独立"（30.6.4）的架构基础。
- 实际问题：数据库连接、网络流、文件读取天然是"流式"的——它们不是随机索引的序列，而是"拉取一步、推进一点"；iter 协议正是为这类对象设计的通用抽象。
- 初学者误区：把"可迭代对象"与"迭代器"混为一谈。list 是可迭代对象但不是迭代器（iter(lst) 才返回迭代器）；而 iter 返回 self 的类（如 Squares）则身兼两职，这在 30.6.2 会暴露"只能扫一遍"的副作用。

30.6.1 用户自定义可迭代对象（User-Defined Iterables）

英文原文

In the iter scheme, classes implement user-defined it

erables by simply implementing the iteration protocol introduced in Chapter 14 and elaborated in Chapter 20. For example, the code in Example 30-2 uses a class to define a user-defined iterable that generates squares on demand, instead of all at once.

Example 30-2. squares.py When imported, its instances can appear in iteration tools just like built-ins:

```
$ python3 >> from squares import Squares >> for i  
in Squares(1, 5):
```

```
... print(i, end=" ") # for calls iter 1 4 9 16 25 # Each iteration  
calls next
```

Here, the iterator object returned by `iter` is simply the instance `self` because the `next` method is part of this class itself. In more complex scenarios, the iterator object may be defined as a separate class and object with its own state information to support multiple active iterations over the same instance data (we'll code an example of this in a moment).

The end of the iteration is signaled with a Python `raise` statement—introduced in Chapter 29 and covered in full in the next part of this book, but which simply raises an exception as if Python itself had done so. Because of all this, manual iterations work the same on user-defined iterables as they do on built-in objects:

```
>> X = Squares(1, 5) # Iterate manually: what loops  
do >> I = iter(X) # iter calls iter >> next(I) # next call
```

```
s next 1 >> next(l) 4 ... more omitted ... >> next(l) 25
>> next(l) # Can catch this in try statement StopIteration
```

An equivalent coding of this iterable with `getitem` might be less natural because the `for` would then iterate through all offsets zero and higher; the offsets passed in would be only indirectly related to the range of values produced (`0...N` would need to map to `start...stop`). Because `iter` objects retain explicitly managed state between `next` calls, they can be more general than `getitem`.

On the other hand, iterables based on `iter` can sometimes be more complex and less functional than those based on `getitem`. They are really designed for iteration, not random indexing—in fact, they don't overload the indexing expression at all, though you can collect their items in a sequence such as a list to enable other operations:

```
>> X = Squares(1, 5) >> X[1] TypeError: 'Squares' object is not subscriptable >> list(X)[1] 4
```

中文翻译

在 `iter` 方案中，类只要实现第 14 章引入、第 20 章详细阐述过的迭代协议，就能实现用户自定义的可迭代对象。例如，Example 30-2 中的代码用一个类定义了一个用户自定义的可迭代对象，它能按需（on demand）逐个生成平方数

，而不是一次性全部生成。Example 30-2. squares.py 导入后，它的实例可以和内建对象一样出现在各种迭代工具中：

```
$ python3 >>> from squares import Squares >>>
for i in Squares(1, 5): ... print(i, end='') # for 调用 iter
1 4 9 16 25 # 每次迭代调用 next 在这里，iter 返回的迭代器对象就是实例自身 self，因为 next 方法本就属于这个类。在更复杂的场景中，迭代器对象可以被定义成独立的类和对象，携带自己的状态信息，以便在同一实例数据上支持多个活跃迭代（我们等会儿就写一个这样的例子）。迭代的结束用 Python 的 raise 语句来发出信号——第 29 章介绍过它，本书下一部分会完整讲解；简单说它就像 Python 自己那样抛出一个异常。有了这一切，手动迭代在用户自定义可迭代对象上的行为，与在内建对象上完全一致：>>> X = Squares(1, 5) # 手动迭代：循环内部就是这样做>>> l = iter(X) # iter 调用 iter >>> next(l) # next 调用 next 1
>>> next(l) 4...更多省略...>>> next(l) 25 >>> next(l)
# 可以在 try 语句里捕获这个异常StopIteration 用 getitem 实现等价的可迭代对象可能不那么自然，因为那样 for 会从 0 开始遍历所有偏移量；传入的偏移量与产生的值区间只有间接关系（0...N 需要映射到 start...stop）。由于 iter 对象在每次 next 调用之间显式维护状态，它们可以比 getitem 更通用。另一方面，基于 iter 的可迭代对象有时比基于 getitem 的更复杂、功能更少。它们是为迭代而设计的，不是为随机索引——事实上它们根本不重载索引表达式；不过你可以把元素收集进一个序列（比如列表），从而启用其他操作：>>> X = Squares(1, 5) >>> X[1] Type Error:'Squares' object is not subscriptable >>> list(X)
[1] 4
```


代码分析 (Example 30-2. squares.py)

```
class Squares:
    def __init__(self, start, stop):      #
        self.value = start - 1          # start
        self.stop = stop
    def __iter__(self):                  #
        return self                     # iter()
    def __next__(self):                  #
        if self.value == self.stop:     # next()
            raise StopIteration         #
        self.value += 1
        return self.value ** 2
```

```
>>> from squares import Squares
>>> for i in Squares(1, 5):
...     print(i, end=' ')              # for __iter__
1 4 9 16 25                           # __next__
```

·解释器如何处理：for i in Squares(1, 5) 首先执行 GETITER：找到 iter，调用得到 self（同一个实例）作为迭代器。随后每一轮执行 FORITER：调用迭代器的 next，把返回值赋给 i；当 next 抛出 StopIteration 时，for 捕获它并正常结束循环，异常不会泄漏到外界。注意 self.value 是实例属性——它在 iter 与 next 之间传递着“现在到哪了”的状态。

·设计思想：iter 返回 self 是“迭代器即本身”的简写——省去单独写一个迭代器类；但代价是实例本身就是唯一的迭代器（共享一份状态），为 30.6.2 的“单次扫描”埋下伏笔。

·对比 getitem：索引方案里 for 的偏移量 (0,1,2,...) 与真实值域 (start...stop) 是两套坐标，靠映射换算；而 it

er 方案把状态存在属性里，值域与推进逻辑直接对接，更自然。但 Squares 也完全丢弃了 `X[i]` 下标能力（`TypeError: not subscriptable`）——想要下标就得两者都写。

深度理解

·核心概念：迭代协议两个接口的分工——`iter` 交出"游标"，`next` 推进"游标"并产出元素，`StopIteration` 宣告"游标越界"。手动 `iter()/next()` 与 `for` 循环底层完全是一回事。

·底层实现：Squares 实例同时充当可迭代对象与迭代器。CPython 的 `tpiter (iter)` 与 `tpiternext (next)` 槽位指向同一对象时，FORITER 调用链就是"对象--next-->值"。

·为什么这样设计：流式处理让大范围（如 `Squares(1, 109)`）不占内存、即产即用；且迭代状态显式存于实例属性，比闭包式的生成器更"看得见摸得着"（类方案的优势在于可继承、可多方法、结构清晰）。

·实际问题：任何"逐个产生、量可能巨大"的数据源——日志流、分页 API、传感器读数、无限序列——都适合用 `iter+next` 类建模。需要下标就用 `list(X)` 物化。

·初学者误区：①`next` 到达终点不抛 `StopIteration` 而返回 `None` 之类，会导致 `for` 认为是正常元素、循环失控或结果多出垃圾；②忘记 `iter` 只写 `next`，`for` 会报 `"object is not iterable"`；③`next` 里忘自增状态，产生无限重复元素。

30.6.2 单次扫描与多次扫描 (Single versus multiple scans)

英文原文

The iter scheme is also the implementation for all the other iteration tools we saw in action for the getitem method—membership tests, type constructors, sequence assignment, and so on. Unlike our prior getitem example, though, we also need to be aware that a class's iter may be designed for a single traversal only, not many. Classes can choose either behavior explicitly in their code.

For example, because the current Squares class's iter always returns self with just one copy of iteration state, it is a single-scan iteration; once you've iterated over an instance of that class, it's empty. Calling iter again on the same instance returns self again—in whatever state it may have been left. As coded, you generally need to make a new iterable instance object for each new iteration:

```
>> X = Squares(1, 5) # Make an iterable with state >
> [n for n in X] # Exhausts items: iter returns self [1, 4, 9, 16, 25]
>> [n for n in X] # Now it's empty: iter returns same self []
>> [n for n in Squares(1, 5)] # Make a new iterable object [1, 4, 9, 16, 25]
>> list(Squares(1, 3)) # A new object for each new iter call [1, 4, 9]
```

To support multiple iterations more directly, we could also recode this example with an extra class or other technique, as we will in a moment. As is, though, by creating a new instance for each iteration, you get a fresh copy of iteration state:

```
>> 36 in Squares(1, 10) # Other iteration tools True
>> a, b, c = Squares(1, 3) # Each calls iter and then next
>> a, b, c (1, 4, 9) >> ':'.join(map(str, Squares(1, 5)))
'1:4:9:16:25'
```

Just like single-scan built-ins such as `map`, converting to a list supports multiple scans as well but adds time and space performance costs, which may or may not be significant to a given program:

```
>> X = Squares(1, 5) >> tuple(X), tuple(X) # Iterator exhausted in second tuple()
((1, 4, 9, 16, 25), ()) >> X = list(Squares(1, 5)) >> tuple(X), tuple(X)
((1, 4, 9, 16, 25), (1, 4, 9, 16, 25))
```

We'll improve this to support multiple scans more directly ahead after a bit of compare and contrast.

中文翻译

`iter` 方案也是我们在 `getitem` 上看到的其他迭代工具（成员测试、类型构造器、序列赋值等）的实现基础。不过与我们之前的 `getitem` 例子不同，我们还需要注意：类的 `iter` 可能只打算支持单次遍历，而不是多次。类可以在代码中显式选择任意一种行为。例如，因为当前 `Squares` 类的 `iter` 总是用同一份迭代状态返回 `self`，所以它是单次扫描（sing

le-scan) 迭代；一旦你遍历过这个类的某个实例，它就空了。对同一实例再调用 `iter` 还是返回 `self`——不过是刚才被用到一半、卡在某个状态上的 `self`。按现在的写法，你要为每次新迭代新造一个可迭代实例对象：

```
>>> X = Squares(1, 5) # 做一个带状态的可迭代对象
>>> [n for n in X] # 耗尽元素：iter 返回 self [1, 4, 9, 16, 25]
>>> [n for n in X] # 现在空了：iter 返回同一个 self []
>>> [n for n in Squares(1, 5)] # 新造一个可迭代对象[1, 4, 9, 16, 25]
>>> list(Squares(1, 3)) # 每次 iter 调用都用新对象[1, 4, 9]
```

若要更直接地支持多次迭代，我们可以用一个额外的类或其他技术重写这个例子，稍后我们就会这么做。而就目前而言，每次迭代新建一个实例，就能得到一份全新的迭代状态：

```
>>> 36 in Squares(1, 10) # 其他迭代工具True
>>> a, b, c = Squares(1, 3) # 每个都调用 iter 和 next
>>> a, b, c (1, 4, 9)
>>> ':'.join(map(str, Squares(1, 5))) '1:4:9:16:25'
```

就像 `map` 这类单次扫描的内建对象一样，转换成 `list` 也能支持多次扫描，但要付出时间和空间上的性能代价——对具体的程序而言这个代价可大可小：

```
>>> X = Squares(1, 5)
>>> tuple(X), tuple(X) # 第二次 tuple() 时迭代器已耗尽((1, 4, 9, 16, 25), ())
>>> X = list(Squares(1, 5))
>>> tuple(X), tuple(X) ((1, 4, 9, 16, 25), (1, 4, 9, 16, 25))
```

经过一番对比之后，我们马上会改进它，让多次扫描得到更直接的支持。

代码分析

```
>>> X = Squares(1, 5)
>>> [n for n in X]           # __iter__  self
[1, 4, 9, 16, 25]
>>> [n for n in X]           # __iter__  self
[]
```

·解释器如何处理：第一次推导式：GETITER 调用 X.iter() 返回 X 自身；FORITER 一路推进 X.value 直到 5 抛 StopIteration——此时 X.value == 5。第二次推导式：iter 又返回同一个 X（状态没复位），第一次 next 就发现 value == stop，立即 StopIteration，结果为空列表。

·核心教训："迭代器即自身"意味着状态共享——可用状态只有一份，耗完即亡。这与 list/str 不同：那些内建容器在 iter 里返回的是独立的游标对象，容器自身永不耗尽。

深度理解

·核心概念：单次扫描 (single-scan) vs 多次扫描 (multi-scan) 是一个设计决策，由 iter 返回什么决定：返回 self (共享状态) = 单次；返回新对象 (独立状态) = 多次。

·底层实现：list 的 iter 内部创建并返回一个新的 listiterator (独立保存当前下标)，所以同列表可同时有多个活跃迭代器；map/生成器则整个对象就是游标，天然单次。

·为什么这样设计：单次扫描在"流式、一次性消费"场景反而是优点 (无需保存全部数据)；让类自行选择，把自由度交给设计者。对数据量极大、"只遍历一次"的场景，单次扫描省内存。

·实际问题：打印/遍历一个超大日志对象后，想再遍历一

次却发现空了——这是新手最容易踩的坑；解法是每次新建实例，或改造成“返回独立迭代器”的结构（见 30.6.4），或先 `list()` 物化。

·初学者误区：以为同一实例能反复 `for`——不行，`Squares` 实例只能扫一遍；想多扫，要么每轮新建实例，要么改成多迭代器架构。`tuple(X), tuple(X)` 输出 `((1, 4, 9, 16, 25), ())` 就是这一陷阱的直观证据。

30.6.3 类与生成器 (Classes versus generators)

英文原文

Notice that the `Squares` iterable of Example 30-2 that we've been using so far would probably be simpler if it was coded with generator functions or expressions—tools introduced in Chapter 20 that automatically produce iterable objects and retain local variable state between iterations: `>>> def gsquares(start, stop): # Generator function ... for i in range(start, stop + 1): ... yield i ** 2 >>> for i in gsquares(1, 5): ... print(i, end="") 1 4 9 16 25 >>> for i in (x ** 2 for x in range(1, 6)): # Generator expression ... print(i, end="") 1 4 9 16 25` Unlike classes, generator functions and expressions implicitly save their state and create the methods required to conform to the iteration protocol—with obvious advantages in code conciseness for simpler examples like these. That is, generators' automatic and implicit `iter` and `next` suffice here. On the other hand, the class's more explicit attributes and methods, extra structur

e, inheritance hierarchies, and support for multiple behaviors may be better suited for richer use cases. Of course, for this artificial example, you could in fact skip both techniques and simply use a for loop, map, or a list comprehension to build the list all at once. Barring performance data to the contrary, the best and fastest way to accomplish a task in Python is often also the simplest: `>>> [x2 for x in range(1, 6)]` `[1, 4, 9, 16, 25]` That said, classes are better at modeling more complex iterations, especially when they can benefit from the assets of classes in general. An iterable that produces items in a complex database or web service result, for example, might be able to take fuller advantage of classes—and leverage more flexible coding structures like that of the next section.

中文翻译

请注意，我们一直在用的 Example 30-2 的 Squares 可迭代对象，如果用生成器函数（generator function）或生成器表达式（generator expression）来编写，可能会更简单——这些工具在第 20 章引入，它们会自动产生可迭代对象，并在各次迭代之间保留局部变量状态：

```
>>> def gsquares(start, stop): # 生成器函数...
    for i in range(start, stop + 1): ...
    yield i2
>>> for i in gsquares(1, 5): ...
    print(i, end='')
1 4 9 16 25
>>> for i in (x2 for x in range(1, 6)): # 生成器表达式...
    print(i, end='')
1 4 9 16 25
```

与类不同，生成器函数与表达式会隐式地保存状态，并自动创建符合迭代协议所需的方法——对这类简单例子来说，代码

简洁性的优势显而易见。也就是说，生成器自动且隐式的 `iter` 与 `next` 在这里已经够用。另一方面，类更显式的属性和方法、额外的结构、继承层次，以及对多种行为的支持，可能更适合更丰富的应用场景。当然，对这个刻意的例子，你其实可以两种技巧都不必用，直接用 `for` 循环、`map` 或列表推导式一次性把列表建好。除非有相反的性能数据，否则在 Python 中完成一项任务最好的、也是最快的方式，往往就是最简的方式：`>>> [x2 for x in range(1, 6)]` `[1, 4, 9, 16, 25]` 话虽如此，类更擅长建模更复杂的迭代——尤其是当它们能从类的一般优势中获益时。例如，一个从复杂数据库或 Web 服务结果中逐个产生元素的可迭代对象，可能就能更充分地利用类的优势——并借助类似下一节的更灵活的编码结构。

代码分析

```
>>> def gsquares(start, stop):    #
...     for i in range(start, stop + 1):
...         yield i ** 2
>>> list(gsquares(1, 5))
[1, 4, 9, 16, 25]
```

·解释器如何处理：只要函数体内含 `yield`，调用它就不会执行函数体，而是创建并返回一个生成器对象（generator object），它的 `iter` 返回自身、`next` 从上次停下的地方继续执行局部代码。执行 `next` 时，函数在 `yield` 处“挂起”（保存整个帧栈：局部变量、指令指针），下次 `next` 恢复执行——这就是“自动保存状态”的真相：帧（frame）对象被保存在堆上而非栈上。

·对比类方案：生成器的状态隐式存在帧里（`start`、`i` 等局

部变量)，无需手写属性；而类方案状态显式存于实例属性。简洁性上生成器完胜；但类可以同时提供 `init` 之外的方法、参与继承、支持多个行为——结构化是它的本钱。

深度理解

·核心概念：同一目标三条路——手写迭代类（显式）、生成器函数/表达式（隐式）、列表推导式（一步到位）。选择标准是复杂度：简单一次性任务用推导式，中等任务用生成器，需要结构化状态与多重行为的才值得上类。

·底层实现：生成器对象内部保存一个 `frame` 引用与 `resume` 指针；`yield` 把结果返回给调用方并暂停，`StopIteration` 在函数自然结束或 `return` 时自动抛出。`iter/next` 由生成器对象的 `C` 类型直接提供，无需你写。

·为什么这样设计："迭代协议 + 自动状态保存"让生成器成为类更简洁的替代；同时两者可混用——把生成器函数嵌在类的 `iter` 里（见 30.6.6），兼得结构性与简洁性。

·实际问题：数据处理流水线（读大文件逐行清洗）、无限序列（斐波那契）、惰性求值都在生成器上建；需要调试、断点、状态查询时，换显式类更好。

·初学者误区：①把"生成器函数"误当成普通函数——调用它不执行、得到的是生成器对象；②以为 `yield` 是"返回列表"——它是逐个产出；③过度使用列表推导式（`[x2 for x in range(1000)]`）浪费内存——算术上该用生成器表达式去迭代。Lutz 的提醒值得记住：最简单的方案往往也最快。

30.6.4 同一对象上的多个迭代器 (Multiple Iterators on One Object)

英文原文

Earlier, it was mentioned in passing that the iterator object (with a `next`) produced by an iterable may be defined as a separate class with its own state information to more directly support multiple active iterations over the same data. To understand this better, consider what happens when we step across a built-in type like a string:

```
>>> S = 'ace'>>> for x in S: ... for y in S: ... print(x + y, end=" ") aa ac ae ca cc ce ea ec ee
```

Here, the outer loop grabs an iterator from the string by calling `iter`, and each nested loop does the same to get an independent iterator. Because each active iterator has its own state information, each loop can maintain its own position in the string, regardless of any other active loops. Moreover, we're not required to make a new string or convert to a list each time; the single string object itself supports multiple scans. We saw related examples earlier, in Chapters 14 and 20. For instance, generator functions and expressions, as well as built-ins like `map` and `zip`, proved to be single-iterator objects, thus supporting a single active scan. By contrast, the `range` built-in, and other built-in types like lists, support multiple active iterators with ind

dependent positions. When we code user-defined iterables with classes, it's up to us to decide whether we will support a single active iteration or many. To achieve the multiple-iterator effect, iter simply needs to define a new stateful object for the iterator instead of returning self for each iterator request. For example, the class SkipObject in Example 30-3 defines an iterable object that skips every other item on iterations. Because its iterator object is created anew from a supplemental class for each iteration, it supports multiple active loops directly. Example 30-3. skipper.py When run, this example works like the earlier nested loops with built-in strings. Each active loop has its own position in the string because each obtains an independent iterator object that records its own state information:

```
$ python3 skipper.py a c e aa ac ae ca cc ce ea ec ee
```

By contrast, our earlier Squares class of Example 30-2 supports just one active iteration unless we call Squares again in nested loops to obtain new objects (else the outer for loop would run just once). Here, there is just one SkipObject iterable, with multiple iterator objects created from it.

中文翻译

前面我们顺带提到：可迭代对象产出的迭代器对象（带 next）可以定义成独立的类，携带自己的状态信息，从而更直接地支持在同一份数据上多个活跃迭代。为了更好地理解，

请看我们对一个内建类型（比如字符串）进行嵌套遍历时会发生什么：

```
>>> S = 'ace'>>> for x in S: ... for y in S: ...  
print(x + y, end='') aa ac ae ca cc ce ea ec ee
```

在这里，外层循环通过调用 `iter` 从字符串取一个迭代器，每个嵌套循环也照做一次，各拿一个独立的迭代器。因为每个活跃迭代器都有自己的状态信息，每个循环都能在字符串中维护自己的位置，与其他活跃循环互不干扰。而且我们不需要每次新建字符串或转成列表；同一个字符串对象本身就支持多次扫描。我们之前在第 14 章和第 20 章见过相关例子。例如，生成器函数与表达式，以及 `map`、`zip` 这类内建对象，被证明是单迭代器对象，因此只支持单个活跃扫描。相比之下，`range` 内建函数以及列表等其他内建类型，则支持多个拥有独立位置的活跃迭代器。当我们用类编写用户自定义可迭代对象时，是否支持单次活跃迭代还是多次，完全由我们决定。要实现多次迭代器效果，`iter` 只需为每次迭代器请求定义一个新的有状态对象，而不是返回 `self`。例如，Example 30-3 中的 `SkipObject` 类定义了一个可迭代对象，在迭代时跳过每隔一个的元素。因为它的迭代器对象每次迭代都由一个辅助类重新创建，所以它直接支持多个活跃循环。Example 30-3. `skipper.py` 运行时，这个示例的表现和前面内建字符串的嵌套循环一样。每个活跃循环在字符串中都有自己的位置，因为每个循环都获得了一个记录着自身状态信息的独立迭代器对象：

```
$ python3 skipper.py a c e aa  
ac ae ca cc ce ea ec ee
```

相比之下，我们之前 Example 30-2 的 `Squares` 类只支持一个活跃迭代——除非我们在嵌套循环里再次调用 `Squares` 取新对象（否则外层 `for` 循环只会跑一遍）。而这里只有一个 `SkipObject` 可迭代对象，由它创建出多个迭代器对象。

代码分析 (Example 30-3. skipper.py)

```
class SkipObject:
    def __init__(self, wrapped):          #
        self.wrapped = wrapped          #
    def __iter__(self):
        return SkipIterator(self.wrapped) #

class SkipIterator:
    def __init__(self, wrapped):
        self.wrapped = wrapped          #
        self.offset = 0                  #
    def __next__(self):
        if self.offset >= len(self.wrapped): #
            raise StopIteration
        else:
            item = self.wrapped[self.offset] #
            self.offset += 2                  # 2
            return item

if __name__ == '__main__':
    alpha = 'abcdef'
    skipper = SkipObject(alpha)          #
    I = iter(skipper)                    #
    print(next(I), next(I), next(I))     # 0, 2, 4
    for x in skipper:                    # for __iter__
        for y in skipper:                # for __iter...
            print(x + y, end=' ')        # offset
```

```
$ python3 skipper.py
a c e
aa ac ae ca cc ce ea ec ee
```

·解释器如何处理：外层 for x in skipper 调 iter 得到迭代器 A（其 offset 从 0 开始，步进 2）；内层 for y in skipper 再调 iter 得到迭代器 B（另一个独立对象，也从头开始）。A 和 B 是 SkipIterator 的两个不同实例，各自

持有独立的 offset——所以 A 停在'a' 时，B 可以从'a' 扫到'e'，互不污染。这就是"多游标"的全部秘密：每次 iter 都新造一个状态对象。

·对比 Squares：Squares.iter 返回 self，只有一个状态；SkipObject.iter 返回新对象，于是状态按需复制。两者行为差异完全由这一行决定。

·设计思想：可迭代对象（容器）+ 迭代器（游标）分离，正是 list/str 内建采用的结构；用户类按同一模式编码，即可获得同样的"多路并发遍历"能力。

深度理解

·核心概念：多迭代器 = "容器一份，游标多份"。iter 每次创建新的迭代器实例，每个实例独立维护推进位置，因此嵌套循环/并发遍历互不干扰。

·底层实现：字符串的 iter 返回新的 striterator（内含被迭代对象引用 + 当前位置索引）；GETITER/FORITER 每次执行都重新取一个游标。类的 Skipliterator 就是在 Python 层复刻这个 C 层结构。

·为什么这样设计：迭代器记录的"当前位置"是遍历性临时数据，不该污染容器自身；把临时状态放进独立对象，既支持多游标，又不破坏容器的广播用途。

·实际问题：数据库查询的多个游标并发读取同一结果集、UI 中多个列表控件同步遍历同一数据源——这类"同数据多处扫描"需求，正用这种架构解决（第 20 章也提到 zip/map 是单游标、range/list 是多游标）。

·初学者误区：①嵌套 for 遍历同一个"可迭代实例"却得到重复/异常结果——因为 iter 返回 self，内外层共享状态，外层把数据耗光了；②以为"可迭代对象被耗尽是必然"——其实只是你选择了单游标设计；改成"每次返回新迭代器"即可多扫。

30.6.5 类与切片相互比较 (Classes versus slices)

英文原文

As before, we could achieve similar results with built-in tools—for example, slicing with a third bound to skip items: >>> S='abcdef'>>> for x in S[::2]: ... for y in S[::2]: ... print(x + y, end='') # New objects on each iteration aa ac ae ca cc ce ea ec ee This isn't quite the same, though, for two reasons. First, each slice expression here will physically store the result list all at once in memory; iterables, on the other hand, produce just one value at a time, which can save substantial space and startup time for large result lists. Second, slices produce new objects, so we're not really iterating over the same object in multiple places here. To be closer to the class, we would need to make a single object to step across by slicing ahead of time: >>> S='abcdef'>>> S = S[::2] >>> S'ace'>>> for x in S: ... for y in S: ... print(x + y, end='') # Same object, new iterators aa ac ae ca cc ce ea ec ee This is more similar to our class-based solution, but it still stores the slice result in memory all at once (there is no generator form

of built-in slicing today), and it's only equivalent for this particular case of skipping every other item. Because user-defined iterables coded with classes can do anything a class can do, they are much more general than this example may imply. Though such generality is not required in all applications, user-defined iterables are a powerful tool—they allow us to make arbitrary objects look and feel like the other sequences and iterables we have met in this book. We could use this technique with a database object, for example, to support iterations over large database fetches, with multiple cursors into the same query result.

中文翻译

和以前一样，我们可以用内建工具实现类似的结果——例如用带第三个界（步长）的切片来跳过元素：

```
>>> S = 'abcdef'
>>> for x in S[::2]: ... for y in S[::2]: ... print(x + y, end='')
aa ac ae ca cc ce ea ec ee
```

不过这并不完全相同，原因有二。第一，这里的每个切片表达式都会把结果列表一次性物理存储在内存中；而可迭代对象一次只产生一个值，对很大的结果列表来说能节省可观的空间和启动时间。第二，切片会产生新对象，所以这里并不是在多个位置遍历同一个对象。要更贴近类的方案，我们需要提前切片，做出一个单一对象来跨步遍历：

```
>>> S = 'abcdef'
>>> S = S[::2]
>>> S
'ace'
>>> for x in S: ... for y in S: ... print(x + y, end='')
aa ac ae ca cc ce ea ec ee
```

这更接近我们基于类的方案，但它仍然把切片结果一次性存在内存里（今天的 Python 还没有内

建的切片生成器形式)，而且只在“每隔一个取一项”这一特定场景下等价。因为用类编写、用户自定义的可迭代对象能做到类能做到的一切，它们比这个例子所暗示的要通用得多。虽然并不是所有应用都需要这种通用性，但用户自定义可迭代对象是一个强大的工具——它让我们能让任意对象看起来、用起来都像本书遇到的其他序列和可迭代对象。例如，我们可以用这个技巧处理数据库对象，支持对大型数据库拉取结果的遍历，用多个游标指向同一份查询结果。

深度理解

·核心概念：切片是“物化”（一次性生成新容器），迭代器是“流式”（逐个产出）。两者外部观感可以一致，但内存与启动成本完全不同。

·底层实现：S[::2] 在 C 层 listslice/sequenceslice 里新分配一个列表并把全部元素复制进去；而迭代器 SkipObject 每次只持有当前下标与底层容器引用，几乎零额外分配。

·为什么这样设计：切片需要随机访问和重新索引，天然要求物化；迭代器面向“逐个消费”，天然省内存。这不是缺陷而是分工。

·实际问题：对“跳过取子集”这类简单需求，切片一行搞定；当结果集可能巨大（数据库查询、日志流），就绝不能切片——要用 iter 类实现“虚拟视图”（virtual view），按需从底层数据源取一行，避免把整个结果集装进内存。

·初学者误区：①以为 iter(X) 比 X[::] “功能弱”——其实流式正是它的优势；②以为类版本“多余”——当数据来自“一

次只能推进一位"的源（游标、套接字）时，根本没法切片，类方案是唯一选择。

30.6.6 编码替代方案：iter 结合 yield (Coding Alternative: iter Plus yield)

英文原文

Now for something more implicit—but potentially useful nonetheless. In some applications, it's possible to minimize coding requirements for user-defined iterables by combining the iter method we're exploring here and the yield generator function statement we studied in Chapter 20.

Because generator functions automatically save local-variable state and create required iterator methods, they fit this role well and complement the state retention and other utility we get from classes.

As a quick review, recall that any function that contains a yield statement is turned into a generator function. When called, it returns a new generator object with automatic retention of local scope and code position; an automatically created iter method that simply returns itself; and an automatically created next method that starts the function or resumes it where it last left off:

```
>> def gen(x):
```

```
... for i in range(x): yield i 2 >> G = gen(5) # Create a
generator with iter and next >> G.iter() is G # Both m
ethods exist on the same object True >> I = iter(G) #
Runs iter: generator returns itself >> next(I), next(I) #
Runs next (0, 1) >> list(gen(5)) # Iteration tools auto
matically run iter and next [0, 1, 4, 9, 16]
```

This is still true even if the generator function with a yield happens to be a method named iter. Whenever invoked by an iteration tool, such a method will return a new generator object with the requisite next. As an added bonus, generator functions coded as methods in classes have access to saved state in both instance attributes and local-scope variables.

For example, the class in Example 30-4 is equivalent to the initial Squares user-defined iterable we coded earlier in Example 30-2, but noticeably shorter (4 lines, for anyone counting).

Example 30-4. squaresyield.py As before, for loops and other iteration tools iterate through instances of this class automatically:

```
$ python3 >> from squaresyield import Squares >>
for i in Squares(1, 5): print(i, end=" ") 1 4 9 16 25 # Ru
ns iter, then next
```

And as always, we can also look under the hood to see how this actually works in iteration tools like for. Running our class instance through iter obtains the result of calling iter as usual. In this case, though, the re

sult is a generator object with an automatically created next of the same sort we always get when calling a generator function that contains a yield.

The only difference here is that the generator function is automatically called on iter. Invoking the result object's next interface produces results on demand:

```
>> S = Squares(1, 5) # Runs init: class saves instance state
>> S <squaresyield.Squares object at 0x109e30b90>
>> I = iter(S) # Runs iter: returns a generator
> I <generator object Squares.iter at 0x109ecb3e0>
>> next(I) 1
>> next(I) # Runs generator's next 4 ... etc ...
>> next(I) # Generator has both instance and local scope state
StopIteration
```

It may also help to notice that we could name the generator method something other than iter and call manually to iterate—Squares(...).gen(), for example. Using the iter name invoked automatically by iteration tools simply skips a manual attribute fetch and call step. Example 30-5 demos the idea.

Example 30-5. squaresyieldmanual.py The example also imports Example 30-4 to inherit its constructor. When run, its results are the same, but we must call the gen method explicitly to fetch an iterable—a step that iter automates and obviates:

```
$ python3
>> from squaresyieldmanual import Squares
>> for i in Squares(1, 5).gen(): print(i, end=") ... same results ...
>> S = Squares(1, 5)
>> I = iter(S.gen(
```

`>>> # Call generator manually for iterable/iterator >>> n
ext(l) ... same results ...`

Coding the generator as `iter` instead cuts out the middleperson in your code, though both schemes ultimately wind up creating a new generator object for each iteration: - With `iter`, iteration triggers `iter`, which returns a new generator with `next`. - Without `iter`, your code calls to make a generator, which returns itself for `iter`.

See Chapter 20 for more on `yield` and generators if this is puzzling, and compare it with the more explicit `next` version in Example 30-2 earlier. If you do, you'll notice that the `squaresyield.py` version is 4 lines shorter (7 versus 11, not counting whitespace).

In a sense, this scheme reduces class coding requirements much like the closure functions of Chapter 17, but in this case does so with a combination of functional and OOP techniques instead of an alternative to classes. For example, the generator method still leverages self attributes.

This may also seem like one too many levels of magic to some observers—it relies on both the iteration protocol and the object creation of generators, both of which are highly implicit (in contradiction of longstanding Python goals). Opinions aside, it's important to understand the non-`yield` flavor of class iterables too, because it's explicit, general, and sometimes broader

in scope.

Still, the iter/yield technique may prove effective in cases where it applies. It also comes with a substantial advantage—as the next section explains.

中文翻译

接下来是更隐式的方案——但可能同样有用。在某些应用中，可以把这里探讨的 iter 方法与第 20 章学习的 yield 生成器函数语句结合起来，从而把用户自定义可迭代对象的编码需求降到最低。因为生成器函数会自动保存局部变量状态并创建所需的迭代器方法，所以它们非常适合这个角色，并与类带给我们的状态持久化等能力相辅相成。快速回顾一下：任何包含 yield 语句的函数都会变成生成器函数。调用它时，会返回一个新的生成器对象，具有：自动保留的局部作用域和代码位置；一个自动创建、只是返回自身的 iter 方法；以及一个自动创建、启动函数或从上次停下的位置继续执行的 next 方法：

```
>>> def gen(x): ... for i in range(x):
yield i 2 >>> G = gen(5) # 创建一个带 iter 和 next 的生成器
>>> G.iter() is G # 两个方法在同一个对象上 True
>>> I = iter(G) # 运行 iter: 生成器返回自身
>>> next(I), next(I) # 运行 next (0, 1)
>>> list(gen(5)) # 迭代工具自动运行 iter 和 next [0, 1, 4, 9, 16]
```

即使这个含 yield 的生成器函数恰好是一个名为 iter 的方法，上面的结论依然成立。每当被迭代工具调用时，这样的方法都会返回一个带有必需 next 的新生成器对象。作为额外好处，作为类方法编写的生成器函数既可以访问实例属性中的状态，也可以访问局部作用域变量中的状态。例如，Example 30-4 中的类等价于我们之前在 Example 30-2 中编写的 Squares 用户自

定义可迭代对象，但明显更短（数一下的话只有 4 行）。Example 30-4. `squaresyield.py`和以前一样，`for` 循环和其他迭代工具会自动遍历这个类的实例：`$ python3 >>> from squaresyield import Squares >>> for i in Squares(1, 5): print(i, end=") 1 4 9 16 25` # 先运行 `iter`，再运行 `next` 而且一如既往，我们也能揭开引擎盖，看看它在 `for` 这类迭代工具中到底是如何工作的。照常把类实例过一遍 `iter`，会得到调用 `iter` 的结果。不过此时的结果是生成器对象，它带有一个自动创建的 `next`——和我们调用任何含 `yield` 的生成器函数时得到的完全相同。这里的唯一区别是：生成器函数是在 `iter` 调用时被自动调用的。调用这个结果对象的 `next` 接口，就会按需产出结果：`>>> S = Squares(1, 5) # 运行 init: 类保存实例状态>>> S <squaresyield.Squares object at 0x109e30b90> >>> I = iter(S) # 运行 iter: 返回一个生成器>>> I <generator object Squares.iter at 0x109ecb3e0> >>> next(I) 1 >>> next(I) # 运行生成器的 next 4...等等...>>> next(I) # 生成器同时拥有实例与局部作用域状态StopIteration 注意到这一点也有帮助：我们可以把生成器方法命名为 iter 以外的名字，然后手动调用来迭代——比如 Squares(...).gen()。而采用 iter 这个名字、由迭代工具自动调用，只是省去了手动获取属性和调用的步骤。Example 30-5 演示了这个想法。Example 30-5. squaresyieldmanual.py这个例子还导入了 Example 30-4，以继承它的构造器。运行时结果相同，但我们必须显式调用 gen 方法来取可迭代对象——而这正是 iter 自动完成、从而省去的一步：$ python3 >>> from squaresyieldmanual import Squares >>> for i in Squares(1, 5).gen(): print(i, end=")...结果相同...>>> S = Squ`

ares(1, 5) >>> l = iter(S.gen()) # 手动调用生成器来得到可迭代/迭代器>>> next(l)...结果相同...把生成器写成 iter，能在你的代码中省掉"中间人"；不过两种方案最终都会为每次迭代创建一个新的生成器对象：- 有 iter：迭代触发 iter，它返回一个带 next 的新生成器。- 没有 iter：你的代码要调用一次来制造生成器，生成器以自身作为 iter 的返回。如果对 yield 和生成器仍有疑惑，请参见第 20 章，并与前面 Example 30-2 更显式的 next 版本对比。对比后你会发现，squaresyield.py 版本短了 4 行（不算空白行，7 行对比 11 行）。从某种意义上说，这个方案削减了类的编码需求，很像第 17 章的闭包函数（closure function）——但这里是用函数式与 OOP 技术结合做到的，而不是作为类的替代。例如，生成器方法仍然利用了 self 属性。有些观察者可能会觉得这"魔法"层数太多——它同时依赖迭代协议和生成器的对象创建，两者都高度隐式（这有悖于 Python 长期坚持的目标）。看法暂且不论，理解类可迭代对象的非 yield 风格也很重要，因为它显式、通用，有时适用范围更广。即便如此，iter/yield 技术在适用的场合还是可能很有效。它还带来一个巨大的优势——下一节就会讲到。

代码分析 (Example 30-4. squaresyield.py 与 Example 30-5)

```
# Example 30-4: squares_yield.py
class Squares:
    def __init__(self, start, stop):    # __next__ /
        self.start = start
        self.stop = stop
    def __iter__(self):                # __iter__ + yield
        for value in range(self.start, self.stop + 1):
            yield value ** 2           #
```

```
# Example 30-5: squares_yield_manual.py
import squares_yield                                # __init__

class Squares(squares_yield.Squares):               # __iter__
    def gen(self):
        for value in range(self.start, self.stop + 1):
            yield value ** 2                          #
```

·解释器如何处理：iter(S) → GETITER 调用 S.iter()。但 iter 是含 yield 的生成器函数，所以调用它不会执行函数体，而是立即返回一个新生成的生成器对象 G。for 接着对 G 执行 FORITER：G.next() 从函数开头（帧的第一个 yield）处开始执行，每次 next 恢复帧、执行到下一个 yield 又挂起——局部变量 value、range 迭代器的当前位置都存在生成器的帧里。同时函数体里还引用了 self.start / self.stop，于是实例属性与局部作用域两个状态并存。

·对比 Example 30-2 手写版：手写版要在 next 里手动 if self.value == self.stop: raise StopIteration；generator 版这些全部由字节码的 FORITER+StopIteration 自动生成。这就是短 4 行的来源。

·isinstance 视角：iter(S) 的返回值类型是 generator，而非 Squares——for/next 完全不懂 Squares，它们只认迭代协议（next）。这也是“鸭子类型”在协议层的例证。

。

深度理解

·核心概念：iter+yield 是“类的外壳 + 生成器的内芯”：类提供结构化、状态、继承；生成器提供自动迭代协议与状态保留。两者组合是书写简洁可迭代类的标准范式。

·底层实现：生成器函数每次调用都会创建一个独立的 generator 对象及其 frame；帧在堆上，yield 挂起 = SAVE 帧状态，next 恢复 = LOAD。所以"每次 iter 都返回新生成器"天然保证了 30.6.7 讲的多迭代器能力。

·为什么这样设计：手写迭代类要把协议细节（next、StopIteration、状态自增）全部摊开，容易出错；生成器把这些全部内建，让开发者只关注"产出什么"，符合"简单优于复杂"。Lutz 也提醒：这依赖两个隐式机制，属于"锦上添花"而非"必学咒语"。

·实际问题：几乎一切现代代码（os.walk、requests 流式下载、pandas 分块读取）都用生成器实现；需要"既是容器又能迭代、还能带方法"时，把它包进类的 iter 是最佳折中。

·初学者误区：①以为 iter 必须 return self——用 yield 时它返回生成器，这是 modern 且更好的写法；②把 Squares(1,5).gen() 误以为返回列表——它是生成器对象，只有在被迭代时才逐步产出；③认为加了 yield 就是"改进了类"——显式/隐式的取舍要按任务权衡，隐式对调试不友好。

30.6.7 用 yield 实现多迭代器 (Multiple iterators with yield)

英文原文

Besides its code conciseness, the user-defined class iterable of the prior section based upon the iter/yield c

ombination has an important added bonus—it also supports multiple active iterators automatically. This naturally follows from the fact that each call to `iter` is a new call to a generator function, which returns a new generator with its own copy of the local scope for state retention.

Using Example 30-4 again:

```
$ python3 >> from squaresyield import Squares >>
S = Squares(1, 5) # Using the iter/yield Squares >> I
= iter(S) >> next(I); next(I) 1 4 >> K = iter(S) # With y
ield, multiple iterators automatic >> next(K) 1 >> ne
xt(I) # I is independent of K: own local state 9
```

Although generator functions are single-scan iterables by nature, the implicit calls to `iter` in iteration tools make new generators supporting new independent scans:

```
>> S = Squares(1, 3) >> for i in S: # Each "for" calls i
ter
... for j in S: ... print(f'{i}:{j}', end='') 1:1 1:4 1:9 4:1 4:4
4:9 9:1 9:4 9:9
```

To do the same without `yield` requires a supplemental class that stores iterator state explicitly and manually, using techniques of the preceding section. Per Example 30-6, this grows to 15 lines: 8 more than with `yield`.

Example 30-6. `squaresnonyield.py` This works the same as the `yield multiscan` version, but with more—and more explicit—code:

```
$ python3 >> from squaresnonyield import Squares
>> for i in Squares(1, 5): print(i, end=" ") 1 4 9 16 25 >
> S = Squares(1, 5) >> I = iter(S) >> next(I); next(I) 1
4 >> K = iter(S) # Multiple iterators without yield >>
next(K) 1 >> next(I) 9 >> S = Squares(1, 3) >> for i in
S: # Each "for" calls iter
... for j in S: ... print(f'{i}:{j}', end=" ") 1:1 1:4 1:9 4:1 4:4
4:9 9:1 9:4 9:9
```

Finally, the generator-based approach could similarly remove the need for an extra iterator class in the prior item-skipper example, `skipper.py` of Example 30-3, thanks to its automatic methods and local variable state retention. Example 30-7 codes the `mod`, which checks in at 9 lines versus the original's 16, sans the original's self-test.

Example 30-7. `skipperyield.py` This works the same as the non-`yield multiscan` version, but with less—and less explicit—code:

```
$ python3 >> from skipperyield import SkipObject >
> skipper = SkipObject('abcdef') >> I = iter(skipper)
>> next(I); next(I); next(I)'a''c''e'>> for x in skipper: #
Each "for" calls iter: new auto generator
... for y in skipper: ... print(x + y, end=" ") aa ac ae ca cc
```

Of course, these are all artificial examples that could be replaced with simpler tools like comprehensions, and their code may or may not scale up in kind to more realistic tasks. Study these alternatives to see how they compare. As so often in programming, the best tool for the job will likely be the best tool for your job.

中文翻译

除了代码简洁之外，上一节基于 `iter/yield` 组合的用户自定义类可迭代对象还有一个重要的额外好处——它自动支持多个活跃迭代器。这一点自然来自如下事实：每次调用 `iter` 都是对生成器函数的一次新调用，而它会返回一个新生成器，包含自己的一份局部作用域副本用于状态保持。再次使用 Example 30-4：

```
$ python3 >>> from squaresyield import Squares >>> S = Squares(1, 5) # 使用 iter/yield 的 Squares >>> I = iter(S) >>> next(I); next(I) 1 4 >>> K = iter(S) # 有了 yield，多迭代器自动成立 >>> next(K) 1 >>> next(I) # I 与 K 相互独立：各有自己的局部状态 9
```

虽然生成器函数本质上就是单次扫描的可迭代对象，但迭代工具中隐式的 `iter` 调用会制造出支持全新独立扫描的新生成器：

```
>>> S = Squares(1, 3) >>> for i in S: # 每个 "for" 都调用 iter ... for j in S: ... print(f'{i}:{j}', end=" ") 1: 1 1:4 1:9 4:1 4:4 4:9 9:1 9:4 9:9
```

不用 `yield` 要做到同样的事，需要一个补充类来显式、手动地存储迭代器状态——也就是上一节的技术。按 Example 30-6 的写法，这会长到 15 行：比用 `yield` 多 8 行。Example 30-6. `squaresno`

nyield.py 它的效果与 yield 多扫描版本相同，但代码更多、也更显式：

```
$ python3 >>> from squaresnonyield import Squares >>> for i in Squares(1, 5): print(i, end=" ") 1 4 9 16 25 >>> S = Squares(1, 5) >>> I = iter(S) >>> next(I); next(I) 1 4 >>> K = iter(S) # 不用 yield 的多迭代器 >>> next(K) 1 >>> next(I) 9 >>> S = Squares(1, 3) >>> for i in S: # 每个 "for" 都调用 iter ... for j in S: ... print(f'{{i}}:{{j}}', end=" ") 1:1 1:4 1:9 4:1 4:4 4:9 9:1 9:4 9:9 最后，基于生成器的方法还能去掉前面跳项示例（Example 30-3 的 skipper.py）里额外的迭代器类，这要归功于它自动的方法和局部变量状态保持。Example 30-7 就编码了这个修改版，算下来是 9 行——相比之下原版是 16 行，还不算原版的自测试。Example 30-7. skipperyield.py 它的效果与非 yield 多扫描版本相同，但代码更少、也更隐式：
```

```
$ python3 >>> from skipperyield import SkipObject >>> skipper = SkipObject('abcdef') >>> I = iter(skipper) >>> next(I); next(I); next(I) 'a' 'c' 'e' >>> for x in skipper: # 每个 "for" 都调用 iter: 新的自动生成器... for y in skipper: ... print(x + y, end=" ") aa ac ae ca cc ce ea ec ee 当然，这些都是可以替换成推导式等更简单工具的刻意示例，它们的代码未必能按原样扩展到更现实的任务。研究这些替代方案，看看它们各自的优缺点。编程中经常如此：最适合这项工作任务的工具，很可能就是最适合你工作的工具。
```

代码分析 (Example 30-6. squaresnonyield.py 与 Example 30-7. skipperyield.py)

```
# Example 30-6: squares_nonyield.py — yield
class Squares:
    def __init__(self, start, stop):    # yield
        self.start = start            #
        self.stop = stop
    def __iter__(self):
        return SquaresIter(self.start, self.stop)    #

class SquaresIter:
    def __init__(self, start, stop):
        self.value = start - 1
        self.stop = stop
    def __next__(self):
        if self.value == self.stop:
            raise StopIteration
        self.value += 1
        return self.value ** 2
```

```
# Example 30-7: skipper_yield.py — __iter__ + yield
class SkipObject:
    def __init__(self, wrapped):        #
        self.wrapped = wrapped         #
    def __iter__(self):
        offset = 0                      #
        while offset < len(self.wrapped):
            item = self.wrapped[offset]
            offset += 2
            yield item                   #
```

·解释器如何处理：squaresnonyield 中，iter(S) 调 iter
 → return SquaresIter(...), 每次制造崭新的迭代器对象
 ; SquaresIter.next 用实例字段记录推进状态。skipperyield 更巧妙：iter(skipper) 调 iter → 调生成器函数 → 创建新生成器，其 offset 是局部变量，被存在生成器帧里，所以每路迭代器都持有自己独立的 offset 副本——无需再为"游标"写类。

·两条路的本质：手写版（30-6）与生成器版（30-7）在协议层面完全等价（GETITER 返回带 next 的对象）；差别只在“状态放哪”——手写版放实例属性，生成器版放帧（frame）。

·行的代价：15 行 vs 7 行（Squares 场景）；隐式的代码少、但不易断点调试。Lutz 的结语很实在：没有银弹——简单的活交给推导式，中等交生成器，复杂的上类。

深度理解

·核心概念：生成器 = “活的单例迭代器”，但 iter 每次调用都产生新生成器 → 多扫描自动达成。这是“对象级的流式状态隔离”，比手写辅助类更省代码。

·底层实现：每个生成器调用在 C 层分配新的 PyGenObject 与 PyFrameObject；帧里存有局部变量表与 flastit（最后执行指令索引）。两路迭代各自持独立帧，所以进度互不相干。

·为什么这样设计：状态隔离是生成器的内建属性（帧即状态），把“多迭代器”从“需设计”变成“免费赠送”，大大降低协议落地门槛。

·实际问题：流式读取文件同时做“正向遍历 + 反向抽查”，或一个 HTTP 分页对象在多处独立翻页——iter+yield 都能直接支持。

·初学者误区：①以为生成器函数“本身就是多遍可迭代”——单个生成器对象仍是单遍（iter(G) is G），多遍来自“每次 iter 新建生成器”；②混淆 next(I) 与 next(K) 的状态——它们毫不相干；③过度设计：仅仅为了嵌套遍历一个

字符串就手写 16 行类——先想想推导式/切片够不够。

30.7 成员测试: `contains`、`iter` 与 `getitem` (Membership)

英文原文

The iteration story is even richer than told thus far. Operator overloading is often layered: classes may provide specific methods or more general alternatives used as fallback options. We've already seen this for general iteration (`iter` or else `getitem`), and will encounter another example ahead when we meet Boolean values.

Also in the iterations domain, classes can implement the `in` membership operator as an iteration, using either the `iter` or `getitem` methods. To support more specific membership, though, classes may code a `contains` method—when present, this method is preferred over `iter`, which is preferred over `getitem`. The `contains` method should define membership as applying to keys for a mapping (and can use quick lookups) and as a search for sequences.

Consider the class `Iters` in Example 30-8. It codes all three methods and tests membership and various iteration tools applied to an instance. To demo, its methods print trace messages when called, its self-test co

de cycles through tests coded with match to call the m out, and its methods' traces show up before those of its tests.

Example 30-8. contains.py As is, the class in this file has an iter that supports only a single active scan at any point in time (e.g., nested loops won't work) because each iteration attempt resets the scan cursor to the front. Now that you know about yield in iteration methods, you should be able to tell that Example 30-9 is equivalent but allows multiple active scans—and judge for yourself whether its more implicit nature is worth the nested-scan support (and 5 lines shaved).

Example 30-9. contains-yield.py When either version of this file runs, its output is as follows—the specific contains intercepts membership, the general iter catches other iteration tools such that next (whether explicitly coded or implied by yield) is called repeatedly, and getitem is never called:

```
$ python3 contains.py In @contains True For @iter
@next 1 | @next 2 | @next 3 | @next 4 | @next Comp
@iter @next @next @next @next @next [1, 4, 9, 16]
Map @iter @next @next @next @next @next ['0b1',
0b10','0b11','0b100'] Manual @iter @next 1 | @next
2 | @next 3 | @next 4 | @next
```

Watch, though, what happens to this code's output if we comment out its contains method—membership is now routed to the general iter instead (add triple q

quotes above and below the method to test live):

```
$ python3 contains.py In @iter @next @next @next
True For @iter @next 1 | @next 2 | @next 3 | @next
4 | @next Comp @iter @next @next @next @next @
next [1, 4, 9, 16] Map @iter @next @next @next @ne
xt @next ['0b1','0b10','0b11','0b100'] Manual @iter
@next 1 | @next 2 | @next 3 | @next 4 | @next
```

And finally, here is the output if both `contains` and `iter` are commented out—the indexing `getitem` fallback is called with successively higher indexes until it raises `IndexError`, for membership and other iteration tools:

```
$ python3 contains.py In @get[0] @get[1] @get[2] T
rue For @get[0] 1 | @get[1] 2 | @get[2] 3 | @get[3] 4
| @get[4] Comp @get[0] @get[1] @get[2] @get[3] @
get[4] [1, 4, 9, 16] Map @get[0] @get[1] @get[2] @g
et[3] @get[4] ['0b1','0b10','0b11','0b100'] Manual @g
et[0] 1 | @get[1] 2 | @get[2] 3 | @get[3] 4 | @get[4]
```

As we've seen, the `getitem` method does other work too: besides iterations, it also intercepts explicit indexing as well as slicing. Slice expressions trigger `getitem` with a slice object containing bounds, both for built-in types and user-defined classes, so slicing is automatic in our class. With `iter` enabled or not:

```
$ python3 >> from contains import Iters >> X = Iter
s('hack') >> X[0] # Indexing: getitem(0)
```

```
@get[0]'h'>> X[1:] # Slicing: getitem(slice(...))
```

```
@get[slice(1, None, None)]'ack'>> X[:-1]
```

@get[slice(None, -1, None)]'hac' Iterations, though, are more selective—we get the first of the following if iter is still commented out and the second if it's not (be sure to restart or reload after the file mod either way):

```
>> list(X) # @get[0] @get[1] @get[2] @get[3] @get[4] ['h','a','c','k'] >> list(X) # @iter @next @next @next @next @next ['h','a','c','k']
```

In more realistic iteration use cases that are not sequence-oriented, though, the iter method may be easier to write since it must not manage an integer index, and contains allows for membership optimization as a special case.

While iteration is a rich topic, it's time to move on to the next stop on our overloading tour.

中文翻译

迭代的故事比目前讲的还要丰富。运算符重载常常是分层的 (layered)：类既可以提供专门的方法，也可以提供更通用的替代方案作为回退选项。这一点我们在通用迭代上已经看到了 (iter 否则 getitem)，接下来在遇到布尔值 (Boolean values) 时还会看到另一个例子。同样在迭代领域，类可以用 iter 或 getitem 方法，把 in 成员运算符当作迭代来实现。不过，要支持更专门的成员测试，类可以编写一个 contains 方法——一旦存在，这个方法的优先级高于 it

er, 而 iter 又高于 getitem。contains 方法应当这样定义成员语义：对映射 (mapping) 而言是键 (key) 的查找 (可以利用快速查找)，对序列 (sequence) 而言是元素的搜索。请看 Example 30-8 中的 Iters 类。它编写了全部三个方法，并测试了应用于一个实例的成员测试和多种迭代工具。为了演示，它的方法在被调用时会打印追踪信息，它的自测代码用 match 编写的测试循环把它们逐一叫出来，方法们的追踪输出显示在测试输出之前。Example 30-8. contains.py 按现状，这个文件里的类有一个只支持单次活跃扫描的 iter (例如嵌套循环不行)，因为每次迭代尝试都会把扫描游标重置到开头。既然你已经了解了迭代方法中的 yield，你应该能看出来：Example 30-9 是等价的，但它允许多次活跃扫描——至于它更隐式的风格是否值得换取嵌套扫描支持 (外加省下 5 行)，由你自己判断。Example 30-9. contains-yield.py 无论运行哪个版本的文件，输出如下——专门的 contains 拦截成员测试，通用的 iter 接住其他迭代工具，使得 next (无论是显式编写还是由 yield 隐含) 被反复调用，而 getitem 从未被调用：\$ python3 contains.py In @contains True For @iter @next 1 | @next 2 | @next 3 | @next 4 | @next Comp @iter @next @next @next @next [1, 4, 9, 16] Map @iter @next @next @next @next ['0b1', '0b10', '0b11', '0b100'] Manual @iter @next 1 | @next 2 | @next 3 | @next 4 | @next 不过请注意：如果注释掉它的 contains 方法，代码输出会发生什么变化——成员测试现在被路由到通用的 iter (可以在方法上下加三引号来实际测试)：\$ python3 contains.py In @iter @next @next @next True For @iter @next 1 | @next 2 | @next 3 | @next 4 | @

next Comp @iter @next @next @next @next @next [1, 4, 9, 16] Map @iter @next @next @next @next @next ['0b1','0b10','0b11','0b100'] Manual @iter @next 1 | @next 2 | @next 3 | @next 4 | @next 最后，如果 contains 和 iter 都被注释掉，输出如下——索引回退 getitem 会以越来越高的索引被调用，直到它抛出 IndexError，无论对成员测试还是其他迭代工具都是如此：

```

$ python 3 contains.py In @get[0] @get[1] @get[2] True For @get[0] 1 | @get[1] 2 | @get[2] 3 | @get[3] 4 | @get[4] Comp @get[0] @get[1] @get[2] @get[3] @get[4] [1, 4, 9, 16] Map @get[0] @get[1] @get[2] @get[3] @get[4] ['0b1','0b10','0b11','0b100'] Manual @get[0] 1 | @get[1] 2 | @get[2] 3 | @get[3] 4 | @get[4]

```

正如我们所见，getitem 方法还做其他工作：除了迭代，它也拦截显式索引和切片。切片表达式会带着一个含边界的 slice 对象触发 getitem——对内建类型和用户自定义类都是如此，所以我们的类里切片是自动可用的，无论 iter 是否启用：

```

$ python3 >>> from contains import Iters >>> X = Iters('hack') >>> X[0] # 索引: getitem(0) @get[0]'h'>>> X[1:] # 切片: getitem(slice(...)) @get[slice(1, None, None)]'ack'>>> X[:-1] @get[slice(None, -1, None)]'ha'

```

不过迭代要更挑剔——如果 iter 仍然被注释掉，我们得到下面第一种输出；没有注释则得到第二种（改文件后无论如何记得重启或重新加载）：

```

>>> list(X) # @get[0] @get[1] @get[2] @get[3] @get[4] ['h','a','c','k'] >>> list(X) # @iter @next @next @next @next @next ['h','a','c','k']

```

不过在更现实、非面向序列的迭代用例中，iter 方法可能更容易编写，因为它不必管理整数索引；而 contains

则把成员测试优化作为一个特例来支持。迭代是个内容丰富的话题，但到了该继续重载之旅下一站的时候了。

代码分析 (Example 30-8. contains.py 与 Example 30-9. contains-yield.py)

```
# Example 30-8: contains.py
def trace(msg, end=''):
    print(f'{msg} ', end=end)      #

class Iters:
    def __init__(self, value):
        self.data = value
    def __getitem__(self, i):      #
        trace(f'@get[{i}]')      #
        return self.data[i]
    def __iter__(self):           #
        trace('@iter')           #
        self.ix = 0
        return self
    def __next__(self):
        trace('@next')
        if self.ix == len(self.data): raise StopIteration
        item = self.data[self.ix]
        self.ix += 1
        return item
    def __contains__(self, x):     # 'in'
        trace('@contains')
        return x in self.data

def self_test(Iters):
    X = Iters([1, 2, 3, 4])      #
    tests = 'In', 'For', 'Comp', 'Map', 'Manual'
    for test in tests:
        trace(test.ljust(max(map(len, tests)) + 1))
        match test:
```



```

        case 'In':
            trace(3 in X)          #
        case 'For':
            for i in X:            # for
                trace(i, end='| ')
        case 'Comp':
            trace([i ** 2 for i in X]) #
        case 'Map':
            trace(list(map(bin, X)))
        case 'Manual':
            I = iter(X)            #
            while True:
                try:
                    trace(next(I), end='| ')
                except StopIteration:
                    break
    print()

if __name__ == '__main__': self_test(Iterators) # Iterators

```

```

# Example 30-9: contains-yield.py — yield __iter__
from contains import *
class ItersYield(Iterators):
    def __iter__(self):          #
        trace('@iter @next')    #
        for x in self.data:      #
            yield x
        trace('@next')
if __name__ == '__main__': self_test(IterYield)

```

·解释器如何处理（三态回退链）：

·全量定义时：3 in X 编译为 CONTAINSOP，虚拟机先查 contains → 命中，直接调用它，输出 In @contains True；for/推导式/map/手动迭代则走 GETITER → iter (@iter) → 反复 next (@next)。getitem 完全休息。

- 注释掉 `contains`: `in` 找不到 `contains`, 回退到 `iter`: 迭代所有元素逐个比较, 直到找到 3 (此时已产出 3 个元素) ——所以 `@iter @next @next @next True`。
- 再注释掉 `iter`: `GETITER` 回退到 `getitem`, 从 `@get[0]` 逐项取到 `@get[4]` (越界抛 `IndexError`) 才停——成员测试因此走了 3 次索引, `For` 走了 5 次。
- 优先级总结 (可背诵): `contains > iter > getitem`; 而切片与显式索引只由 `getitem` 负责, 与迭代优先级无关。
- 设计思想: 分层 = "专门方法提供最优实现, 通用方法兜底"。映射用 `contains` 可 $O(1)$ 查键 (哈希), 序列用 `iter` 线性扫描; 什么都不写也能靠 `getitem` 工作——优雅的渐进增强。

深度理解

- 核心概念: `in` 运算符是"优先级链"的典型示范——越专门的方法越优先: `contains` (专为 `in`) $\rightarrow$ `iter` (遍历找) $\rightarrow$ `getitem` (索引找)。其他内建运算 (如 `print` 的 `str/rrepr`、真值的 `bool/len`、`+=` 的 `iadd/add`) 都是同一套"分层回退"思想的产物。
- 底层实现: `CONTAINSOP` 字节码依次探测 `sqcontains` (`contains` 的 C 槽位)、`tpiter (iter)`、`sqitem (getitem)`, 逐级回退; `PySequenceContains` 是这条链的 C 实现。每次回退都有明确的性能含义: 哈希查找 $O(1) \rightarrow$ 迭代 $O(n)$ 。
- 为什么这样设计: 让类作者按"需求与成本"自由取舍——只需最便宜的接口就能融入语言; 框架 (如 `collections.a`

bc.Mapping) 也借此定义清晰的协议层。

·实际问题: set/dict 靠 contains 哈希命中; 自定义数据库表对象让 key in table 触发 SQL 查询而不是全表遍历——这正是 contains 的"快速查找"语义。

·初学者误区: ①以为"定义了 iter 就万事大吉"——in 还有 contains 这层优先接口, 不定义会退化为 $O(n)$ 扫描; ②以为 getitem 会被 for 优先使用——不, iter 存在时它根本不露面; ③在 getitem 里没做越界抛 IndexError, 导致回退链失效、in 结果错乱。

30.8 属性访问: getattr 和 setattr (Attribute Access)

英文原文

In Python, classes can also intercept basic attribute access (a.k.a. qualification) when needed or useful. Specifically, for an object created from a class, the dot operator expression `object.attribute` can be implemented by your code too, for reference, assignment, and deletion contexts. We explored a limited example in this category which rerouted attribute fetches in the tutorial of Chapter 28, but will review and expand on the topic here.

中文翻译

在 Python 中，类还可以在需要或有用的时候拦截基本的属性访问（又称限定名访问，qualification）。具体来说，对于从类创建的对象，点运算符表达式 `object.attribute` 的引用、赋值和删除三种场景，也都可以由你的代码来实现。我们在第 28 章的教程里已经探索过一个重路由属性获取的有限示例，但这里将复习并展开这个话题。

深度理解

- 核心概念：属性访问三件套——`obj.attr`（读）、`obj.attr = v`（写）、`del obj.attr`（删）——分别对应 `getattr/getattribute`、`setattr`、`delattr` 三个钩子。类借此“接管”成员访问。
- 底层实现：CPython 中属性访问编译为 `LOADATTR/STOREATTR/DELETEATTR` 字节码；执行时走 `tpgetattr/tpsetattr` 槽位，先查实例 dict、再查类型 MRO，找不到时才轮到 `getattr`（详见下节）。
- 为什么这样设计：把“属性”从“固定槽位”升级为“可编程的入口”，支撑代理（proxy）、委托（delegation）、动态计算属性、只读/私有模拟等一大批设计模式。
- 实际问题：ORM 模型的懒加载字段（`user.profile` 首次访问才查库）、Mock 框架自动生成任意属性、RPC 客户端把属性访问翻译成远程调用——全建立在这组钩子上。
- 初学者误区：把 `getattr` 与 `getattribute` 混为一谈（详见 30.8.3）；或以为“拦截了 `setattr` 就能拦截 `self.dict`”

写入"——dict 键赋值是绕过钩子的后门。

30.8.1 属性引用 (Attribute Reference)

英文原文

The `getattr` method intercepts attribute references. It's called with the attribute name as a string whenever you try to qualify an instance with an undefined (non-existent) attribute name. It is not called if Python can find the attribute using its inheritance tree search procedure. Because of its behavior, `getattr` is useful as a hook for responding to attribute requests in a generic fashion. It's commonly used to delegate calls to embedded (or "wrapped") objects from a proxy controller object—of the sort introduced in Chapter 28's introduction to delegation. This method can also be used to adapt classes to an interface or add accessors for data attributes after the fact—logic in a method that validates or computes an attribute after it's already being used with simple dot notation (possibly after a rename of the original). The basic mechanism underlying these goals is straightforward—the following class catches attribute references, computing the value for one dynamically, and triggering an error for others unsupported with the `raise` statement described earlier in this chapter for iterators (and again, fully covered in Part VII):

```
>>> class Empty: ... def getattr(self, attrname): ... # On self.undefined ... if attrname == 'age': .
```

```
.. return 40 ... else: ... raise AttributeError(attrname) >
>> X = Empty() >>> X.age # Becomes X.getattr('age'
) 40 >>> X.name # Unsupported attribute... error tex
t omitted ...AttributeError: name Here, the Empty clas
s and its instance X have no real attributes of their o
wn, so the access to X.age gets routed to the getattr
method; self is assigned the instance (X), and attrnam
e is assigned the undefined attribute name string ('a
ge'). The class makes age look like a real attribute by
returning a real value as the result of the X.age qualif
ication expression (40). In effect, age becomes a dyna
mically computed attribute—its value is formed by ru
nning code, not fetching an object. For attributes tha
t the class doesn't know how to handle, getattr raises
the built-in AttributeError exception to tell Python th
at these are bona fide undefined names; asking for X.
name triggers the error. You'll see getattr again when
we explore delegation and properties at work in the
next two chapters; let's move on to related tools here
.
```

中文翻译

getattr 方法拦截属性引用。每当你试图用一个未定义（不存在）的属性名对实例做限定访问时，它都会带着一个字符串形式的属性名被调用。如果 Python 能用它的继承树搜索过程找到该属性，就不会调用它。正因为这种行为，getattr 是一个很有用的钩子，可以用通用方式响应属性请求。它常被用来把调用委托（delegate）给被代理控制器对象嵌

入（或"包裹"）的对象——就是第 28 章介绍委托（delegation）时提到的那类东西。这个方法还可以用来让类适配某个接口，或事后为数据属性添加访问器（accessor）——在属性已经用简单点号语法被使用之后（可能是在原属性改名之后），用一个方法里的逻辑去校验或计算该属性。这些目标背后的基本机制很直白——下面的类捕获属性引用：为其中一个属性动态计算值，对不支持的属性用 raise 语句触发错误（raise 在本章前面的迭代器中提过，第七部分会完整覆盖）：

```
>>> class Empty: ... def getattr(self, attrname): ... # 在 self.undefined 时调用... if attrname == 'age': ... return 40 ... else: ... raise AttributeError(attrname) >>> X = Empty() >>> X.age # 变成 X.getattr('age') 40 >>> X.name # 不支持的属性...错误文本省略...AttributeError: name 在这里，Empty 类和它的实例 X 都没有任何真正的自有属性，所以 X.age 的访问被路由到 getattr 方法；self 被赋值为实例（X），attrname 被赋值为那个未定义属性名的字符串（'age'）。这个类让 age 看起来像一个真正的属性，方法是：把真实的值（40）作为 X.age 限定表达式的返回结果。实际上，age 变成了一个动态计算属性——它的值来自运行代码，而不是取现成的对象。对于类不知道如何处理的属性，getattr 抛出内建的 AttributeError 异常，告诉 Python 这些是货真价实的未定义名字；访问 X.name 就会触发这个错误。下一章我们探索委托和 property 属性时会再次见到 getattr；这里先继续看相关工具。
```

代码分析

```
>>> class Empty:
...     def __getattr__(self, attrname):
...         # self.undefined
...         if attrname == 'age':
...             return 40
...         else:
...             raise AttributeError(attrname)
>>> X = Empty()
>>> X.age                                # X.__getattr__('age')
40
```

·解释器如何处理：X.age 编译为 LOADATTR 字节码。虚拟机在 X.dict（实例命名空间）中找 age——没有；再到 type(X) 的 MRO 上找——Empty 类也没有 age 属性；在常规查找全部落空后，解释器才检查类上是否有 getattrib，找到后调用 Empty.getattr(X,'age')。方法返回 40，成为整个表达式的值。X.name 同理进入 getattr，但方法内 raise AttributeError('name')——解释器看到的是"真·未定义"，于是原样抛出。

·关键细节：getattr 是"最后一步"——只在常规查找失败后才触发。因此 X.dict、X.class 这类本来就存在的属性不会经过它；它天然只处理"孤儿属性"。

深度理解

·核心概念：getattr(self, name) 是"未定义属性"的兜底处理器——参数 name 永远是字符串。它让"属性"变成可计算的、可委托的、可校验的逻辑入口。

·底层实现：LOADATTR 的 C 实现路径：实例 dict → 类 MRO（含父类）→ 描述符协议 → getattr 兜底。这一顺序决定了 getattr 永远不会抢占真实属性。

·为什么这样设计：委托（delegation）是"组合优于继承"的实现基础——控制器对象把不认识的一切属性请求转发给内嵌对象，从而"替别人做事却长得像自己"。

·实际问题：API 兼容层（旧接口名映射到新实现）、ORM 懒加载、Mock 测试替身、配置对象的任意键访问——`getattr` 一钩多用。

·初学者误区：①在 `getattr` 里访问同类未定义属性（如 `self.foo` 且 `foo` 不存在）会递归炸栈——这是最常见的 `RecursionError` 来源；②忘记 `raise AttributeError` 而返回 `None`，会让 `hasattr()` 永远返回 `True`、IDE 提示错乱；③以为 `getattr` 拦截所有属性——它只拦截"找不到的"。

30.8.2 属性赋值与删除 (Attribute Assignment and Deletion)

英文原文

In the same department, the `setattr` intercepts all attribute assignments. If this method is defined or inherited, `self.attr = value` becomes `self.setattr('attr', value)`. Like `getattr`, this allows your class to catch attribute changes and validate or transform as desired. This method is a bit trickier to use, though, because assigning to any self attributes within `setattr` calls `setattr` again, potentially causing an infinite recursion loop (and a fairly quick stack overflow exception!). In fact, this applies to all self attribute assignments anywhere in the class—all are routed to `setattr`, even those in other

r methods, and those to names other than that which may have triggered setattr in the first place. So be warned: this catches all attribute assignments. If you wish to use this method, you can avoid loops by coding instance attribute assignments as assignments to attribute dictionary keys. That is, use `self.dict['name'] = x`, not `self.name = x`; because you're not assigning to dict itself, this avoids the loop:

```
>>> class Accesscontrol: ... def setattr(self, attr, value): ... if attr == 'age': ... self.dict[attr] = value + 10 # Not self.name=val or setattr ... else: ... raise AttributeError(attr + ' not allowed') >>> X = Accesscontrol() >>> X.age = 40 # Becomes X.setattr('age', 40) >>> X.age # Found in dict as usual 50

>>> X.name = 'Pat' # Unsupported attribute... text omitted ... AttributeError: name not allowed
```

If you change the dict assignment within this class to either of the following, it triggers the infinite recursion loop and exception—both dot notation and its setattr built-in function equivalent (the assignment analog of getattr) fail when age is assigned outside the class:

```
self.age = value + 10 # Loops! setattr(self, attr, value + 10) # Loops! (attr is 'age')
```

An assignment to any other self attribute within the class triggers a recursive setattr call too, though in this class ends less dramatically in the manual Attribute

Error exception:

`self.other = 99 # Recurs + fails, but doesn't loop` It's also possible to avoid recursive loops in a class that uses `setattr` by rerouting any attribute assignments to a higher superclass with a call, instead of assigning keys in dict:

`object setattr(self, attr, value + 10) # OK: doesn't loop (preview)`

This uses an explicit-class call to the implied object's superclass above all topmost classes (and has a `super().setattr(...)` equivalent sans `self` per the sidebar "The super Alternative"). In fact, this alternative may be required in some classes per the upcoming note, though this is rare in practice. A third attribute-management method, `delattr`, is passed the attribute name string and invoked on all attribute deletions (i.e., `del object.attr`). Like `setattr`, it must avoid recursive loops by running attribute deletions within the class using dict, or rerouting them to a superclass. NOTE — Attribute outliers: Preceding chapters mentioned that attributes coded with advanced class tools such as slots and properties are not physically stored in the instance's dict namespace dictionary—and slots may even preclude a dict altogether. As noted, `dir` and `getattr` might be needed for listing and fetching attributes in classes using these tools, but assignment is similarly impacted: to support such "virtual" attributes, `setattr` may need

d to use the object.setattr scheme shown here, not self.dict indexing. You'll learn much more about these attribute tools in upcoming chapters.

中文翻译

同一部门里，setattr 拦截所有属性赋值。如果定义或继承了该方法，self.attr = value 就变成了 self.setattr('attr', value)。和 getattr 一样，它让你的类能够捕获属性变更，并按需校验或转换。不过这个方法用起来有点棘手：因为在 setattr 内部对任何 self 属性赋值都会再次调用 setattr，可能造成无限递归循环（而且会相当快地栈溢出异常！）。事实上，这适用于类中任何地方对 self 属性的所有赋值——它们全部会被路由到 setattr，包括其他方法里的赋值，也包括最初可能并没有触发 setattr 的其他名字的赋值。所以要警告你：它捕获所有属性赋值。如果你想用这个方法，可以通过把实例属性赋值写成对属性字典键的赋值来避免循环。也就是说，用 self.dict['name'] = x，而不是 self.name = x；因为你没有给 dict 本身赋值，这就绕开了循环：

```
>>> class Accesscontrol: ... def setattr(self, attr, value): ... if attr == 'age': ... self.dict[attr] = value + 10 # 不要用 self.name=val 或 setattr ... else: ... raise AttributeError(attr + ' not allowed') >>> X = Accesscontrol() >>> X.age = 40 # 变成 X.setattr('age', 40) >>> X.age # 照常 dict 中找到 50 >>> X.name = 'Pat' # 不支持的属性...文本省略...AttributeError: name not allowed 如果你把类里的 dict 赋值改成下面任意一种写法，都会触发无限递归循环和异常——点号记法与它的内建函数等价物 setattr (getattr 的赋值版本) 在类外给 age 赋值时都会失败：
```

`self.age = value + 10 # 死循环! setattr(self, attr, value + 10) # 死循环! (attr 是'age')` 在类内对任何其他 `self` 属性的赋值也会触发递归的 `setattr` 调用, 只是在这个类里不会循环到栈溢出, 而是以手动抛出的 `AttributeError` 异常收场: `self.other = 99 # 递归+失败, 但不会死循环` 在使用 `setattr` 的类里, 还有一种避免递归循环的方法: 不往 `dict` 里写键, 而是用一次调用把属性赋值改道到更高的父类: `object.setattr(self, attr, value + 10) # 正确: 不会循环 (预览)` 这是对凌驾于所有顶层类之上的隐含 `object` 父类的显式类调用 (按边栏 "The super Alternative", 也有不带 `self` 的 `super().setattr(...)` 等价写法)。事实上, 按后面的注, 在某些类中这种替代方案可能是必需的, 不过在实践里很少见。第三个属性管理方法是 `delattr`, 它接收属性名字符串, 并在所有属性删除时被调用 (即 `del object.attr`)。和 `setattr` 一样, 它必须通过在类内使用 `dict` 执行属性删除、或把删除改道给父类, 来避免递归循环。注——属性离群者 (Attribute outliers): 前面章节提到过, 用 `slots` 和 `properties` 这类高级类工具编写的属性, 并不物理存储在实例的 `dict` 命名空间字典里——而 `slots` 甚至可能让 `dict` 完全不存在。如前所述, 在使用这些工具的类中, 列出和获取属性可能需要 `dir` 和 `getattr`; 但赋值同样受到影响: 为了支持这类 "虚拟" 属性, `setattr` 可能需要使用这里展示的 `object.setattr` 方案, 而不是 `self.dict` 索引。你将在后续章节学到更多关于这些属性工具的内容。

代码分析

```
>>> class Accesscontrol:
...     def __setattr__(self, attr, value):
...         if attr == 'age':
...             self.__dict__[attr] = value + 10 # self.name...
...         else:
...             raise AttributeError(attr + ' not allowed')
>>> X = Accesscontrol()
>>> X.age = 40 # X.__setattr__('age', 40)
>>> X.age # __dict__
50
```

·解释器如何处理：X.age = 40 编译为 STOREATTR 字节码；虚拟机调用 Accesscontrol.setattr(X,'age', 40)。方法内 self.dict[attr] = value + 10：self.dict 是一个读取 (LOADATTR, 能正常找到)，随后是对字典键的赋值 (STORESUBSCR) ——这既不是 self.xxx = v 也不是 setattr(self, ...)，因此不会再次进入 setattr。于是 X.dict['age'] 被写为 50。稍后 X.age 的读取经 LOADATTR 在 dict 中找到 50，完全不碰 getattr。

·对比死循环：若写成 self.age = value + 10，则在 setattr 内部再次触发 STOREATTR → setattr →无限嵌套，直到 RecursionError。setattr(self,'age', ...) 是同一个陷阱的另一种写法——setattr(obj, n, v) 内部就是 type(obj).setattr(obj, n, v)。

·第三种出路：object.setattr(self, attr, value + 10) 是直接调用父类实现，绕开当前类的 setattr 一层——像爬梯子从"自定义层"跨到"基类层"，既不循环又保留真实赋值语义；对 slots/property 属性（不在 dict 里）这甚至是必需手段。

深度理解

- 核心概念：setattr 拦截每一个属性赋值（含 init 里的 self.x = ...! ），是"写通道"的总开关；delattr 对应删除。用它们做校验/转换时，必须用"绕行方案"落地真实写入。
- 底层实现：STOREATTR → tpsetattr → 若类定义了 setattr 则调用之；内部真实写入的标准路径是 object.setattr（C 层 PyObjectGenericSetAttr，处理 dict、slots、描述符）。三条路：dict[k]（仅适用于普通属性）、object.setattr（通用）、super().setattr（同上，隐式）。
- 为什么这样设计：把"赋值"变成可编程事件，才有了数据校验（非负、类型检查）、日志审计、版本化快照、不可变对象模拟等能力；递归风险是"钩子全面覆盖"的必然代价，官方文档也专门为此提醒。
- 实际问题：ORM 中把 obj.field = v 标记为 dirty 以便延迟写库；配置类拒绝未知键；权限系统拦截敏感属性写入。删除钩子用于联动清理缓存。
- 初学者误区：①在 setattr 里随手写 self.x = v 引发 RecursionError——这是 Python 面试经典送命题；②以为"只有显式调用 setattr 才拦截"——所有赋值都自动拦截；③init 里的赋值也逃不过，所以必须在构造路径上就绕行；④不知道 setattr() 函数与点号赋值殊途同归，同样会进钩子。

30.8.3 其他属性管理工具 (Other Attribute-Management Tools)

英文原文

The three attribute-access overloading methods we've met so far allow you to control or specialize access to attributes in your objects. They tend to play highly specialized roles, some of which we'll explore later in this book. For another example of `getattr` at work, see Chapter 28's `person-composite.py` (Example 28-11). And for future reference, keep in mind that there are other ways to manage attribute access in Python:- The `getattribute` method intercepts all attribute fetches, not just those that are undefined; like `setattr`, it must avoid loops for other attribute fetches in the class, usually with object rerouting.- The `property` built-in function allows us to associate methods with fetch and set operations on a specific class attribute; it cannot catch accesses generically but can define what some do.- Descriptors provide a protocol for associating get and set methods of a class with accesses to a specific instance or class attribute; they are as focused as `property` and, in fact, are used to implement it.- Slots attributes are declared in classes but create implicit storage in each instance; if present, generic tools may need to list, fetch, and assign with schemes described in the preceding note. Because these are all advanced tools that are not of interest to every Python programmer, we'll defer the details of other attribute-management techniques until Chapter 32, and await their f

ocused coverage in Chapter 38.

中文翻译

我们到目前为止认识的三个属性访问重载方法，让你能够控制或专门化对象中属性的访问。它们往往扮演高度专门化的角色，其中一些我们会在本书后面探索。想看 `getattr` 实际应用的另一个例子，请参见第 28 章的 `person-composite.py` (Example 28-11)。另外为了将来参考，请记住 Python 中还有其他管理属性访问的方式：- `getattribute` 方法拦截所有属性获取，而不仅仅是未定义的那些；和 `setattr` 一样，它必须用 `object` 改道来避免类内其他属性获取造成的循环。- `property` 内建函数允许我们把方法关联到某个特定类属性的获取与设置操作上；它无法通用地捕获访问，但可以定义某些访问的行为。- 描述符 (Descriptor) 提供一种协议，把一个类的 `get` 和 `set` 方法关联到对某个特定实例或类属性的访问上；它们和 `property` 一样聚焦，事实上 `property` 就是用描述符实现的。- `Slots` 属性在类中声明，但会在每个实例中创建隐式存储；如果存在，通用工具可能需要用前面注中描述的方案来列出、获取和赋值。由于这些都是进阶工具，并非每个 Python 程序员都感兴趣，我们把其他属性管理技术的细节推迟到第 32 章，并在第 38 章做重点覆盖。

深度理解：getattr vs getattribute 的完整对比 (重点)

·核心区别一句话：`getattr` 只在找不到属性时兜底；`getattribute` 无条件拦截每一次属性读取 (`dict`、`class`、甚至 `self.getattribute` 本身都被它先看到)。

·底层实现：LOADATTR 的 C 路径中，只要类定义了 `getattribute`（`tpgetattro` 槽位指向它），它就代替默认查找流程成为唯一入口，内部必须自己完成"查找+返回"或"调用 `object.getattribute` 走默认流程"。而 `getattr` 只是在默认流程失败后由 C 层补调（`PyObject_GenericGetAttr` 尾部逻辑）。

·为什么这样设计：`getattribute` 是"重写规则"，适合拦截一切（如代理转发、审计）；`getattr` 是"缺口补丁"，只干预不存在的名字，日常 99% 的场景用它就够、且没有性能损耗与递归风险。

·实用性：自定义 `getattribute` 时，任何内部属性读取（哪怕 `self.x`）都会再次触发它——几乎必须用 `object.getattribute(self, name)` 绕行；而 `getattr` 无此负担，这也是它更常用的原因。

·初学者误区：①以为两者等价可互换——`getattribute` 会拦截已存在属性，`getattr` 不会；②在 `getattribute` 里写 `return self.name` 造成无限递归；③以为 `property` 是"另一个魔法函数"——它就是描述符的语法糖（`property(fget, fset, ...)` 内部构造一个描述符对象）。

·选择指南：普通拦截 → `getattr`；一切读取都要过手 → `getattribute`（要会绕行）；单个属性定制 → `property`；一族属性复用逻辑 → 描述符类。

30.8.4 模拟实例属性私有性：第一部分（Emulating Privacy for Instance Attributes: Part 1）

英文原文

As another use case for attribute tools, the code in Example 30-10—file `private0.py`—generalizes the previous example, to allow each subclass to have its own list of private names that cannot be assigned to its instances (and raises a built-in exception with `raise`, which you'll have to take on faith until Part VII). Example 30-10. `private0.py` This is a first-cut solution for an implementation of attribute privacy in Python—disallowing changes to attribute names outside a class. Although Python doesn't support private declarations per se, techniques like this can emulate much of their purpose. This particular attempt, though, is a partial—and even clumsy—solution. To make it more effective, we must augment it to allow classes to set their private attributes more naturally, without having to go through `dict` each time, as the constructor must do here to avoid triggering `setattr` and an exception. A better and more complete approach might require a wrapper ("proxy") class to check for private attribute accesses made outside the class only and a `getattr` to validate attribute fetches too. We'll postpone a more complete solution to attribute privacy until Chapter 39, where we'll use class decorators to intercept and validate attributes more generally. Even though privacy can be emulated this way, though, it almost never is in practice. Python programmers are able to write large O

OP frameworks and applications without private declarations—an interesting finding about access controls in general that is beyond the scope of our purposes here. Still, catching attribute references and assignments is generally a useful technique; it supports delegation, a design technique that allows controller objects to wrap up embedded objects, add new behaviors, and route other operations back to the wrapped objects. Because they involve design topics, we'll revisit delegation and wrapper classes in the next chapter. Here, it's time to move ahead in the operator overloading domain.

中文翻译

作为属性工具的另一个用例，Example 30-10 的代码——文件 `private0.py`——对前面的例子做了泛化：让每个子类都有自己的私有名字列表，这些名字不能被赋给它的实例（并且用 `raise` 抛出内建异常——这一点在读到第七部分之前你先接受下来即可）。Example 30-10. `private0.py` 这是在 Python 中实现属性私有性（attribute privacy）——禁止在类外修改属性名——的第一个粗略方案。虽然 Python 本身不支持私有声明（private declaration），但这样的技巧可以模拟其大部分目的。不过，这个特定尝试是一个不完整——甚至笨拙——的方案。要让它更有效，我们必须增强它，让类能够更自然地设置自己的私有属性，而不必每次都绕过 `dict`——就像这里的构造器为了避免触发 `setattr` 和异常而必须做的那样。更好更完整的方案可能需要一个包装（wrapper/"代理"）类，只检查类外发起的私有属性

访问，并用一个 `getattr` 来校验属性获取。我们将把属性私有性更完整的方案推迟到第 39 章，那里将用类装饰器（`class decorator`）更通用地拦截和校验属性。不过，即使私有性可以这样模拟，实践中也几乎从来没人这么做。Python 程序员无需私有声明也能写出大型 OOP 框架和应用——这是关于访问控制（`access control`）本身的一个有趣发现，超出了我们这里的目的范围。不过，捕获属性引用和赋值总体上仍然是有用的技术；它支撑着委托（`delegation`）——一种允许控制器对象包装内嵌对象、添加新行为、并把其他操作路由回被包装对象的设计技巧。由于涉及设计主题，我们将在下一章重新探讨委托与包装类。这里，是时候在运算符重载领域继续前进了。

代码分析（Example 30-10. `private0.py`）

```
class Privacy:
    def __setattr__(self, attr, value):          # self.attr = value...
        if attr in self.privates:              #
            raise NameError(f'{attr!r} for {self}') #
        else:
            self.__dict__[attr] = value        #

class Test1(Privacy):
    privates = ['age']                          #

class Test2(Privacy):
    privates = ['name', 'pay']
    def __init__(self):
        self.__dict__['name'] = 'Pat'          # 39

if __name__ == '__main__':
    x = Test1()
    x.name = 'Sue'                             # 'name' Test1
    print(x.name)
```

```
# x.age = 40          # 'age'
y = Test2()
y.age = 30           #
print(y.age)
# y.name = 'Bob'      # 'name'
```

·解释器如何处理：x.name = 'Sue' → STOREATTR → Privacy.setattr(x, 'name', 'Sue')。方法内先读 self.privates（继承自 Test1 的类属性 ['age']）判断 'name' 是否私有——不是，于是 self.dict['name'] = 'Sue' 直接写实例字典。而 y.name = 'Bob' 时 attr='name' 命中 Test2.privates，立即 raise NameError。

·本方案的缺陷：Test2.init 要给私有属性 name 设初值时，只能绕道 self.dict['name'] = 'Pat'——因为 self.name = 'Pat' 也会被 setattr 拦截而报错。这就是“不完整”的根源：私有校验不区分调用方在类内还是类外。

·设计思想：Python 的哲学是“成年人的自律”——用约定（单下划线 x）而非强制来实现“私有”。Lutz 的观察很深刻：能模拟私有，但实践里几乎没人做；访问控制的价值更多在“防止误用”而非“防黑客”。

深度理解

·核心概念：私有性 = “在 setattr 里查黑名单”。这是把“赋值事件”变成“校验关卡”的典型示范，也是第 39 章类装饰器方案的雏形。

·底层实现：校验发生在 STOREATTR 的 Python 层；绕行靠 dict 键赋值（STORESUBSCR，不经 setattr）。删除私有属性同理需要 delattr + dict。

·为什么这样设计（以及为什么不常做）：Python 默认"一致接口 + 约定俗成"。强制私有会破坏动态特性（猴子补丁、框架自省），所以连语言本身都不提供 `private` 关键字——这是与 Java/C++ 的哲学分水岭。

·实际问题：与其说"私有"，不如说"白名单校验"更常用——如配置系统只允许已注册的键；`getattr+setattr` 组合还能实现完整"代理"（下一章专讲）。

·初学者误区：①以为 Python 有真正的私有——`x` 只是约定（"别碰我"），`x`（双下划线）是名字改写（`name mangling`）而非访问控制；②在 `__init__` 里直接 `self.privateattr = ...` 触发 `setattr` 报错却不明白原因；③把 `NameError` 当私有语义的一部分——它只是这个例子选的异常，`AttributeError` 更贴切。

30.9 字符串表示：repr 和 str (String Representation)

英文原文

Our next methods deal with display formats—a topic we've already explored in prior chapters but will summarize and formalize here. To serve as a guinea pig, the following codes the `__init__` constructor and the `__add__` overload method, both of which we've already seen (`+` is an in-place operation here, just to show that it can be; this may be better coded as a named method per Chapter 27, or the in-place `iadd` covered ahead). As

we've learned, the default display of instance objects for a class like this is neither generally useful nor aesthetically pretty: >>> class adder: ... def init(self, value=0): ... self.data = value # Initialize data ... def add(self, other): ... self.data += other # Add other in place >>> x = adder() # Default displays: >>> print(x) # str or else repr <main.adder object at 0x106110c80> >>> x # repr <main.adder object at 0x106110c80> But coding or inheriting string representation methods allows us to customize the display—as in the following, which defines a repr method in a subclass that returns a string representation for its instances: >>> class addrepr(adder): ... def repr(self): # Inherit init, add ... return f'addrepr({self.data})' # Add string representation >>> x = addrepr(2) # Convert to as-code string >>> x + 1 >>> x # Runs repr addrepr(3) >>> print(x) # Runs repr addrepr(3) >>> str(x), repr(x) # Runs repr for both ('addrepr(3)', 'addrepr(3)') If defined, repr (or its close relative, str) is called automatically when class instances are printed or converted to strings. These methods allow you to define a better display format for your objects than the default instance display. Here, repr uses basic string formatting to convert the managed self.data object to a more human-friendly string for display.

中文翻译

我们接下来的方法处理显示格式——这个话题我们在前面章节已经探索过，但这里将做总结和形式化。为了当“实验对象”，下面代码编写了 `init` 构造器和 `add` 重载方法，两者我们都见过了（这里的 `+` 是就地操作，只是为了展示它可以这样；按第 27 章的说法，这可能更适合写成命名方法，或使用后面要讲的就地 `iadd`）。正如我们学过的，像这样的类的实例对象，默认显示既谈不上有用，也谈不上好看：

```
>>> class adder: ... def init(self, value=0): ... self.data
= value # 初始化数据... def add(self, other): ... self.data
+= other # 就地加 other >>> x = adder() # 默认显示:
>>> print(x) # str 否则 repr <main.adder object at 0x
106110c80> >>> x # repr <main.adder object at 0x1
06110c80> 但编写或继承字符串表示方法就能定制显示——
如下面的子类定义了一个 repr 方法，为它的实例返回字符串表示:
>>> class addrepr(adder): ... def repr(self):
# 继承 init、add ... return f'addrepr({self.data})' # 添加
字符串表示>>> x = addrepr(2) # 转换为“可复制回代码”
的字符串>>> x + 1 >>> x # 运行 repr addrepr(3) >>>
print(x) # 运行 repr addrepr(3) >>> str(x), repr(x) # 两
者都运行 repr ('addrepr(3)','addrepr(3)') 如果定义了 re
pr（或它的近亲 str），当类实例被打印或转换为字符串时
，它会被自动调用。这些方法让你能为对象定义比默认实例
显示更好的显示格式。这里，repr 用基本的字符串格式化
，把所管理的 self.data 对象转成更友好的字符串用于显示
```

。

代码分析

```
>>> class addrepr(add):
...     def __repr__(self):          # __init__ add__
...         return f'addrepr({self.data})' #
>>> x = addrepr(2)
>>> x + 1
>>> x                                # __repr__
addrepr(3)
>>> print(x)                         # __repr__
addrepr(3)
```

·解释器如何处理：print(x) 在 C 层调用 PyObjectStr(x)：先尝试 str——addrepr 没有定义它，于是回退到 repr，得到'addrepr(3)'再输出。REPL 回显 x 走 PyObjectRrepr(x)，直接用 repr。两者路径不同、殊途同归——这正是"repr 是无处不在的兜底"的体现。

·设计思想：repr 返回 addrepr(3) 这种"长得像代码"的字符串——理想情况下，把 repr 的输出粘贴回 Python 里能重新创建等价对象 (eval(repr(x)) 精神)。这是调试器的语言：开发者一看便知这个对象是什么构造出来的。

深度理解

·核心概念：显示两兄弟 str (用户友好) 与 repr (开发者友好/可重建)，共用"默认显示 <main.adder object at 0x...>"这个 object 基类的兜底。

·底层实现：print 内部 = PyFileWriteObject → str(obj) → tpstr 槽位：先找 str，没有则退 tprepr (repr)。REPL 与 repr() 则直取 tprepr，从不尝试 str。

·为什么这样设计：同一对象有"两种观众"——终端用户要美 ([Value: 5])，开发者要真 (addboth(5))；两者分

离避免了"显示好看"与"可调试信息"打架。

·实际问题：日志与异常信息用 repr（拿到可重建细节），业务输出用 str；f'{obj!r}' 强制走 repr，f'{obj!s}' 强制走 str，混搭输出更灵活。

·初学者误区：①str/repr 返回非字符串（如数字）——Python 不做类型转换直接 TypeError: str returned non-string；②只写 str 却发现 REPL 回显还是地址——因为回显永远走 repr；③以为容器会调用元素的 str——列表打印元素时用的是 repr。

30.9.1 为什么有两个显示方法？（Why Two Display Methods?）

英文原文

So far, this section has largely been review. But while these methods are generally straightforward to use, their roles and behavior have some subtle implications for both design and coding. In particular, Python provides its two display methods to support alternative displays for different audiences: - str is tried first for the print operation and the str built-in function (the internal equivalent of which print runs). It generally should return a user-friendly display. - repr is used in all other contexts: for interactive echoes, the repr function, and nested appearances, as well as by print and str if no str is present. It should generally return an ASCII string that could be used to re-create the object

t or a detailed display for developers. That is, repr is used everywhere, except by print and str when a str is defined. This means you can code a repr to define a single display format used everywhere and may code a str to either support print and str exclusively or to provide an alternative display for them. General tools may also prefer str to leave other classes the option of adding an alternative repr display for use in other contexts, as long as print and str displays suffice for the tool. Conversely, a general tool that codes a repr still leaves clients the option of adding alternative displays with a str for print and str. In other words, if you code either, the other is available for an additional display. In cases where the choice isn't clear, use str for higher-level displays and repr for lower-level displays and all-inclusive roles. Let's write some code to illustrate these two methods' distinctions in more concrete terms. The prior example in this section showed how repr is used as the fallback option in many contexts. However, while printing falls back on repr if no str is defined, the inverse is not true—other contexts, such as interactive echoes, use repr only and don't try str at all:

```
>>> class addstr(adder): ... def str(self): # str but no repr ... return f'[Value: {self.data}]' # Convert to nice string
>>> x = addstr(3) >>> x + 1 >>> x # Default repr (in object) <main.addstr object at 0x106111b20>
>>> print(x) # Runs str (in addstr) [Value: 4]
```

```
>>> str(x), repr(x) ('[Value: 4]', '<main.addstr object at 0x106111b20>')
```

Because of this, `repr` may be best if you want a single display for all contexts. By defining both methods, though, you can support different displays in different contexts—for example, a class's end-user display with `str`, and a class's developer display with `repr`. In effect, `str` simply overrides `repr` for more user-friendly display contexts:

```
>>> class addboth(adder): ... def str(self): ... return f'[Value: {self.data}]' # User-friendly string ... def repr(self): ... return f'addboth({self.data})' # As-code string
>>> x = addboth(4) >>> x + 1 >>> x # Runs repr addboth(5)
```

```
>>> print(x) # Runs str [Value: 5]
```

```
>>> str(x), repr(x) ('[Value: 5]', 'addboth(5)')
```

Bonus: your classes' `str` and `repr` are also run automatically by all three string-formatting tools of Chapter 7. This may not be a shocker, given these tools are defined to work like `str` and `repr` in this role, but display overloading is not just about REPL echoes and `print`:

```
>>> f'{x!s} {x!r}', '{!s} {!r}'.format(x, x), '%s %r' % (x, x) (
'[Value: 5] addboth(5)', '[Value: 5] addboth(5)', '[Value: 5] addboth(5)')
```

中文翻译

到目前为止，这一节大部分是复习。不过，虽然这些方法用起来一般都直截了当，但它们的角色和行为对设计与编码都有一些微妙的含义。尤其是：Python 提供两个显示方法，是为了给不同的观众提供替代的显示方式：- str 在 print 操作和 str 内建函数（print 内部运行的就是它）中首先被尝试。它一般应当返回用户友好的显示。- repr 用于所有其他上下文：交互式回显、repr 函数、嵌套出现，以及在没有 str 时由 print 和 str 使用。它一般应当返回"可复制回代码"（as-code）的字符串，可以用来重新创建对象，或返回面向开发者的详细显示。也就是说，除了 print 和 str 在定义了 str 的情况下，repr 被用于所有地方。这意味着你可以只写一个 repr 来定义"处处通用"的单一显示格式，也可以再写一个 str，专门支持 print 和 str，或为它们提供替代显示。通用工具也可能更愿意实现 str，把"增加替代 repr 显示用于其他场景"的选项留给其他类——只要 print 和 str 的显示对该工具够用。反过来，实现 repr 的通用工具也仍然把"为 print 和 str 添加 str 替代显示"的选项留给使用方。换句话说，你只要实现其中任何一个，另一个就有机会被补充成一个额外显示。在难以抉择的情况下，用 str 做高层级显示，用 repr 做低层级、全场景的显示。让我们写点代码，更具体地说明这两个方法的区别。本节前面的例子展示了 repr 在许多场景中作为回退选项被使用。然而，虽然打印在未定义 str 时会回退到 repr，反向却不成立——其他场景，比如交互式回显，只用 repr，根本不会尝试 str：

```
>>> class addstr(adder): ... def str(self): # 只有 str 没有 repr ... return f'[Value: {self.data}]' # 转成好看的字符
```

串>>> x = addstr(3) >>> x + 1 >>> x # 默认 repr (在 object 中) <main.addstr object at 0x106111b20> >>> print(x) # 运行 str (在 addstr 中) [Value: 4] >>> str(x), repr(x) ('[Value: 4]', '<main.addstr object at 0x106111b20>') 正因如此, 如果你想要一个全场景统一的显示, 只实现 repr 可能最好。不过, 通过同时定义两个方法, 你可以在不同场景支持不同显示——比如用 str 提供终端用户显示, 用 repr 提供开发者显示。实际上, str 就是在"更用户友好"的显示场景中覆盖了 repr: >>> class addboth(adder): ... def str(self): ... return f'[Value: {self.data}]' # 用户友好字符串... def repr(self): ... return f'addboth({self.data})' # 可复制回代码的字符串>>> x = addboth(4) >>> x + 1 >>> x # 运行 repr addboth(5) >>> print(x) # 运行 str [Value: 5] >>> str(x), repr(x) ('[Value: 5]', 'addboth(5)') 额外福利: 你的类的 str 和 repr 也会被第 7 章的全部三种字符串格式化工具自动运行。考虑到这些工具被定义成在这一角色上效仿 str 和 repr, 这也许不令人惊讶——但显示重载不只是关于 REPL 回显和 print: >>> f'{x!s} {x!r}', '{!s} {!r}'.format(x, x), '%s %r' % (x, x) ('[Value: 5] addboth(5)', '[Value: 5] addboth(5)', '[Value: 5] addboth(5)')

代码分析

```
>>> class addboth(adder):
...     def __str__(self):
...         return f'[Value: {self.data}]' #
...     def __repr__(self):
...         return f'addboth({self.data})' #
>>> x = addboth(4)
>>> x                                     # __repr__
addboth(5)
>>> print(x)                             # __str__
[Value: 5]
>>> str(x), repr(x)
('[Value: 5]', 'addboth(5)')
>>> f'{x!s} {x!r}', '{!s} {!r}'.format(x, x), '%s %r' % (x, x)
('[Value: 5] addboth(5)', '[Value: 5] addboth(5)', '[Value: 5] add...
```

· 解释器如何处理（完整的分派矩阵）：

· REPL 回显 / repr(x) → tprepr → repr → addboth(5)
（永不尝试 str）。

· print(x) / str(x) → tpstr → 找到 str → [Value: 5]。

· 只有 str（无 repr）时：print 用 str，但回显/repr() 落到 object 基类的默认实现 → 内存地址。

· 只有 repr（无 str）时：所有场景都走 repr（tpstr 的 C 实现就是“先 str 后 repr”）。

· 格式化工具：f'{x!s}'/'{}'.format/%s 等价 str(x)；f'{x!r}'/%r 等价 repr(x)——三种格式化语法共享同一对槽位。

· 决策树记忆法：先画 str 有吗？→ 有：print/str 用它，其余全用 repr（无论自定义与否）；→ 没有：print/str 也回退到 repr。repr 永远不会输。

深度理解

·核心概念：两个方法的取舍本质是观众分众：str 给终端用户（好看），repr 给开发者（可重建、信息全）。"只实现一个"的推荐是 repr——它覆盖全场景，str 自动获得默认。

·底层实现：tpstr 的 C 实现逻辑为"类有 str 用之，否则用 repr 的结果"；tprepr 无此回退。这是不对称的根源，务必记死。

·为什么这样设计：历史继承——repr() 从一开始就是"调试用"，str() 是"人用"；保留两个槽位让库作者可以同时提供两种视图，比如 datetime 的 repr 带时区细节、str 则简明。

·实际问题：异常类通常只写 str (str(exc) 显示给用户)；数据类/值对象最好写 repr (日志、测试断言肉眼可读)；容器嵌套时元素全走 repr——所以想让列表里"好看"，就得写 repr。

·初学者误区：①以为"定义了 str，回显也会变"——回显只认 repr；②%s 与 %r 混用不知所以——%s=str、%r=repr 的规则同样适用；③返回 str(self.data) 之外的"自造字符串"却忘记把值转换进去——格式化空对象导致输出'None' 一类怪象。

30.9.2 显示方法使用注意 (Display Usage Notes)

英文原文

Though generally simple to use, three points regarding these display methods are worth calling out here. First, keep in mind that `str` and `repr` must both return strings; other result types are not converted and raise errors, so be sure to run them through a to-string converter (e.g., `str` or `f'...'`) if needed.

Second, depending on a container's string-conversion logic, the user-friendly display of `str` might only apply when objects appear at the top level of a print operation; objects nested in larger objects might still print with their `repr` or its default. The following illustrates both of these points:

```
>> class Printer:
... def init(self, val): ... self.val = val ... def str(self): ... re
turn str(self.val) # Used for instance itself >> objs = [
Printer(2), Printer(3)] >> for x in objs: print(x) # str ru
n when instance printed 2 # But not when instance is
in a list! 3 >> print(objs) [<main.Printer object at 0x1
06110c80>, <main.Printer object at 0x1060d2570>]
>> objs [<main.Printer object at 0x106110c80>, <ma
in.Printer object at 0x1060d2570>]
```

To ensure that a custom display is run in all contexts regardless of the container, code `repr`, not `str`; the former is run in all cases if the latter doesn't apply, including nested appearances:

```
>> class Printer:
```

```
... def init(self, val): ... self.val = val ... def repr(self): ...  
return str(self.val) # repr used by print if no str >> ob  
js = [Printer(2), Printer(3)] >> for x in objs: print(x) #  
No str: runs repr 2 3 >> print(objs) # Runs repr, not s  
tr [2, 3] >> objs [2, 3]
```

Third, and perhaps most subtle, the display methods also have the potential to trigger infinite recursion loops in rare contexts—because some objects' displays include displays of other objects, it's not impossible that a display may trigger a display of an object being displayed, and thus loop.

This is rare and obscure enough to skip here but watch for an example of this looping potential to appear for these methods in a note near the end of the next chapter about its listinherited.py class of Example 31-12, where repr can loop.

In practice, str and its more inclusive relative, repr, seem to be the second most commonly used operator-overloading methods in Python scripts, behind init. Anytime you can print an object and see a custom display, one of these two tools is probably in use.

For additional examples of these tools at work and the design trade-offs they imply, see Chapter 28's case study and Chapter 31's class-lister mix-ins, as well as their role in Chapter 35's exception classes, where str is required over repr.

中文翻译

虽然这些显示方法一般使用简单，但有三个要点值得在这里指出。第一，请记住 `str` 和 `repr` 都必须返回字符串；其他结果类型不会被转换，而是直接报错，所以需要时务必让它们经过字符串转换器（例如 `str` 或 `f'...'`）。第二，取决于容器的字符串转换逻辑，`str` 的用户友好显示可能只在对象位于 `print` 操作的顶层时生效；嵌套在更大对象中的对象，可能仍然按 `repr` 或其默认显示打印。下面两点都通过这个例子说明：

```
>>> class Printer: ... def init(self, val): ... self.val = val ... def str(self): ... return str(self.val) # 实例本身被打印时使用>>> objs = [Printer(2), Printer(3)]>>> for x in objs: print(x) # 实例被打印时运行 str 2 # 但实例在列表里时不会! 3>>> print(objs) [<main.Printer object at 0x106110c80>, <main.Printer object at 0x1060d2570>]>>> objs [<main.Printer object at 0x106110c80>, <main.Printer object at 0x1060d2570>] 要确保自定义显示在任何容器、任何场景下都被运行，就写 repr 而不是 str——在 str 不适用时，前者在所有场景（包括嵌套出现）都会被运行：
```

```
>>> class Printer: ... def init(self, val): ... self.val = val ... def repr(self): ... return str(self.val) # 没有 str 时 print 用 repr>>> objs = [Printer(2), Printer(3)]>>> for x in objs: print(x) # 没有 str: 运行 repr 2 3>>> print(objs) # 运行 repr, 而不是 str [2, 3]>>> objs [2, 3]
```

第三，也许最微妙的一点：显示方法在罕见的场景下还有触发无限递归循环的可能——因为某些对象的显示包含对其他对象的显示，所以并非不可能出现“显示触发了正在被显示的对象”从而循环。这种情况罕见

且晦涩，这里略过；但请留意下一章接近末尾处关于其 listinherited.py 类 (Example 31-12) 的一条注，那里有个 repr 会循环的例子。在实践中，str 与它更全能的亲戚 repr，似乎是 Python 脚本中第二常用的运算符重载方法——仅次于 init。任何时候你能打印一个对象并看到自定义显示，就说明这两个工具之一在起作用。更多关于这些工具的实际应用与其设计取舍的例子，请参见第 28 章的案例研究、第 31 章的类列出器混入 (class-lister mix-ins)，以及第 35 章异常类中的角色——那里 str 被要求优先于 repr。

代码分析

```
>>> class Printer:
...     def __init__(self, val):
...         self.val = val
...     def __str__(self):
...         return str(self.val) #
>>> objs = [Printer(2), Printer(3)]
>>> for x in objs: print(x)      # __str__
2
3
>>> print(objs)                  # repr
[<__main__.Printer object at 0x106110c80>, <__main__.Printer objec...
```

解释器如何处理：for x in objs: print(x) 中每个元素是“顶层对象”，print 对它走 tpstr → str → '2'。而 print(objs) 时，C 层 list 的 tprepr 遍历元素并对每个元素调用 PyObjectRepr (不是 PyObjectStr!) ——于是 Printer 没有 repr，落到 object 的默认实现，输出内存地址。这就是“列表里显示失效”的机制真相：容器格式化元素时一律用 repr。

- 对策：把自定义显示放进 repr，则打印、回显、嵌套三种场景全覆盖；print(x) 没有 str 也会自动回退到 repr。
- 隐患提醒：若 repr 里又去格式化包含"自身"的容器（如 self.items 里恰好有 self 或相互引用），list.repr → 元素 repr → ...形成无限递归，最终 RecursionError——下一章 listinherited.py 会演示真实案例。

深度理解

- 核心概念：显示方法的三大纪律——①必须返回 str；②容器内嵌元素一律走 repr，想全场景生效就写 repr；③显示逻辑可能递归（环形引用时要防御）。
- 底层实现：list.repr 的 C 实现 (listrepr) 对每个元素调用 PyObjectRepr；dict.repr 对键值都调 PyObjectRepr。所以 str([obj]) 与 str(obj) 的结果可以截然不同——它们走的是不同槽位。
- 为什么这样设计：容器必须"可重建地显示"（[obj1, obj2] 是 repr(obj1), repr(obj2) 的产物），若用 str 则输出可能含糊（如 [2, 3] 里的元素到底是谁？）；repr 优先保证信息无损。
- 实际问题：日志记录对象列表（要详细信息）、异常链显示、pprint/json.dumps 自定义序列化——都依赖"repr 优先"的约定；写库前先想清楚该给观众哪个视图。
- 初学者误区：①写完 str 后 print(listofobjs) 仍然是一堆地址——不是 bug，是规范；②repr 返回非字符串时报 TypeError；③str 与 repr 互相调用（str 里 return self.repr() 且 repr 里又 str(self)）造成循环。

30.10 右端与就地运算: radd 和 iadd (Right-Side and In-Place Ops)

英文原文

Our next group of overloading methods extends the functionality of binary operator methods such as `add` and `sub` (called for `+` and `-`), which we've already seen. As mentioned earlier, part of the reason there are so many operator-overloading methods is that they come in multiple flavors—for every binary expression, we can implement a left, right, and in-place variant. Though defaults are also applied if you don't code all three, your objects' roles dictate how many variants you'll need to code.

中文翻译

我们接下来的一组重载方法扩展了 `add`、`sub` (为 `+` 和 `-` 调用) 这类二元运算符方法的功能, 这些我们已经见过了。如前所述, 运算符重载方法之所以这么多, 部分原因就是它们有多种变体——对每个二元表达式, 我们都可以实现左端 (left)、右端 (right) 和就地 (in-place) 三个变体。即使你不把三个都写齐, 默认规则也会生效; 你的对象扮演什么角色, 决定了你需要写多少个变体。

深度理解

·核心概念：一个二元表达式有三种"站位"： $X + Y$ 中 X 在左、 Y 在右 (`add/radd`)， $X += Y$ 是就地 (`iadd`，缺省回退 `add`)。这是"同一语义、三种落点"的设计。

·底层实现：BINARYOP 的 C 分派：先看左操作数类型的方法 (`nbadd`)，失败或返回 `NotImplemented` 再看右操作数的反射方法 (`nbadd` 的反射，即 `radd`)；INPLACEADD (3.12 后并入 BINARYOP 的标志位) 优先 `nbplaceadd`。

·为什么这样设计： $1 + x$ 中 1 是内建 `int`，不认识你的类——必须给右操作数一个机会自己处理；"位置感知"让运算符天然支持交换律场景。

·实际问题：向量、分数、矩阵等数学类型需要 $2v$ 与 v^2 都能工作；ORM 表达式、pandas 的 $1 / df$ 都靠 `rdiv` 家族。

·初学者误区：写完 `add` 后 $2 + x$ 报 `TypeError` 就放弃——那是忘了写 `radd`；或以为 `radd` 的参数顺序与 `add` 相同——不，`self` 在右、`other` 在左，顺序是反的。

30.10.1 右端加法 (Right-Side Addition)

英文原文

For instance, the `add` methods coded so far technically do not support the use of instance objects on the right side of the `+` operator:


```
>> class Adder:
```

```
... def init(self, value=0): ... self.data = value ... def ad  
d(self, other): ... return self.data + other >> x = Adde  
r(5) >> x + 2 7 >> 2 + x TypeError: unsupported ope  
rand type(s) for +:'int' and'Adder'
```

To implement more general expressions and hence support commutative-style operators, code the `radd` method as well. Python calls `radd` only when the object on the right side of the `+` is your class instance, but the object on the left is not an instance of your class. The `add` method for the object on the left is called instead in all other cases. As a demo, consider the class in Example 30-11 (which we'll be adding to in a moment).

Example 30-11. `commuter.py` (start) Because this defines both left- and right-side overloads for `+`, instances can appear on either side, or both:

```
>> from commuter import Commuter1 >> x = Commuter1(88) >> y = Commuter1(99) >> x + 1 # add: i  
nstance + noninstance add 88 1 89 >> 1 + y # radd:  
noninstance + instance radd 99 1 100 >> x + y # add  
: instance + instance => triggers radd add 88 <com  
muter.Commuter1 object at 0x1011b63f0> radd 99 8  
8 187
```

Notice how the order is reversed in `radd`: `self` is really on the right of the `+`, and `other` is on the left. Also note that `x` and `y` are instances of the same class here

; when instances of different classes appear mixed in an expression, Python prefers the class of the one on the left. When we add the two instances of this class together, Python runs `add`, which in turn triggers `radd` by simplifying the left operand and re-adding.

中文翻译

例如，到目前为止编写的 `add` 方法在技术上都不支持实例对象出现在 `+` 运算符的右侧：

```
>>> class Adder: ... def init(self, value=0): ... self.data = value ... def add(self, other): ... return self.data + other >>> x = Adder(5) >>> x + 2 7 >>> 2 + x TypeError: unsupported operand type(s) for +: 'int' and 'Adder'
```

要实现更通用的表达式、从而支持交换律风格的运算符，就要同时编写 `radd` 方法。只有当 `+` 右侧的对象是你的类实例、而左侧对象不是你的类实例时，Python 才会调用 `radd`。其他所有情况下，都改调左侧对象的 `add` 方法。作为演示，请看 Example 30-11 中的类（我们马上会继续给它加东西）。Example 30-11. `commuter.py`（开始部分）因为这里为 `+` 同时定义了左端和右端重载，实例可以出现在任意一侧，或两侧同时出现：

```
>>> from commuter import Commuter1 >>> x = Commuter1(88) >>> y = Commuter1(99) >>> x + 1 # add: 实例 + 非实例 add 88 1 89 >>> 1 + y # radd: 非实例 + 实例 radd 99 1 100 >>> x + y # add: 实例 + 实例 => 触发 radd add 88 <commuter.Commuter1 object at 0x1011b63f0> radd 99 88 187
```

注意 `radd` 中顺序是反过来的：`self` 其实在 `+` 的右边，`other` 在左边。还要注意这里的 `x` 和 `y` 是同一个类的实例；当不同类的实例混

在一个表达式里时，Python 偏好左侧对象的类。当我们把本类的两个实例相加时，Python 运行 `add`，它通过简化左操作数并重新相加，进而触发 `radd`。

代码分析 (Example 30-11. `commuter.py`)

```
class Commuter1:
    def __init__(self, val):
        self.val = val
    def __add__(self, other):
        print('add', self.val, other)      # self other
        return self.val + other
    def __radd__(self, other):
        print('radd', self.val, other)     # self other
        return other + self.val
```

· 解释器如何处理（完整分派流程）：

· `x + 1`：左操作数 `x` 是 `Commuter1` → 调用 `x.add(1)` → `88 + 1`。

· `1 + y`：左操作数 `1` 是 `int` → `int` 的 `add` 遇到不认识的类型，返回 `NotImplemented`（这是“C 层拒绝合作”的信号）→ 解释器反射到右操作数 `y.radd(1)` → `1 + 99`。这正是 `radd` 被调用的唯一时机：左边不认识你、右边认识你。

· `x + y`：左操作数 `x` 被选中 → `x.add(y)`，打印 `add 88 < Commuter1 ...>`，方法体内 `self.val + other = 88 + Commuter1` 实例 → 这又是一个二元加法！左操作数 `int` 返回 `NotImplemented` → 再次反射到 `other`（即 `y`）的 `radd(88)` → `88 + 99`。这就是输出里“add 后跟着 radd”的来历——一次表达式嵌套了两层加法分派。

·关键细节: `radd` 里 `other + self.val` 中 `other` 是 `int` (左侧的 88), `self.val` 也是 `int`——此时双方都是 `int`, 直接走 C 层加法, 不会再递归。

·设计思想: 把 `x + y` (同类相加) 变成"左操作数的 `add` 把右操作数压平再重加"——自动完成"解包+重算", 使同类加法能复用非同类加法逻辑。

深度理解

·核心概念: `radd` = "右侧反射方法", 只在左操作数拒绝处理、右操作数是你的实例时被调用; 参数顺序反转 (`self` 在右)。

·底层实现: `BINARYOP` 的分派算法: `nbadd(left, right)` 若返回 `NotImplemented`, 则调 `nbadd(right, left)` (Python 层即 `radd`); 两者都拒绝才抛 `TypeError: unsupported operand type(s)`。

·为什么这样设计: 任何内建类型 (`int/str/list`) 都不知道自定义类的存在; 反射机制让"我在右、它在左"时仍有解——这是运算符语义完整性的兜底设计。

·实际问题: 实现 `datetime + timedelta` 两侧对称、2 Vector 标量乘法、分数 `1 + Fraction(1,2)`——凡"内建 + 自定义"混算且期望交换律, 都要 `radd` 家族。

·初学者误区: ①把 `radd` 的参数顺序写反 (`return self.val + other` 虽结果碰巧对, 但语义错位、`print` 顺序迷惑); ②以为 `radd` 会在"左边也是自己类"时被调用——不会, 左类优先; ③`add` 遇到不认识的操作数不返回 `NotImplemented` 而直接做算术, 会埋下隐蔽错误。

30.10.2 在 radd 中复用 add (Reusing add in radd)

英文原文

For truly commutative operations that do not require special-casing by position, it is also sometimes sufficient to reuse `add` for `radd`, either by calling `add` directly; by swapping order and re-adding to trigger `add` in directly; or by simply assigning `radd` to be an alias for `add` at the top level of the class statement (i.e., in the class's scope). The alternatives in Example 30-12 implement all three of these schemes and return the same results as the original—though the last saves an extra call or dispatch and hence may be quicker (in all, `radd` is run when `self` is on the right side of a `+`). Example 30-12. `commuter.py` (continued)

中文翻译

对于真正交换律的运算——即不需要按位置特殊处理的运算——有时也足以把 `add` 复用来充当 `radd`：可以直接调用 `add`；可以交换顺序重新相加从而间接触发 `add`；也可以干脆在类语句的顶层（即类的命名空间里）把 `radd` 赋值成 `add` 的别名。Example 30-12 中的三个替代方案实现了这三种做法，返回值与原版相同——其中最后一种省掉了一次额外的调用或分派，因此可能更快（总之，当 `self` 在 `+` 的右侧时运行的就是 `radd`）。Example 30-12. `commuter.py`（继续）

代码分析 (Example 30-12. commuter.py)

```
class Commuter2:
    def __init__(self, val):
        self.val = val
    def __add__(self, other):
        print('add', self.val, other)
        return self.val + other
    def __radd__(self, other):
        return self.__add__(other)      # __add__

class Commuter3:
    def __init__(self, val):
        self.val = val
    def __add__(self, other):
        print('add', self.val, other)
        return self.val + other
    def __radd__(self, other):
        return self + other             #

class Commuter4:
    def __init__(self, val):
        self.val = val
    def __add__(self, other):
        print('add', self.val, other)
        return self.val + other
    __radd__ = __add__                  #
```

· 解释器如何处理：

- Commuter2: $1 + y \rightarrow \text{int 拒绝} \rightarrow y.\text{radd}(1) \rightarrow \text{方法体内 } \text{self.add}(1)$ (一次显式方法调用) $\rightarrow 99 + 1$ 。
- Commuter3: $y.\text{radd}(1) \rightarrow \text{self} + \text{other}$ 即 $y + 1 \rightarrow$ 又是一个 BINARYOP! 这次左操作数 y 是 Commuter3 $\rightarrow$ 调 $y.\text{add}(1)$ 。嵌套一次分派，逻辑等价但多一层。

- Commuter4: 类语句执行时, radd 这个名字直接绑定到 add 函数对象。运行时 `y.radd(1)` 与 `y.add(1)` 是同一个函数——无需跳转, 最快。
- 三种方案的取舍: 方案一最明确; 方案二最"原理驱动"但有额外分派开销; 方案三零开销但要求运算真交换律 (add 的 print 里参数顺序仍以 self 在前打印, 输出可对比验证)。
- 设计思想: "复用而非重写"——交换律运算无需两份实现, DRY 原则在运算符领域同样适用。

深度理解

- 核心概念: radd 可以是 add 的"视图": 调用、重加、别名三种复用手法任选; 前提是运算真·交换律 (顺序不影响结果与副作用)。
- 底层实现: 方案三的"别名"发生在类体执行时——`radd = add` 只是普通赋值语句, 把两个属性名指向同一个函数对象; LOADATTR 时两者结果完全相同, 故零额外开销。
- 为什么这样设计: 位置感知 (left/right) 是语言提供的机制, 但业务语义决定是否需要区分; 把"机制丰富"与"编码成本"解耦, 让类作者按需选择复杂度。
- 实际问题: 矩阵乘法 (非交换!) 绝不能这样复用——`A @ B != B @ A`, 必须分别实现 `matmul/rmatmul`; 加法、标量乘法 ($2v$ 与 v^2 同义) 则可放心复用。
- 初学者误区: ①把非交换运算也别名复用, 结果"碰巧对"但 - (减法) 等就露馅; ②以为"别名后 radd 不再存在"

——它存在，只是与 `add` 同一函数；③调试时被 `add 88 1` 这种"逆序打印"误导，以为参数顺序写错了。

30.10.3 传播类类型 (Propagating class type)

英文原文

In more realistic classes where the class type may need to be propagated in results, things can become trickier: type testing may be required to tell whether it's safe to convert and thus avoid nesting. For instance, without the built-in `isinstance` test in Example 30-13, we could wind up with a `Commuter5` whose `val` is another `Commuter5` when two instances are added and `add` triggers `radd`.

Example 30-13. `commuter.py` (continued) When this version is run, `+` results retain the `Commuter5` type for future operations:

```
>> from commuter import Commuter5 >> x = Commuter5(88) >> y = Commuter5(99) >> x + y Commuter5(187) >> x + 1 # Result is another Commuter instance Commuter5(89) >> 1 + y Commuter5(100) >> z = x + y # Not nested: doesn't recur to radd Commuter5(187) >> z + 10 Commuter5(197) >> z + z Commuter5(374) >> z + z + 1 Commuter5(375)
```

The need for the `isinstance` type test here is very subtle—uncomment, run, and trace to see why it's required. If you do, you'll see that the last part of the prece

ding test winds up differing and nesting objects—which still do the math correctly but kick off pointless recursive calls to simplify their values and extra constructor calls to build results:

```
>> z = x + y # With isinstance test+action commented-out
>> z
Commuter5(Commuter5(187))
>> z + 10
Commuter5(Commuter5(197))
>> z + z
Commuter5(Commuter5(Commuter5(374)))
>> z + z + 1
Commuter5(Commuter5(Commuter5(Commuter5(375))))
```

Another way out of this dilemma is to test and simplify in the constructor instead, per Example 30-14.

Example 30-14. `commuter.py` (continued) This last version works the same as the non-nesting `Commuter5`. To test all of this section's classes, the rest of `commuter.py` in Example 30-15 looks and runs like this—like functions, classes can again be used in tuples naturally because they are first-class objects.

Example 30-15. `commuter.py` (conclusion) Experiment with these classes on your own for more insight. Aliasing `radd` to add in `Commuter5` and `Commuter6`, for example, works and saves a line but doesn't prevent object nesting without these classes' `isinstance` tests.

See also Python's manuals for a discussion of other options in this domain; for example, classes may also return the special `NotImplemented` object for unsupported operands to influence method selection (this is

treated as though the method were not defined—and differs from the prior chapter's `NotImplementedError`).

中文翻译

在更现实的类中，如果类类型需要被传播到结果里，事情会变得更棘手：可能需要类型测试来判断转换是否安全，从而避免嵌套。例如，没有 Example 30-13 中内建的 `isinstance` 测试，当两个实例相加、`add` 触发 `radd` 时，我们可能得到一个 `val` 是另一个 `Commuter5` 的 `Commuter5`。Example 30-13. `commuter.py` (继续) 运行这个版本时，+ 的结果保持了 `Commuter5` 类型，可供后续操作继续使用：

```
>>> from commuter import Commuter5 >>> x = Commuter5(88) >>> y = Commuter5(99) >>> x + Commuter5(88) >>> x + 1 # 结果是另一个 Commuter 实例 Commuter5(89) >>> 1 + y Commuter5(100) >>> z = x + y # 不嵌套：不会递归到 radd Commuter5(187) >>> z + 10 Commuter5(197) >>> z + z Commuter5(374) >>> z + z + 1 Commuter5(375) 这里的 isinstance 类型测试为什么必需，非常微妙——取消注释、运行并追踪，你就会明白。你会看到，前面测试的后半部分最终出现差异并嵌套对象——结果算术仍然正确，但引发了无意义的递归调用来简化值，以及额外的构造器调用来构建结果：
```

```
>>> z = x + y # 注释掉 isinstance 测试+处理 >>> z + Commuter5(Commuter5(187)) >>> z + 10 Commuter5(Commuter5(197)) >>> z + z Commuter5(Commuter5(Commuter5(Commuter5(374)))) >>> z + z + 1 Commuter5(Commuter5(Commuter5(Commuter5(375)))) 摆脱
```

这个困境的另一种方法，是改在构造器里做测试和简化，如 Example 30-14。Example 30-14. `commuter.py`（继续）这最后一个版本与不嵌套的 `Commuter5` 行为相同。为了测试本节的所有类，`commuter.py` 剩下的部分（Example 30-15）长这样、跑起来也这样——和函数一样，类因为是一等对象（first-class object），可以自然地放进元组里使用。Example 30-15. `commuter.py`（结尾）建议你亲手实验这些类以获得更多洞察。例如，在 `Commuter5` 和 `Commuter6` 里把 `radd` 别名到 `add` 也能工作并省一行，但没有这些类的 `isinstance` 测试就无法阻止对象嵌套。另外请参见 Python 手册关于该领域其他选项的讨论；例如，类对不支持的操作数还可以返回特殊的 `NotImplemented` 对象来影响方法选择（这会被当作“方法未定义”处理——并且与上一章的 `NotImplementedError` 不同）。

代码分析（Example 30-13、30-14、30-15. `commuter.py`）

```
# Example 30-13: Commuter5 —
class Commuter5:
    def __init__(self, val):
        self.val = val
    def __add__(self, other):
        if isinstance(other, Commuter5):      #
            other = other.val                  #
        return Commuter5(self.val + other)    # + Commut...
    def __radd__(self, other):
        return Commuter5(other + self.val)
    def __repr__(self):
        return f'Commuter5({self.val})'
```

```
# Example 30-14: Commuter6 —
class Commuter6:
    def __init__(self, val):
        if isinstance(val, Commuter6):           #
            self.val = val.val                   #
        else:
            self.val = val
    def __add__(self, other):
        return Commuter6(self.val + other)
    def __radd__(self, other):
        return Commuter6(other + self.val)
    def __repr__(self):
        return f'Commuter6({self.val})'
```

·解释器如何处理（嵌套是怎么发生的）： $x + y$ ($\text{Commuter5} + \text{Commuter5}$) $\rightarrow x.\text{add}(y)$ ；若没有 `isinstance` 分支，`self.val + other` 就是 `88 + Commuter5` 实例 $\rightarrow \text{int}$ 拒绝 $\rightarrow$ 反射到 `other.radd(88)` $\rightarrow \text{Commuter5}(88 + 99) = \text{Commuter5}(187)$ ——注意此时 `Commuter5.add` 返回的 `Commuter5(self.val + other)` 在 `other` 仍是对象时，`88 + obj` 触发 `radd` 后得到的是 `Commuter5` 对象，于是外层再包一层：`Commuter5(Commuter5(187))`。每加一次就多裹一层， $z + z$ 甚至裹四层——算术都对（`radd` 内部最终用底层值算了），但产生“套娃”对象。

· `isinstance` 分支的作用：同类相加时先解包（`other = other.val`），让 `self.val + other` 变成纯 `int` 运算，不再反射、不再嵌套。

· `Commuter6` 的替代思路：把“解包”搬到构造器——`Commuter6(Commuter6(187))` 构造时自动压平为 `Commuter6(187)`。两种策略殊途同归；`Commuter6` 更省（`ad`

d/radd 不用再分支)。

· NotImplemented 与 NotImplementedError 之辨：前者是单例对象，运算符分派视其为“我不处理，请试试对方/抛 TypeError”；后者是异常类，raise NotImplementedError() 用于“抽象方法未实现”。一个是协议语言，一个是错误报告——完全两回事（上一章讲的是后者）。

深度理解

· 核心概念：类型传播 (type propagation) = 运算结果保持自定义类型，使 $z + 10$ 、 $z + z$ 可链式继续；代价是必须管理“操作数是否为同类实例”，否则发生嵌套。

· 底层实现：嵌套的本质是 BINARYOP 反射分派在“int + Commuter5”时把控制权交回 radd，而 radd 又构造出 Commuter5——外层 add 再把该对象当 other 包起来。递归的深度随运算次数指数增长 ($z+z+z$ 裹 8 层)。

· 为什么这样设计：类型传播是“值对象”（如 Decimal、Fraction、Vector）的行业标准—— $1 + \text{Decimal}('2.5')$ 必须返回 Decimal，否则链式计算很快变成 float 精度灾难；传播与防嵌套是一体两面，语言给出机制，防嵌套要靠设计者。

· 实际问题：实现自定义数值类型时，“构造器压平”（Commuter6 风格）是更优雅的默认——所有构造入口统一净化输入，业务方法无需到处 isinstance。

· 初学者误区：①以为“算术对了就万事大吉”——对象嵌套会让 repr 越来越长、== 语义异常、性能雪崩；②用 type(other) is Commuter5 代替 isinstance——对子类会

误判；③混淆 `NotImplemented`（返回它）与 `NotImplementedError`（抛出它）两个名字相似、角色天差地别的对象。

30.10.4 就地加法 (In-Place Addition)

英文原文

To also implement `+=` in-place augmented addition, code either an `iadd` or an `add`. The latter is used if the former is absent. In fact, the prior section's `Commuter` classes already support `+=` for this reason—Python runs `add` and assigns the result manually. The `iadd` method, though, allows for more efficient in-place changes to be coded where applicable; its return value is assigned to the target on the left of the `+=`:

```
>>> class Number: ... def init(self, val): ... self.val = val ... def iadd(self, other): # iadd explicit: x += y ... self.val += other # Usually returns self ... return self # Else None is returned+assigned >>> x = Number(5) >>> x += 1 >>> x += 1 >>> x.val 7
```

For mutable objects, this method can often specialize for quicker in-place changes:

```
>>> y = Number([1]) # In-place change faster than + >>> y += [2] >>> y += [3] >>> y.val [1, 2, 3]
```

The normal `add` method is run as a fallback, but may not be able to optimize in-place cases:

```
>>> class Number: ... def init(self, val): ... self.val = v
al ... def add(self, other): # add fallback: x = (x + y) ...
return Number(self.val + other) # Propagates class ty
pe >>> x = Number(5) >>> x += 1 >>> x += 1 # An
d += does concatenation here >>> x.val 7
```

Though we've focused on + here, keep in mind that every binary operator has similar right-side and in-place overloading methods that work the same (e.g., mul, rmul, and imul). Still, these methods tend to be uncommon in practice; you only code right-side methods for either-side roles and in-place methods for code economy or speed—and only if you need to support such operators at all. For instance, a Vector class may use these tools, but a Button class probably would not. Button presses, though, may run the next section's method.

中文翻译

要实现 += 就地增强加法，可以编写 iadd 或 add。缺了前者就用后者。事实上，上一节的 Commuter 类已经因此支持 +=——Python 运行 add 并手动把结果赋回去。而 iadd 方法在适用时允许编写更高效的就地修改；它的返回值会被赋给 += 左侧的目标：>>> class Number: ... def init(self, val): ... self.val = val ... def iadd(self, other): # 显式 iadd: x += y ... self.val += other # 通常返回 self ... return self # 否则 None 会被返回并赋值>>> x = Number(5) >>> x += 1 >>> x += 1 >>> x.val 7 对可变对象来说，这个方法常常能专门做更快的就地修改：>>> y = N

umber([1]) # 就地修改比 + 更快>>> y += [2] >>> y += [3] >>> y.val [1, 2, 3] 普通的 add 方法作为回退会被运行, 但它可能无法优化就地场景: >>> class Number: ... def init(self, val): ... self.val = val ... def add(self, other): # add 回退: x = (x + y) ... return Number(self.val + other) # 传播类类型>>> x = Number(5) >>> x += 1 >>> x += 1 # 而且这里的 += 是做拼接>>> x.val 7 虽然这里我们聚焦在 + 上, 但请记住: 每一个二元运算符都有类似的右端与就地重载方法, 工作机制相同 (例如 mul、r mul、imul)。不过, 这些方法在实践中并不常用; 你只在"任一侧角色"时才编写右端方法, 只在"代码经济或速度"时才编写就地方法——而且仅当你的类确实需要支持这些运算符。例如, Vector 类可能会用到这些工具, 但 Button 类大概不会。不过, 按下按钮可能会运行下一节的方法。

代码分析

```
>>> class Number:
...     def __init__(self, val):
...         self.val = val
...     def __iadd__(self, other):      # __iadd__ x += y
...         self.val += other          # self
...         return self                # None
>>> x = Number(5)
>>> x += 1
>>> x.val
6
```

·解释器如何处理: `x += 1` 在 Python 3.12 编译为 BINA
RYOP += (带就地标志)。虚拟机先查 `x` 的类是否有 `iadd`——有, 则调用 `x.iadd(1)`, 把返回值赋回 `x`。本例返回 `self` (同一个对象), 所以 `x` 的标识不变——这就是"就地

"的语义：不新建对象，只改状态。

·列表案例的威力：y.val 是 list 时，self.val += other 内部调用的是 list 的 iadd (list.iadd 在 C 层 listextend, 原地扩展、零复制) ——比 self.val = self.val + other (先新建拼接列表再替换) 快得多。这就是 iadd 的典型优化点。

·回退语义：若没有 iadd, $x += 1$ 等价 $x = x + 1$ (运行 add 后赋值) ——Number 这类不可变风格类由此自动获得 += 支持，但每步都新建对象。

·返回值陷阱：iadd 不写 return self 时返回 None, $x += 1$ 会把 None 赋给 x——对象被"吞噬"。官方约定：就地方法应返回 self (C 层 sqinplaceconcat 等也遵守此约定)。

深度理解

·核心概念：+= 的三级分派：iadd (最优先，原地改) → 缺省用 add (新建对象再赋值)。返回值被赋给左目标，这是 += 的隐藏语义。

·底层实现：INPLACE 指令在左操作数类型上查 nb inplaceadd；无则回退普通二元路径；对可变内建类型 (list/dict/set) 就地操作直接 C 层修改并返回原对象，对不可变类型 (int/str/tuple) 就地与普通加法完全等价 (都是新建)。

·为什么这样设计："就地"是给可变类型的性能优化窗口，不是新语义—— $a += b$ 在 Python 里永远可以理解为 $a = a + b$ (行为上 iadd 只是更快地达成等价结果)。

·实际问题：大列表 `l += [x]`（原地 append 级扩展）比 `l = l + [x]`（全量复制）快得多；`str += ch` 在循环里会退化 $O(n^2)$ ——字符串不可变，没有 iadd 优化。

·初学者误区：①忘记 `return self` 导致变量变 `None`；②以为 `x += 1` 对自定义类“自动改原位”——没有 iadd 时它等价于 `x = x + 1`，引用可能被换新对象；③把“就地”理解成“语法层面修改左侧变量”——`a = a + b` 与 `a += b` 的差别只在方法分派，不影响语义框架。

30.11 调用表达式：call (Call Expressions)

英文原文

On to our next overloading method: the `call` method is called when your instance is called. No, this isn't a circular definition—if defined, Python runs a `call` method for function-call expressions applied to your instances, passing along whatever positional or keyword arguments were sent. This allows instances to conform to a function-based API:

```
>> class Callee:
```

```
... def call(self, pargs, kargs): ... print(f'Called: {pargs=} {kargs=}') >> C = Callee() >> C(1, 2, 3) # C is a "callable" object Called: pargs=(1, 2, 3) kargs={} >> C(1, 2, 3, x=4, y=5) Called: pargs=(1, 2, 3) kargs={'x': 4, 'y': 5}
```

More formally, all the argument-passing modes we explored in Chapter 18 are supported by the `call` meth

od—whatever is passed to the instance is passed to this method, along with the usual implied instance argument `self`. For example, the method definitions:

```
class C: def call(self, a, b, c=5, d=6): ... # Normals and defaults
class C: def call(self, pargs, kargs): ... # Collect arbitrary arguments
class C: def call(self, pargs, d=6, kargs): ... # 3.X keyword-only argument
all match all the following instance calls after assigning X to C()
:
```

```
X(1, 2) # Omit defaults
X(1, 2, 3, 4) # Positionals
X(a=1, b=2, d=4) # Keywords
X([1, 2], dict(c=3, d=4)) # Unpack arbitrary arguments
X(1, (2,), c=3, dict(d=4)) # Mixed modes
```

See Chapter 18 for a refresher on function arguments. The net effect is that classes and instances with a `call` support the exact same argument syntax and semantics as normal functions and methods.

Intercepting call expression like this allows class instances to emulate the look and feel of things like functions, but also retain state information for use during calls. The following, for example, defines callable objects with per-call info specified by explicit attribute assignments:

```
>> class Prod:
... def init(self, value): # Accept just one argument ... self.value = value
... def call(self, other): ... return self.v
```

```
value other >> x = Prod(2) # "Remembers" 2 in state
>> x(3) # 3 (passed) 2 (state) 6 >> x(4) 8
```

In this example, the call may seem a bit gratuitous at first glance. A simple method can provide similar utility:

```
>> class Prod:
... def init(self, value): ... self.value = value ... def comp
(self, other): ... return self.value other >> x = Prod(3)
>> x.comp(3) 9
```

However, call can become more beneficial when using APIs (i.e., libraries) that expect functions—it allows us to code objects that conform to an expected function-call interface but also retain state information and other class assets such as inheritance. In fact, it may be the third most commonly used operator-overloading method, behind the init constructor and the str and repr display-format alternatives (qualitatively speaking).

中文翻译

接下来是我们的下一个重载方法：call 方法在你的实例被调用时被调用。不，这不是循环定义——如果定义了它，Python 就会对应用于实例的函数调用表达式运行 call 方法，并把传进来的位置参数和关键字参数原样传递。这让实例能够符合基于函数的 API：

```
>>> class Callee: ... def call(self, pargs, kargs): ... print(f'Called: {pargs=} {kargs=}')
>>> C = Callee() >>> C(1, 2, 3) # C 是一个"可调用"对
```

象 Called: pargs=(1, 2, 3) kargs={} >>> C(1, 2, 3, x=4, y=5) Called: pargs=(1, 2, 3) kargs={'x': 4, 'y': 5} 更正式地说，我们在第 18 章探索过的所有参数传递模式，call 方法都支持——传给实例的一切都会被传给它，外加通常隐含的实例参数 self。例如，以下方法定义：

```
class C:
    def call(self, a, b, c=5, d=6): ... # 普通参数与默认值
class C:
    def call(self, pargs, kargs): ... # 收集任意参数
class C:
    def call(self, pargs, d=6, kargs): ... # 3.X 关键字专用参数
```

在把 X 赋为 C() 之后，它们都能匹配以下所有实例调用：

```
X(1, 2) # 省略默认值
X(1, 2, 3, 4) # 位置参数
X(a=1, b=2, d=4) # 关键字参数
X([1, 2], dict(c=3, d=4)) # 解包任意参数
X(1, (2,), c=3, dict(d=4)) # 混合模式函数参数
```

请回顾第 18 章。净效果是：带 call 的类和实例支持与普通函数、方法完全一致的参数语法和语义。像这样拦截调用表达式，让类实例能够模仿函数的外观和感觉，同时还能在调用期间保留状态信息。例如，下面的代码定义了可调用对象，每次调用的信息由显式的属性赋值来指定：

```
>>> class Prod: ...
def init(self, value): # 只接受一个参数...
    self.value = value ...
def call(self, other): ...
    return self.value * other
>>> x = Prod(2) # "记住"状态里的 2
>>> x(3) # 3 (传入) 2 (状态)
6
>>> x(4) 8
```

在这个例子里，call 乍一看可能有点多余。一个普通方法也能提供类似功能：

```
>>> class Prod: ...
def init(self, value): ...
    self.value = value ...
def comp(self, other): ...
    return self.value * other
>>> x = Prod(3)
>>> x.comp(3) 9
```

不过，当使用期望函数的 API（即库）时，call 的优势就显现出来了——它让我们能编写出符合预期函数调用接口、同时又保留状态信息和继承等其他类资产的对象。事实上，按经验（qualitatively）来说，它可能是

第三常用的运算符重载方法——仅次于 `init` 构造器以及 `str/repr` 显示格式替代方法。

代码分析

```
>>> class Callee:
...     def __call__(self, *pargs, **kargs):
...         print(f'Called: {pargs=} {kargs=}')
>>> C = Callee()
>>> C(1, 2, 3, x=4, y=5)      # C ""
Called: pargs=(1, 2, 3) kargs={'x': 4, 'y': 5}
```

·解释器如何处理：`C(1, 2, 3, x=4, y=5)` 编译为 `CALL` (0 参数位置指令)，虚拟机取 `type(C).tpcall` 槽位——找到 `call`，把它作为“可调用对象”执行，栈上的参数先被整理成 `(1, 2, 3)` 元组与 `{'x':4,'y':5}` 字典，再按签名绑定。第 18 章的全部传参语法（解包、解包、混合）都在 `CALL` 指令的实参整理阶段统一处理，与普通函数无差别。

·与“对象是否是函数”的关系：`callable(C)` 返回 `True` 的判断依据就是 `tpcall` 是否存在——所以 `call` 让实例成为一等“可调用对象” (`callable`) 。

·设计思想：`Prod` 对比 `comp` 方法的关键差异在调用方——`x(3)` 与任何函数接口无缝衔接（回调、`map` 函数、API 的 `handler=` 参数），而 `x.comp(3)` 要求调用方知道方法名。接口对齐是 `call` 的存在意义。

深度理解

·核心概念：`call` 让“实例可以像函数一样被调用”，同时保留对象的身份与状态——这就是有状态函数 (`stateful function`) / 函数对象 (`functor`) 模式。

- 底层实现：tpcall 槽位；CALL 指令把实例作为函数对象压栈执行；self 由调用绑定自动注入。callable() 即检查此槽位。
- 为什么这样设计：很多库/框架的钩子只接受"可调用" (callback、handler、key 函数)；普通方法没法注册进去，call 实例能。它是"接口适配器"而非新语法。
- 实际问题：GUI 事件回调 (tkinter command=)、sorted(key=...) 的有状态排序键、函数装饰器 (第 32/39 章的核心实现手法)、策略模式的算法对象——都是 call 的主场。
- 初学者误区：①把 init 的调用 (C()) 与 call 混淆——C() 走元类 type.call, c() (小写实例) 才走 call；②以为 call 只能接受固定参数——args/kwargs 全套可用；③忘了 call 存在却还在回调场景里写"方法+lambda 包一层"的绕路代码。

30.11.1 函数接口与基于回调的代码 (Function Interfaces and Callback-Based Code)

英文原文

As an example, Python's tkinter GUI module lets you register functions as event handlers (a.k.a. callbacks) —when events occur, tkinter calls the registered objects. If you want an event handler to retain state between events, you can register a class instance that conforms to the expected interface with call. Chapter 17's closure functions can achieve similar effects but do

n't provide as much support for multiple operations or customization. To demo the concept, here's a hypothetical example of call applied to the GUI domain. The following class defines an object that supports a function-call interface but also has state information that remembers the color a button should change to when it is later pressed:

```
class Callback: def init(self, color): # Function + state
information self.color = color def call(self): # Support
calls with no arguments print('turn', self.color) In the
context of a GUI, we can register instances of this cla
ss as event handlers for buttons, even though the GU
I expects to be able to invoke event handlers as simpl
e functions with no arguments:
```

```
cb1 = Callback('blue') # Remember blue cb2 = Callb
ack('green') # Remember green B1 = Button(comman
d=cb1) # Register handlers B2 = Button(command=c
b2) When the button is later pressed, the instance ob
ject is called as a simple function with no arguments,
exactly like in the following calls. Because it retains st
ate as instance attributes, though, it remembers what
to do—it becomes a stateful function object:
```

```
cb1() # On event: prints'turn blue'cb2() # On event:
prints'turn green'
```

In fact, some consider such classes to be the best way to retain state information in the Python language. With OOP, the state remembered is made explicit wit

h attribute assignments. This is different than other state retention techniques (e.g., global variables, enclosing function scope references, and default mutable arguments), which rely on more limited or implicit behavior. Moreover, the added structure and customization in classes goes beyond state retention. On the other hand, tools such as closure functions are useful in basic state retention roles too, and the nonlocal statement makes enclosing scopes a viable alternative in more programs. We'll revisit such trade-offs when we start coding substantial decorators in Chapter 39, but here's a quick closure equivalent:

```
def callback(color): # Enclosing scope versus attrs def oncall(): print('turn', color) return oncall cb3 = callback('yellow') # Handler to be registered cb3() # On event: prints'turn yellow'
```

Before we move on, there are two other ways that Python programmers sometimes tie information to a callback function like this. One option is to use default arguments in lambda functions:

```
cb4 = (lambda color='red': # Defaults retain state to print('turn', color)) cb4() # On event: prints'turn red'
```

The other is to use bound methods of a class—covered in the next chapter but simple enough to preview here. A bound-method object is created for instance methods referenced but not called and remembers b

oth the instance and the referenced function. This object may therefore be called later as a simple function without an instance:

```
class Callback: def init(self, color): # Class with state information self.color = color def changeColor(self): # A normally named method print('turn', self.color) cb1 = Callback('blue') cb2 = Callback('yellow') B1 = Button(command=cb1.changeColor) # Bound method: reference, not call B2 = Button(command=cb2.changeColor) # Remembers instance + function pair In this case, when this button is later pressed it's as if the GUI does the following, which invokes the instance's changeColor method to process the cb1 object's state information instead of calling the instance itself:
```

```
cb1 = Callback('blue') cmd = cb1.changeColor # Registered event handler cmd() # On event: prints'turn blue'
```

Note that a lambda is not required here unless extra arguments must be passed, because a bound method reference by itself already defers a call until later. This technique doesn't require overloading calls with call, but either scheme may be preferred for a given program. Again, watch for more about bound methods in the next chapter. You'll also see another call example in Chapter 32, where we will use it to implement a function decorator—a callable object often used to add a layer of logic on top of an embedded function.

Because call allows us to attach state information to a callable object, it's a natural implementation technique for a function that must remember to call another function when called itself. For more call examples, also watch for the more advanced decorators and metaclasses of Chapters 39 and 40.

中文翻译

举个例子：Python 的 tkinter GUI 模块允许你把函数注册为事件处理器（又称回调，callback）——当事件发生时，tkinter 调用注册好的对象。如果你希望事件处理器在事件之间保留状态，就可以注册一个用 call 符合预期接口的类实例。第 17 章的闭包函数能达到类似效果，但对多操作或定制化的支持没那么强。为了演示这个概念，这里有一个把 call 应用于 GUI 领域的假设性示例。下面的类定义了一个对象：它支持函数调用接口，同时带有状态信息——记住按钮日后被按下时应变成什么颜色：

```
class Callback:
    def __init__(self, color): # 函数 + 状态信息
        self.color = color
    def call(self): # 支持无参调用
        print('turn', self.color)
```

在 GUI 的语境里，我们可以把这个类的实例注册为按钮的事件处理器——尽管 GUI 期望事件处理器是能无参调用的普通函数：

```
cb1 = Callback('blue') # 记住 blue
cb2 = Callback('green') # 记住 green
B1 = Button(command=cb1) # 注册处理器
B2 = Button(command=cb2)
```

之后按钮被按下时，实例对象作为无参的普通函数被调用——与下面的调用一模一样。不过，因为它把状态保留在实例属性中，它记得自己该做什么——它成了一个有状态函数对象（stateful function object）：

```
cb1() # 事件发生时：打印'turn bl
```

ue'cb2() # 事件发生时：打印'turn green'事实上，有人把这类类视为 Python 中保留状态信息的最佳方式。借助 OOP，被记住的状态通过属性赋值被显式地表达出来。这与其他的状态保留技巧（例如全局变量、外围函数作用域引用、默认可变参数）不同——那些依赖更有限或更隐式的行为。而且，类带来的额外结构与定制化能力，超出了状态保留本身。另一方面，闭包函数这类工具在基本状态保留角色上同样有用，nonlocal 语句也让外围作用域在更多程序中成为可行选择。我们在第 39 章编写真正的装饰器时会重新审视这些取舍，这里先给一个快速的闭包等价写法：

```
def callback(color): # 外围作用域 vs 属性
def oncall(): print('turn', color)
return oncall
cb3 = callback('yellow') # 待注册的处理器
cb3() # 事件发生时：打印'turn yellow'继续之前还要提一下：Python 程序员有时还会用另外两种方式把信息绑到这样的回调函数上。一种是在 lambda 函数里用默认参数：
cb4 = (lambda color='red': # 默认值也保留状态
print('turn', color))
cb4() # 事件发生时：打印'turn red'另一种是使用类的绑定方法（bound method）——下一章会讲到，但这里足以简单预览。实例方法被引用而未调用时，会创建一个绑定方法对象，它同时记住实例与被引用的函数。因此这个对象日后可以像一个不带实例的普通函数那样被调用：
class Callback:
def init(self, color): # 带状态信息的类
self.color = color
def changeColor(self): # 普通命名的方法
print('turn', self.color)
cb1 = Callback('blue')
cb2 = Callback('yellow')
B1 = Button(command=cb1.changeColor) # 绑定方法：引用而非调用
B2 = Button(command=cb2.changeColor) # 记住实例 + 函数
```

这一对这种情况下，当按钮日后被按下，就相当于 GUI 执行

下面的代码——它调用实例的 `changeColor` 方法去处理 `cb1` 对象的状态信息，而不是调用实例本身：`cb1 = Callback('blue')` `cmd = cb1.changeColor` # 注册的事件处理器 `cmd()` # 事件发生时：打印'turn blue'注意：除非必须传递额外参数，否则这里并不需要 `lambda`——因为绑定方法引用本身就已经把调用推迟到日后。这个技巧不需要用 `call` 重载调用，但某个程序用哪种方案都行。再次提醒：下一章会详细讲绑定方法。你还会在第 32 章看到另一个 `call` 示例——那里我们用它实现函数装饰器（function decorator）：一个可调用对象，常用来在内嵌函数之上添加一层逻辑。因为 `call` 允许我们给可调用对象附加状态信息，它天然适合实现“自己被调用时必须记住再去调用另一个函数”的函数。更多 `call` 示例，请留意第 39、40 章更高级的装饰器和元类。

代码分析

```
class Callback:
    def __init__(self, color):      # +
        self.color = color
    def __call__(self):            #
        print('turn', self.color)

cb1 = Callback('blue')            # blue
cb2 = Callback('green')           # green
B1 = Button(command=cb1)          #
B2 = Button(command=cb2)

cb1()                             # 'turn blue'
```

·解释器如何处理：`Button(command=cb1)` 把实例对象存进按钮内部（参数是引用传递，不调用任何方法）。按

按钮按下时，tkinter 内部执行 `self.command()`——CALL 指令发现 `cb1` 有 `tpcall`，于是调用 `cb1.call()`，方法体读取 `self.color` 输出。状态在构造时注入、调用时消费——对象把“数据（color）”与“行为（打印）”打包成一体。

·四种“函数+状态”方案的对照：

方案	状态存哪	显式性	多操作/定制
call 实例	实例属性	最显式	最强（还能加方法、继承）
闭包函数	外围作用域（cell 变量）	隐式	单一操作
lambda 默认参	默认实参	隐式	最弱
绑定方法 <code>cb.changeColor</code>	实例 + 函数对	显式	中等（必须有方法名）

·设计思想：tkinter 这类框架只认“可调用对象”这一接口；四种方案都能填进 `command=` 槽位。选型的本质是“状态放哪、显式还是隐式”的权衡——类属性最利于调试与扩展，闭包最省代码。

深度理解

·核心概念：回调（callback）场景 = “注册一个可调用对象，事件到来时被框架调用”。call 实例是其中的“重型”方案，胜在状态显式、可继承、可多方法。

·底层实现：绑定方法对象（bound method）在 C 层是 `method` 类型，内部持 `self`（实例）与 `func`（函数），调用时自动注入实例参数——这是 `tpcall` 的又一种实现者。

·为什么这样设计：Python 的“可调用”是一个宽泛协议（函数、生成器、类、call 实例、绑定方法、`functools.par`

tial 对象皆可)；框架只依赖协议，不依赖具体类型——这让回调风格代码高度灵活。

·实际问题：GUI 事件、定时器回调、异步任务 (asyncio 的 partial)、tkinter/pytest fixture 注册、sorted(key=)——处处是"函数接口"的接口 (interfaces that expect functions)。

·初学者误区：①command=cb1 写成 command=cb1()——后者在注册时就执行了 (返回 None 注册了个寂寞)；②以为绑定方法"复制了函数"——它是"实例+函数"的引用对，实例被替换后行为随之改变；③把 call 与"构造函数调用"混在一起调试，分不清 C 与 c 的大小写语义。

30.12 比较运算：lt、gt 等 (Comparisons)

英文原文

Our next batch of overloading methods supports object comparisons. As suggested in Table 30-1, classes can define methods to catch all six comparison operators: <, >, <=, >=, ==, and !=. These methods are generally straightforward to use, but keep the following qualifications in mind: - Unlike the add/radd pairings discussed earlier, there are no right-side variants of comparison methods. Instead, reflective methods are used when only one operand supports comparison (e.g., lt and gt are each other's reflection). - There are no implicit relationships among the comparison opera

tors. The truth of `==` does not imply that `!=` is false, for example, so both `eq` and `ne` should be defined to ensure that both operators behave correctly.

We don't have space for an in-depth exploration of comparison methods, but as a quick introduction, consider the following class and tests:

```
>> class Vetter:
... data = 'hack' ... def gt(self, other): ... print(f'gt: {self=}
... {other=}') ... return self.data > other ... def lt(self, other): ... print(f'lt: {self=} {other=}') ... return self.data <
other >> X = Vetter() >> X > 'code', X < 'code' gt: self=
<main.Vetter object at 0x10898f650> other='code' lt: self=
<main.Vetter object at 0x10898f650> other='code' (True, False) >> 'code' < X, 'code' > X gt: self=
<main.Vetter object at 0x10898f650> other='code' lt: self=
<main.Vetter object at 0x10898f650> other='code' (True, False) >> X < X lt: self=
<main.Vetter object at 0x10898f650> other=
<main.Vetter ... etc ...> gt: self=
<main.Vetter object at 0x10898f650> other='hack' False
```

When run, the class's methods intercept and implement comparison expressions as noted by their trace outputs. Importantly, `lt` is also used for sorts—both the `list` method and built-in function:

```
>> class Order:
... def init(self, data): ... self.data = data ... def lt(self, o
```



```
ther): ... return self.data < other.data ... def repr(self):  
... return f'Order({self.data})'>>> sorted(Order(i) for i in  
[3, 1, 4, 2]) [Order(1), Order(2), Order(3), Order(4)]
```

Consult Python's manuals for more details in this category. As you'll find there, the `eq` method run for value equality is coupled with the `hash` method run for as-key and set-object roles (in ways that should send most readers screaming into the night); and comparison methods can also return `NotImplemented` for unsupported arguments (but again, not `NotImplementedError`, an exception with a similar name but very different roles).

中文翻译

我们的下一批重载方法支持对象比较。如表 30-1 所示，类可以定义方法捕获全部六个比较运算符：`<`、`>`、`<=`、`>=`、`==` 和 `!=`。这些方法用起来一般直截了当，但请记住以下限定：- 与前面讨论的 `add/radd` 配对不同，比较方法没有右端变体。取而代之的是，当只有一方支持比较时，会使用反射方法（例如 `lt` 与 `gt` 互为对方的反射）。- 比较运算符之间没有隐含关系。例如，`==` 为真并不自动意味着 `!=` 为假——所以 `eq` 和 `ne` 都应该定义，才能确保两个运算符行为都正确。我们没有篇幅深入探索比较方法，但作为快速入门，请看下面的类和测试：

```
>>> class Vetter: ... data  
='hack'... def gt(self, other): ... print(f'gt: {self=} {other=  
=}') ... return self.data > other ... def lt(self, other): ...  
print(f'lt: {self=} {other=}') ... return self.data < other  
>>> X = Vetter() >>> X > 'code', X < 'code'gt: self=<
```

```
main.Vetter object at 0x10898f650> other='code'lt: s
elf=<main.Vetter object at 0x10898f650> other='cod
e'(True, False) >>>'code' < X,'code' > X gt: self=<ma
in.Vetter object at 0x10898f650> other='code'lt: self
=<main.Vetter object at 0x10898f650> other='code'(
True, False) >>> X < X lt: self=<main.Vetter object at
0x10898f650> other=<main.Vetter ...等...gt: self=<m
ain.Vetter object at 0x10898f650> other='hack'False
```

运行时，类的这些方法按追踪输出所示拦截并实现比较表达式。重要的是，`lt` 也用于排序——无论是列表方法还是内建函数：

```
>>> class Order: ... def init(self, data): ... self.d
ata = data ... def lt(self, other): ... return self.data < ot
her.data ... def repr(self): ... return f'Order({self.data})'
>>> sorted(Order(i) for i in [3, 1, 4, 2]) [Order(1), Ord
er(2), Order(3), Order(4)]
```

这一类别更多的细节请查阅 Python 手册。你会在那里发现：用于值相等（value equality）的 `eq` 方法与用于“作为字典键与集合元素”角色的 `hash` 方法相互耦合（其复杂程度足以让大多数读者半夜尖叫）；比较方法对不支持的参数也可以返回 `NotImplemented`（但同样地，不是 `NotImplementedError`——那个名字相似但角色截然不同的异常）。

代码分析

```
>>> class Vetter:
...     data = 'hack'
...     def __gt__(self, other):
...         print(f'gt: {self=} {other=}')
...         return self.data > other
...     def __lt__(self, other):
...         print(f'lt: {self=} {other=}')
...         return self.data < other
>>> X = Vetter()
>>> 'code' < X          # str.__lt__ X.__gt__('code')
gt: self=<Vetter object> other='code'
True
```

·解释器如何处理：'code' < X 编译为 COMPAREOP <。左操作数 str 的 lt 与 X 比较时返回 NotImplemented，解释器于是反射到右操作数：X.gt('code') (gt 是 lt 的反射，等于 X.lt(other) 的"镜像")。这就是为什么"'code' < X"打印的是 gt:——调用的是 gt 而非 lt。同理'code' > X 会调用 X.lt('code')。

·X < X 的连环追踪：左 X.lt(X) 里 self.data > other 是'hack' > X——str 拒绝 → 反射到 X.gt('hack')，打印 gt:；于是最终结果是 False ('hack' > 'hack' 为假)。

·排序的秘密：sorted()/list.sort() 的 TimSort 排序算法只用 lt 做元素比较 (C 层 PyObjectRichCompareBool 的 < 分支) ——所以只需 lt + eq 就能让对象可排序，gt 等其余四个方法对排序无用武之地。

深度理解

·核心概念：六比较算子的分派是按对反射的：</>、<=/>=、==/!= 互为镜像；且不存在"定义了 < 就自动有 >"

的推导关系——每个都得自己定义（或用 `functools.total_ordering` 装饰器补全）。

·底层实现：COMPAREOP 指令 → `PyObjectRichCompare` → 左操作数的富比较槽位，返回 `NotImplemented` 时反射到右操作数；`sorted` 的 C 实现只请求 `<` 结果。

·为什么这样设计：六运算符若靠推导 (`a > b` not (`a <= b`)) 会引入三值逻辑与 `NotImplemented` 的纠缠，语言选择"显式定义 + 反射兜底"以保持语义简单可靠。

·实际问题：`eq` 与 `hash` 的耦合规则（可变对象不该可哈希；定义 `eq` 后默认 `hash` 被置为 `None`）是面试高频题；`functools.total_ordering` 用 `lt/eq` 自动生成其余四个。

·初学者误区：①定义了 `lt` 就以为 `>` 自动可用——`X > Y` 时反射到 `gt`，没定义就 `TypeError`；②只定义 `eq` 却不定义 `ne`，`!=` 行为退化（Python 3 中 `!=` 默认取 `eq` 的取反——注意这是 3.x 的默认回退，与 Lutz 强调的"别依赖隐含关系"并不冲突，显式定义更稳）；③对象变可哈希后用作字典键，状态一变键就找不到——值对象要按"不可变 + `hash` + `eq`"三位一体设计。

30.13 布尔测试：bool 和 len (Boolean Tests)

英文原文

The next set of methods is truly useful (pun intended). As you've learned, every object is inherently true or

false in Python. When you code classes, you can define what this means for your objects by coding methods that give the True or False values of instances on request.

In Boolean contexts, Python first tries `bool` to obtain a direct Boolean value; if that method is missing, Python tries `len` to infer a truth value from the object's length—a length of zero means empty, which is always false. The first of these generally uses object state or other information to produce a Boolean result:

```
>> class Truth:
... def bool(self): return True >> X = Truth() >> if X: p
rint('yes!') yes!
```

```
>> class Truth:
... def bool(self): return False >> X = Truth() >> bool(
X) False
```

If this method is missing, Python falls back on `length` because a nonempty object is considered true. That is, a nonzero length is taken to mean the object is true, and a zero length means it is false—just as for built-in objects:

```
>> class Truth:
... def len(self): return 0 >> X = Truth() # Empty means
false too >> if not X: print('no!') no!
```

If both methods are present Python prefers `bool` over

r len, because it is more specific:

```
>> class Truth:
... def bool(self): return True # Preferred over length ..
. def len(self): return 0 # Object length: fallback >> if
Truth(): print('yes!') yes!
```

If neither truth method is defined, the object is vacuously considered true (though any existential implications of this are strictly out of scope here):

```
>> class Truth:
... pass >> X = Truth() >> bool(X) True
```

中文翻译

接下来的一组方法真的很有用（pun intended：这里既指"有用"也指"真值"的谐音双关）。正如你学过的，Python 中每个对象生来就为真或为假。编写类时，你可以通过编码方法，在请求时给出实例的 True 或 False 值，从而定义这对你的对象意味着什么。在布尔上下文中，Python 首先尝试 bool 获取直接的布尔值；如果缺少该方法，Python 尝试 len，从对象的长度推断真值——长度为 0 表示空，永远为假。前者一般利用对象状态或其他信息产生布尔结果：

```
>>
> class Truth: ... def bool(self): return True >>> X = T
ruth() >>> if X: print('yes!') yes! >>> class Truth: ... d
ef bool(self): return False >>> X = Truth() >>> bool(
X) False
```

如果缺少这个方法，Python 回退到长度，因为非空对象被视为真。也就是说，非零长度意味着对象为真，零

长度意味着为假——与内建对象一致：>>> class Truth: .. def len(self): return 0 >>> X = Truth() # 空也意味着假>>> if not X: print('no!') no! 如果两个方法都在，Python 优先用 bool 而不是 len，因为它更专门：>>> class Truth: ... def bool(self): return True # 优先于长度... def len(self): return 0 # 对象长度：回退方案>>> if Truth(): print('yes!') yes! 如果两个真值方法都没定义，对象会被空泛地（vacuously）视为真（至于这在存在论上意味着什么，严格来说不在我们的讨论范围内）：>>> class Truth: .. pass >>> X = Truth() >>> bool(X) True

代码分析

```
>>> class Truth:
...     def __len__(self): return 0    #
>>> X = Truth()
>>> if not X: print('no!')
no!
```

·解释器如何处理：if not X 编译为 POPJUMPIFFALSE（3.12 为真值判断指令），虚拟机调用 PyObjectIsTrue(X)：先查 bool——没有；再查 len——有，调用得到 0 → 判定为假。整条链是"bool → len → 默认真"的三级回退。

·优先级矩阵（需记牢）：bool 存在 → 用其返回值（必须是 bool 或可转 bool 的对象）；否则 len 存在 → len != 0 即真；否则永远为真（这是自定义类的默认，与内建 object 一致）。

·用途：容器类（队列、缓冲、缓存）用 len 表达"空即假"——if queue: 判断"还有没有东西"，比 if len(queue) >

0: 更地道; bool 适合"非空但要看状态"的对象(如传感器读数、连接对象)。

深度理解

·核心概念: Python 的真值不是"非空即真"一刀切——它是可编程的: bool 直接给结论, len 用"长度语义"间接给结论。

·底层实现: PyObjectIsTrue 的 C 路径正是上述三级回退; bool 的返回值会经 bool() 归一化。注意 len 返回负数会抛 ValueError, 返回非整数会被强转。

·为什么这样设计: 长度是真值的最自然代理(容器哲学: 空即假), bool 则允许"状态决定真值"(如 if connection.isalive 逻辑), 分层设计兼顾两种直觉。

·实际问题: if not results: (空结果集即假)、while queue: (队列非空持续处理)、pandas 里 if df.empty (DataFrame 的 bool 会抛异常防止歧义——这是"布尔上下文不可用时显式报错"的经典案例)。

·初学者误区: ①以为"定义了 len 返回非零, 对象就为真"——对, 但反之"长度为 0 为假"也让某些"有属性但长度 0"的对象莫名变假; ②bool 里返回非 bool (如整数 1) 虽能工作但风格糟糕; ③在 bool 里做重活(如数据库查询)——if obj: 可能被隐式触发无数次, 性能杀手; ④以为 bool(obj) 与 obj is True 等价——后者是身份比较, 风马牛不相及。

30.14 对象销毁: del (Object Destruction)

英文原文

It's time to close out this chapter—and learn how to do the same for our class objects. You've seen how the `__init__` constructor is called whenever an instance is generated (and noted how `__new__` is run first to make the object). Its counterpart, the destructor (less commonly known as finalizer) method `__del__`, is run automatically when an instance's space is being reclaimed (i.e., at garbage-collection time):

```
>>> class Life: ... def __init__(self, name): ... print('Hello', name) ... self.name = name ...
def __live__(self): ... print(self.name + '...') ... def __del__(self): ...
print('Goodbye', self.name) >>> pat = Life('Pat') Hello Pat >>> pat.__live__() Pat... >>> pat = 'end'Goodbye Pat
```

Here, when `pat` is assigned a string at the end, we lose the last reference to the `Life` instance and so trigger its destructor method. This works, and it may be useful for implementing some cleanup activities, such as terminating a server connection. However, destructors are not as commonly used in Python as in some OOP languages, for a number of reasons that the next section describes.

中文翻译

是时候结束本章了——并且学习如何为我们的类对象“送终”。你已经看到：每当生成一个实例时 `__init__` 构造器就会被调

用（也注意到 new 先运行来造出对象）。与它相对的析构器（destructor，不太常见地称为终结器 finalizer）方法 del，在实例的空间被回收时（即垃圾回收时刻）自动运行：

```
>>> class Life: ... def init(self, name): ... print('Hello', name) ... self.name = name ... def live(self): ... print(self.name + '...') ... def del(self): ... print('Goodbye', self.name) >>> pat = Life('Pat') Hello Pat >>> pat.live() Pat.. >>> pat = 'end' Goodbye Pat
```

在这里，当结尾处 pat 被赋值为一个字符串时，我们失去了 Life 实例的最后一个引用，从而触发它的析构器方法。这是可行的，而且对实现某些清理活动（比如终止服务器连接）可能有用。然而，析构器在 Python 中并不像在有些 OOP 语言里那样常用，原因有若干——下一节会描述。

代码分析

```
>>> class Life:
...     def __init__(self, name):
...         print('Hello', name)
...         self.name = name
...     def live(self):
...         print(self.name + '...')
...     def __del__(self):
...         print('Goodbye', self.name)
>>> pat = Life('Pat')           # __init__ Hello Pat
>>> pat = 'end'                 # → __del__
Goodbye Pat
```

·解释器如何处理：CPython 用引用计数管理对象生命周期。pat = 'end' 执行时，Life 实例的引用计数从 1 降为 0——PyDECREf 发现归零，立即调用该类型的 tpfinalize（del）并释放内存。整个过程确定且即时（这是 CPython

n 与其他 GC 语言的重大差异)。

·触发时机提示：引用计数归零的时机通常可预测（离开作用域、重绑定、del pat、容器移除元素），但不保证——循环引用（AB 互相引用）时，计数永远不为零，要等分代循环垃圾收集器（gc 模块）介入（3.4+ 连带 del 的对象也能被回收）。

·最佳实践：资源清理（关文件、断连接、释放锁）应优先用 with 语句（enter/exit，第 34 章）或 try/finally，而不是依赖 del——因为后者时机不可控。

30.14.1 析构器使用注意 (Destructor Usage Notes)

英文原文

The destructor method works as documented, but it has some well-known caveats and a few outright dark corners that make it somewhat rare to see in Python code: Need — For one thing, destructors may not be as useful in Python as they are in some other OOP languages. Because Python automatically reclaims all memory space held by an instance when the instance is reclaimed, destructors are not necessary for space management. In the current CPython implementation of Python, you also don't need to close file objects held by the instance in destructors because they are automatically closed when reclaimed. As mentioned in Chapter 9, though, it's still sometimes best to run file

close methods anyhow because this autoclose behavior may vary in alternative Python implementations.

Predictability — For another, you cannot always easily predict when an instance will be reclaimed. In some cases, there may be lingering references to your objects in system tables that prevent destructors from running when your program expects them to be triggered. Python also does not guarantee that destructor methods will be called for objects that still exist when the interpreter exits.

Exceptions — In fact, `del` can be tricky to use for even more subtle reasons. Exceptions raised within it, for example, simply print a warning message to `sys.stderr` (the standard error stream) rather than triggering an exception event, because of the unpredictable context under which it is run by the garbage collector—it's not always possible to know where such an exception should be delivered.

Cycles — In addition, cyclic (a.k.a. circular) references among objects may prevent garbage collection from happening when you expect it to. An optional cycle detector, enabled by default, can automatically collect such objects eventually (including objects with `del` methods, as of Python 3.4). Since this is relatively obscure, we'll ignore further details here; see Python's standard manuals' coverage of both `del` and the `gc` garbage collector module for more information. Because of these downsides, it's often better to code termination activities in an explicitly called method (e.g., `shutdown`).

As described in the next part of the book, the try/finally statement also supports termination actions, as does the with statement for objects that support its context-manager model.

中文翻译

析构器方法按文档工作，但它有一些众所周知的注意事项和几处不折不扣的黑暗角落，这让它在 Python 代码中相当少见：必要性（Need）——首先，析构器在 Python 中可能不像在其他一些 OOP 语言中那么有用。因为 Python 在实例被回收时会自动回收实例占用的所有内存空间，析构器对空间管理并不是必需的。在当前的 CPython 实现中，你也不必在析构器里关闭实例持有的文件对象——它们在回收时会自动关闭。不过，如第 9 章所述，最好还是显式运行文件关闭方法，因为这种自动关闭行为在其他 Python 实现中可能不同。可预测性（Predictability）——其次，你并不总能轻松预测实例何时被回收。有些情况下，系统表里可能残留着对你的对象的引用，从而在你期望触发析构器时阻止它运行。Python 也不保证：解释器退出时仍然存在的对象的析构器方法会被调用。异常（Exceptions）——事实上，由于更微妙的原因，del 甚至可能很难用。例如，在它内部抛出的异常只会把警告信息打印到 sys.stderr（标准错误流），而不会触发异常事件——因为垃圾收集器运行它的上下文不可预测，并非总能知道这样的异常应该送往哪里。循环引用（Cycles）——此外，对象之间的循环（又称环形）引用可能会在你想让它发生时阻止垃圾回收。一个可选的循环检测器（默认启用）最终会自动收集这类对象（自 Python 3.4 起，包括带 del 方法的对象）。由于这相对冷

门，我们就不深入细节了；更多信息请参见 Python 标准手册中关于 del 与 gc 垃圾收集器模块的覆盖。因为有这些缺点，把终止活动写进一个显式调用的方法（例如 shutdown）通常是更好的选择。正如本书下一部分所述，try/finally 语句也支持终止动作；对支持上下文管理器模型的对象，with 语句同样如此。

深度理解

- 核心概念：del 的四大坑——非必需（内存自动回收）、不可预测（时机不定、退出不保证）、异常哑火（只能打印到 stderr）、循环引用（可能根本等不来）。结论：Python 世界把"析构"降级为"可选锦上添花"，正牌清理用 with/try-finally。

- 底层实现：CPython 引用计数 + 分代 GC 双轨制：计数归零立即 del；循环垃圾由 gc 模块在"代阈值"触发时回收；del 里抛异常 → PyErrWriteUnraisable 打到 stderr（解释器没法把它路由给谁）。Jython/IronPython 等无引用计数的实现，时机更不可控。

- 为什么这样设计：引用计数保证"即时回收"（内存好管理），但终结器（finalizer）一旦参与就引入不确定性与死锁风险（如 del 里又访问全局状态）——所以语言层刻意"半支持"它。

- 实际问题：数据库连接、socket、文件句柄的关闭绝不依赖 del；现代写法是 with open(...) as f:（自动 exit）或显式 conn.close()/shutdown()。weakref.finalize 是官方推荐的"有秩序终结"替代品。

·初学者误区：①以为 `del x` 立刻调用 `del`——`del` 只是删除名字（减引用），其他引用还存在就不触发；②在 `del` 里 `raise` 期待异常被捕获——它只会悄悄打印警告；③把 `del` 当“资源释放的保险”——循环引用或解释器退出时它可能根本不执行。

30.15 章节小结 (Chapter Summary)

英文原文

That's as many overloading examples as we have space for here. Most of the other operator-overloading methods work similarly to the ones we've explored, and all are just hooks for intercepting built-in type operations. Some overloading methods, for example, have unique argument lists or return values, but the general usage pattern is the same. You'll see a few others in action later in the book:— Chapter 34 uses `enter` and `exit` in with statement context managers.— Chapter 38 uses the `get` and `set` class descriptor `fetch/set` methods.— Chapter 40 uses the new object creation method in the context of metaclasses. In addition, some of the methods we've studied here, such as `call` and `str`, will be employed by later examples in this book. For complete coverage, though, it must defer to other documentation sources—see Python's language manual or other reference resources for details on additional overloading methods. In the next chapter, we

leave the realm of class mechanics behind to explore design—the ways that classes are commonly used and combined to optimize code reuse. After that, we'll survey a gumbo of advanced class topics and move on to exceptions, the last core subject of this book. Before you read on, though, take a moment to work through the chapter quiz below to review the concepts we've covered here.

中文翻译

这是我们在篇幅允许范围内能展示的全部重载示例了。大多数其他运算符重载方法与我们已经探索的这些工作方式类似，而且它们都只是拦截内建类型操作的钩子。例如，有些重载方法有独特的参数列表或返回值，但总体使用模式相同。你会在本书后面看到另外几个方法登场：- 第 34 章在 with 语句上下文管理器（context manager）中使用 enter 和 exit。- 第 38 章使用 get 和 set 类描述符获取/设置方法。- 第 40 章在元类语境中使用 new 对象创建方法。此外，我们在这里学过的一些方法——比如 call 和 str——也会被本书后面的例子采用。不过要获得完整覆盖，还得求助于其他文档来源——更多重载方法的细节请参见 Python 语言手册或其他参考资料。在下一章，我们将告别类机制领域，去探索设计——类被普遍使用与组合以优化代码复用的方式。之后，我们会浏览一大堆高级类话题，然后转向异常——本书的最后一个核心主题。不过在你继续读下去之前，请花点时间完成下面的随章测验，复习我们这里覆盖的概念。

深度理解

·核心概念：本章的"钩子全集"是后续章节的地基——with 语句（第 34 章）、描述符（第 38 章）、元类（第 40 章）将复用本章建立的"特殊方法 = 协议"心智模型。

·设计思想：运算符重载的本质是统一接口——语言用一套预定义名字与内建运算绑定，类按协议实现即可无缝融入（打印、排序、迭代、真值测试.....）。"多数方法模式相同"意味着：学会一个，就会一类。

·学习策略：不必死记每个方法名——需要时查手册（dir(object)、help()、语言参考的 "special method names" 一节），但要记住分派优先级链（contains> iter > getitem; str 优先于 repr; iadd 优先于 add 等），这是设计与调试的核心。

30.16 自测：Quiz（随章测验）

测验题目（中英对照）

1. What two operator-overloading methods can you use to support iteration in your classes?你的类可以用哪两个运算符重载方法支持迭代？
2. What two operator-overloading methods handle printing, and in what contexts?哪两个运算符重载方法处理打印，分别用于什么场景？
3. How can you intercept slice operations in a class?在类中如何拦截切片操作？
4. How can you catch in-place addition in a class?在类中如何捕获就地加法？
5. W

When should you provide operator overloading?什么时候应该提供运算符重载?

参考答案

1. Classes can support iteration by defining (or inheriting) `getitem` or `iter`. In all iteration tools, Python tries to use `iter` first, which returns an object that supports the iteration protocol with a `next` method: if no `iter` is found by inheritance search, Python falls back on the `getitem` indexing method, which is called repeatedly, with successively higher indexes. If used, the `yield` statement can create the `next` method automatically. 类可以通过定义（或继承）`getitem` 或 `iter` 来支持迭代。在所有迭代工具中，Python 优先尝试 `iter`——它返回一个带 `next` 方法、支持迭代协议的对象；如果继承搜索找不到 `iter`，Python 就回退到 `getitem` 索引方法，用不断递增的索引反复调用它。如果使用 `yield` 语句，可以自动创建 `next` 方法。

2. The `str` and `repr` methods implement object print displays. The former is called by the `print` and `str` built-in functions; the latter is called by `print` and `str` if there is no `str`, and always by the `repr` built-in, interactive echoes, and nested appearances. That is, `repr` is used everywhere, except by `print` and `str` when a `str` is defined. A `str` is usually used for user-friendly displays; `repr` gives extra details or the object's as-code form. String formatting runs these methods too. `str` 与 `repr` 方法实现对象的打印显示。前者由 `print` 和 `str` 内置函数调用；后者在无 `str` 时由 `print` 和 `str` 调用，并且总

是被 repr 内建函数、交互式回显和嵌套出现调用。也就是说，除了 print 和 str 在定义了 str 的情况下，repr 被用于一切场合。str 通常用于用户友好的显示；repr 提供额外细节或对象的"可复制回代码"形式。字符串格式化工具也会运行这些方法。

3. Slicing is caught by the getitem indexing method: it is called with a slice object instead of a simple integer index, and slice objects may be passed on or inspected as needed.切片由 getitem 索引方法捕获：它被调用时收到的不是简单整数索引，而是一个 slice 对象；slice 对象可以按需转发或检查。

4. In-place addition tries iadd first, and add with an assignment second. The same policy holds true for all binary operators. The radd method is also available for right-side addition.就地加法优先尝试 iadd，其次是用 add 加赋值。同一策略适用于所有二元运算符。radd 方法也可用于右端加法。

5. When a class naturally matches, or needs to emulate, a built-in type's interfaces. For example, collections might imitate sequence or mapping interfaces, and callables might be coded for use with an API that expects a function. You generally shouldn't implement expression operators if they don't naturally map to your objects naturally and logically, though—use normally named methods instead.当类自然地匹配、或需要模仿某内建类型的接口时。例如，集合类可能模仿序列或映射接口，可调用类可能按"期望函数"的 API 编写。不过，如果表达式运算符不能自然、合乎逻辑地映射到你的对象上，一般就不该实现它们——这时请改用普通命名的方法。

本章总结

技术拓展 (Technical Expansion)

1. 实际项目中的应用场景

- 自定义集合类型：用 `getitem/setitem/len/iter/contains` 五件套实现"序列协议"，让自定义容器无缝接入 `len()`、`for`、`in`、切片、`sorted()`。
- 框架回调用例：GUI (`tkinter/PyQt` 的 `command=`)、异步事件循环 (`asyncio` 的 `loop.calllater`)、定时任务库 (`APScheduler`) ——`call` 实例是"有状态回调"的标准答案；配合 `functools.partial` 与绑定方法可应对所有注册场景。
- 数据类与值对象：`repr` (日志与调试) + `eq/hash` (可哈希、可比较) + `lt` (可排序) 三件套，是现代 `@dataclass` 自动生成的底层机制——读懂本章才能读懂 `dataclass` 生成器产出的魔法方法。
- 表达式构建器：ORM 与 `pandas` 的链式 API (`Table.column > 5`) 依靠 `lt/gt` 返回"待执行的表达式节点"而非布尔值——理解"比较方法可以返回任何对象"是这类设计的前提。
- 性能敏感路径：`len` 与 `getitem` 在 C 层有槽位直接映射，优先用内建函数而非手动调用特殊方法 (本章 `timeit` 实测：`len(L)` 比 `L.len()` 快一倍)。

2. 与其他语言的区别 (Java/C++)

维度	Python	Java	C++
运算符重载	任意类可重载 (add 等)	不支持	支持 (operator +) , 函数级
触发机制	解释器按特殊方法名自动分派	—	编译器静态解析 , 重载决议在编译期
右端/就地	radd/iadd 三级分派	—	只有成员/非成员函数之分, 无反射约定
析构器	del, 时机不确定、不推荐	finalize() (已弃用, 用 try-with-resources)	~T(), 确定性析构 (RAII)
真值测试	bool/len 动态真值	无内置 (需 .isEmpty())	operator bool (explicit)
打印/显示	str/repr 双轨	toString()	operator<< 流插入
相等性	eq+hash 耦合 , 可重载	equals()+hashCode() 约定	operator==

核心洞察: Java 用"接口+方法名约定" (Iterable、Comparable、toString) 模拟 Python 的协议——但方法名由开发者自选、无语言级统一; C++ 的重载在编译期决议, Python 在运行期按名字分派, 因此 Python 更灵活 (可动态补方法), 代价是少一层编译期检查。C++ 的 RAII 确定性析构是 Python del 无法承诺的, 所以 Python 用 with 语句找回确定性——这是跨语言设计权衡的绝佳标本。

3. Python 发展历史背景

·特殊方法体系源自 C 语言槽位: Python 1.x 时代, 内建类型的操作直接映射 C 结构体函数指针 (tpasnumber、tpassequence、tpasmapping); 用户类通过特殊方法

名接入同一套槽位——这是本章所有"钩子"的祖先。

- 迭代协议的演进：早期只有 `getitem`（序列迭代）；Python 2.2 引入 `iter/next` 统一迭代协议（PEP 234），`getitem` 降级为回退；Python 2.3 加入 `bool`（PEP 285，旧名 `nonzero`）；`index` 在 Python 2.5 为 `numpy` 引入（PEP 357）。

- Python 3 的变化：`str/repr` 必须返回 `str`；比较运算统一为富比较（`lt` 家族，不再有 `cmp`）；`div` 被 `truediv` 取代；`print` 成为函数后 `str` 的角色更纯粹。Lutz 的 "Measure twice" 笔记还提醒：`len()` vs `len()` 的性能差自 Python 2.X 延续至今，C 层槽位直达是内建函数快的原因。

4. 高级开发者需要掌握的相关知识

- `NotImplemented` 单例：运算符方法返回它表示"我不处理"，触发对方反射方法；`NotImplementedError` 是异常——两者绝不可混用。

- `functools.totalordering`：用 `lt+eq` 自动补全其余四个比较方法，但会带来额外分派开销，排序密集场景慎用。

- `eq` 与 `hash` 耦合规则：定义 `eq` 后默认 `hash` 被禁用；不可变对象可哈希、可变对象不可哈希；自定义哈希要保证"相等对象哈希值相等"。

- 协议分层思维：本章的"专门方法优先、通用方法兜底"（`contains`→`iter`→`getitem`）是 Python 所有协议（真值、显示、就地运算）的通用模式；设计 API 时同样适用——提供精确入口 + 合理回退。

- collections.abc 与鸭子类型：Sequence/Mapping/Iterable 抽象基类（ABC）用特殊方法名定义协议，isinstance(x, Sequence) 这类检查就是按特殊方法注册表工作。
- 装饰器与描述符的基石：第 32 章函数装饰器用 call 实现"有状态包装器"；第 38 章描述符用 get/set 把属性访问重定向——都是本章"钩子"思想的直接延伸。

学习建议 (Learning Advice)

- 重要程度：★★★★★ (5/5) 。运算符重载是 Python 面向对象的中枢：@dataclass、pandas、Django ORM、asyncio 的"魔法"全部建立在特殊方法之上。不懂本章，你将永远在"框架为什么这样工作"的迷雾里。
- 应该掌握到什么程度：
 - 必会（能默写+能讲清时机）：init/del 的调用时机（含 new 先行的构造管线）；getitem 一法三用（索引/切片/迭代回退）；iter/next 协议与 StopIteration；getattr（仅兜底）与 setattr（全拦截+递归陷阱，用 dict 或 object.setattr 绕行）与 getattribute（全拦截）三者区别；str vs repr 的分派矩阵（repr 全场景，str 只用于 print/str 且优先）；bool→len→默认真正的三级真值链；add/radd/iadd 三级分派与参数方向。
 - 应掌握：contains 的优先级与快速查找语义；call 的回调应用（与闭包、lambda 默认参、绑定方法对比）；比较方法的反射机制与 eq/hash 耦合；del 的四大缺陷与 with 替代。

- 了解：index、enter/exit、描述符与元类的特殊方法（留待第 34/38/40 章深化）。
 - 后续学习内容：
 - 立即：第 31 章"类编码细节"（绑定方法、mro、混入）——本章的 getattr 委托、绑定方法回调都会在那里深化。
 - 近程：第 32 章"扩展内建类型"（getitem 复用、call 装饰器）；第 34 章 with 与上下文管理器（enter/exit）。
 - 进阶：第 38 章描述符与 property（getattr/getattribute 的工程化延伸）；第 39 章类装饰器（属性私有性的完整方案）；第 40 章元类（new 的最终舞台）。
 - 实践建议：① 亲手重写本章所有示例并逐行追踪输出（特别是 commuter.py 的嵌套演示与 contains.py 的三种注释组合），把分派优先级内化成直觉；② 用 dis 模块查看 `x[i]`、`x += y`、`if x` 的字节码，确认"特殊方法分派"发生在指令层；③ 写一个同时实现序列协议与迭代协议的迷你类（如 Range），再对照 `collections.abc.Sequence` 源码验证协议要求；④ 在 REPL 里跑 `help()` 查看 object 的默认实现，理解"默认显示/默认真值"从何而来。
-

本章完。下一章：Chapter 31 Class Coding Details（类编码细节）